

2025年最新前端工程化面试攻略

1. [Webpack] 配置代码太多，达到数千行，这个时候该如何优化配置代码【热度: 186】

当 Webpack 配置代码变得冗长和难以管理时，可以采取以下方法来优化配置：

- 配置文件拆分

将配置文件分成多个部分，每个文件只负责一部分逻辑。比如基础配置、开发环境配置、生产环境配置等。

```
1 // webpack.base.js - 存放共同的配置项// webpack.dev.js - 开发环境特定的配置项//  
   webpack.prod.js - 生产环境特定的配置项
```

- 使用环境变量

通过环境变量来区分不同的配置环境，使用 `webpack-merge` 或 `env-cmd` 这样的库来合并配置。

```
1 // 通过环境变量确定不同的配置文件const env = process.env.NODE_ENV;  
2  
3 let config;  
4 if (env === "production") {  
5   config = require("./webpack.prod.js");  
6 } else {  
7   config = require("./webpack.dev.js");  
8 }  
9  
10 module.exports = config;
```

- 模块化配置

将常用的 loader、plugins、entry 等配置项封装成函数或者模块，然后在主配置文件中引入。

```
1 // loaders.js - 存放所有loader的配置// plugins.js - 存放所有plugins的配置//  
   entries.js - 存放入口文件配置// webpack.config.jsconst loaders =  
   require("./loaders");  
2 const plugins = require("./plugins");
```

```
3  const entries = require("./entries");
4
5  module.exports = {module: {
6    rules: loaders.getRules(),
7  },
8  plugins: plugins.getPlugins(),
9  entry: entries.getEntries(),// ...
10 };
```

- 使用 webpack-merge 抽离通用配置

检查配置中的重复部分，将它们抽象成共用的配置，再使用 `webpack-merge` 来合并多个配置文件，保持配置的清晰和可维护性。

```
1  const { merge } = require("webpack-merge");
2  const baseConfig = require("./webpack.base.js");
3  const devConfig = require("./webpack.dev.js");
4
5  module.exports = merge(baseConfig, devConfig);
```

- 统一管理插件和加载器

如果项目中使用了大量插件和加载器，请考虑将它们的实例化和配置逻辑封装在单独的函数或文件中。然后根据不同的环境，直接 pick 不同的配置即可。可以达到配置的 loader 和 plugin 集中管理。

2. [Webpack] 你用过哪些可以提高效率的插件？【热度：179】

- webpack-dashboard：可以更友好的展示相关打包信息。
- webpack-merge：提取公共配置，减少重复配置代码
- speed-measure-webpack-plugin：简称 SMP，分析出 Webpack 打包过程中 Loader 和 Plugin 的耗时，有助于找到构建过程中的性能瓶颈。
- size-plugin：监控资源体积变化，尽早发现问题
- HotModuleReplacementPlugin：模块热替换
- webpack.ProgressPlugin：打包进度分析
- webpack-bundle-analyzer：打包结果分析
- friendly-errors-webpack-plugin：代码源码编译报错友好提示

3. 在做 eslint 和 commitlint 的时候，可以使用 --no-verify 跳过，这种情况下该如何强制卡点【热度: 233】

跳过 `eslint` 和 `commitlint` 的钩子，使用 `--no-verify`（对于 `git commit` 来说是 `-n`），的确是一个容许开发者在紧急情况下超越钩子检查的手段。然而，这也削弱了代码质量保证的制度。以下是一些方法，可以用来加强这些卡点的靠谱办法：

- CI/CD 流水线中增加检查：在你的 CI/CD 流程中增加 `eslint` 和 `commitlint` 的检查步骤。如果检查失败，则阻止代码合并或部署。
- 强制挂钩：虽然开发者可能在本地禁用钩子，但你不能控制别人的本地环境。相反，你可以编写服务器端的钩子，比如在 Git 仓库的服务器上使用 `pre-receive` 钩子，来拒绝不符合规范的提交。
- 定期自动化检查：定期运行一个自动化的脚本或 GitHub Action，检查代码库的 `eslint` 与 `commitlint` 违规情况，并自动创建一个修复问题的 issue 或拉取请求。

你可以最大限度地减少绕过 `eslint` 和 `commitlint` 检查的情况。然而，值得记住的是，在极少数情况下，可能存在合法的理由需要紧急提交代码。因此，为了灵活性和效率，完全禁止 `--no-verify` 可能不是一个最佳的选择。好的实践中应该找到安全和灵活性之间的平衡，核心在于建立一个质量意识，制定明智的操作规范。

4. 如何做 commit lint 【热度: 425】

Commit lint 是一种实践，用于在代码库中规范化提交信息的格式。这种做法通常有助于团队成员理解代码库的历史记录，以及自动化生成变更日志。下面是实施 Commit lint 的一些基本步骤：

1. 选择 Commit 信息规范：首先，你需要选择一个提交信息的规范，最常见的是 [Conventional Commits](#)，它具有明确的结构和规则。
2. 配置 Linter 工具：`commitlint` 是一个流行的工具，用于检查提交信息是否符合规定的格式。安装 `commitlint`，通常是作为项目的开发依赖。

```
1 npm install --save-dev @commitlint/{config-conventional,cli}
```

1. 设置 commitlint 配置：在你的项目根目录下创建一个名为 `commitlint.config.js` 的文件，并且导入你选择的规范：

```
1 module.exports = { extends: ["@commitlint/config-conventional"] };
```

1. 安装钩子（Hook）管理工具：`Husky` 是一个钩子管理工具，它可以助你轻松的在 Git 挂钩中添加脚本（例如，在 `commit` 之前检查提交信息格式）。

```
1 npm install husky --save-dev
```

1. 配置 Husky 来使用 commitlint:

- 初始化 husky:

```
1 npx husky install
```

- 添加 `commit-msg` 钩子来使用 commitlint。执行非交互式的命令配置钩子脚本:

```
1 npx husky add .husky/commit-msg 'npx --no-install commitlint --edit "$1"'
```

1. 这行代码会在 `.husky/commit-msg` 文件中创建一个钩子，并且在你试图创建提交时，会调用 commitlint 来检查你的提交信息。
2. 提交代码：当你提交代码时，Husky 会触发 `commit-msg` 钩子调用 commitlint 检查提交信息。如果信息不符合规范，提交将被拒绝，并显示错误信息。
3. 配置 CI/CD 流水线：为了确保规范被强制执行，可以在 CI/CD 流水线中添加一步来执行 commitlint。这样，如果提交的信息不符合规范，构建将会失败。

5. 编写 npm 包的时候，可以办法自动生成 changlog 与自动更新 tag 【热度: 455】

在编写 npm 包时，可以使用自动化工具来生成 changelog 和自动更新 tag。以下是你可以使用的一些流行的工具以及它们的基本用法。

1. semantic-release: 这是一个全自动的版本管理和包发布工具。它能根据 commit 信息来自动决定版本号、生成变更日志（changelog）以及发布。
2. 要使用 semantic-release，你需要按照以下步骤操作：
 - 安装 semantic-release 工具:

```
1 npm install -D semantic-release
```

- 在项目中添加配置文件 (`semantic-release.config.js`) 或在 `package.json` 中配置
- 在 CI 工具中（例如 GitHub Actions、Travis CI）配置发布脚本。

- 遵循规范化的 commit 消息风格（如 Angular 规范），因为 semantic-release 会根据 commit 消息来确定版本号和生成 changelog。
3. standard-version: 如果你更希望进行半自动化的版本管理，standard-version 是一个很好的替代选择。它可以自动地根据 commit 记录来生成 changelog。
 4. 使用 standard-version 的大致步骤如下：
 - 安装 standard-version 工具：

```
1 npm install --save-dev standard-version
```

- 在 `package.json` 中配置脚本：

```
1  {
2    "scripts": {
3      "release": "standard-version"
4    }
5  }
```

- 当你准备发布新版本时，运行以下命令：

```
1 npm run release
```

- standard-version 会自动根据 commit 消息创建一个新的 tag，并更新 changelog。然后，你可以手动推送这些改动到仓库。

在这两种情况下，都推荐使用遵循某种规范的 commit 消息，如 Conventional Commits 规范，这样可以让工具更准确地解析 commit 消息来进行版本管理。此外，确保你的 CI/CD 系统有足够的权限来推送 tags 到远程仓库。

6. [Webpack] ts 编写的库，在使用 webpack 构建的时候，如何对外提供 d.ts 【热度: 224】

在 TypeScript (TS) 中使用 Webpack 构建并为库提供 `.d.ts` 类型声明文件，需要遵循以下步骤：

1. 配置 TypeScript 编译选项：

在库项目的根目录下创建或编辑 `tsconfig.json` 文件，确保编译器配置选项如下：

```
1  {
```

```

2  "compilerOptions": {
3    "declaration": true, // 生成对应的 '.d.ts' 文件
4    "declarationDir": "types", // 指定生成的声明文件存放目录
5    "outDir": "lib" // 指定编译后文件的输出目录
6    // 其他需要的编译选项},
7  "include": ["src/**/*"], // 包含源码的目录
8  "exclude": ["node_modules"] // 排除的目录
9  }

```

- `declaration`: 这个选项会告诉 TypeScript 编译器为每个 `.ts` 文件生成相应的 `.d.ts` 声明文件。
- `declarationDir`: 这是指定声明文件的输出目录。

1. 配置 Webpack:

在我们的 Webpack 配置中（通常是 `webpack.config.js`），我们需要设置 `output` 以指向我们的输出目录，同时可能需要使用一些加载器(loader)如 `ts-loader` 或 `babel-loader` 来处理 TypeScript 文件。

2. 一个简单的 webpack 配置示例可能如下:

```

1  const path = require("path");
2
3  module.exports = {
4    entry: "./src/index.ts", // 入口文件
5    rules: [
6      {
7        test: /\.tsx?$/,
8        use: "ts-loader",
9        exclude: /node_modules/,
10     },
11   ],
12 },
13 resolve: {
14   extensions: [".tsx", ".ts", ".js"],
15 },
16 output: {
17   filename: "your-library.js", // 输出文件名
18   path: path.resolve(__dirname, "lib"), // 输出文件夹
19   libraryTarget: "umd", // 使库支持各种模块系统
20   globalObject: "this",
21 },
22 };

```

1. 发布包：

当你发布你的库时，你需要确保 `package.json` 文件中包含 `types` 或 `typings` 字段指向入口 `.d.ts` 文件。

2. 例如：

```
1  {
2    "name": "your-library",
3    "version": "1.0.0",
4    "main": "lib/your-library.js",
5    "typings": "types/index.d.ts"
6    // 其他配置项...
7  }
```

1. 这告诉使用你库的 TypeScript 用户，在哪里可以找到类型声明文件。

2. 保证类型声明文件的发布：

如果你的 npm 发布流程排除了 `types` 目录，你需要更新 `.npmignore` 文件来确保 `.d.ts` 文件会被包含在发布的 npm 包中。

完成这些配置后，当你用 webpack 构建并发布你的库时，用户将能够获得与 JavaScript 文件关联的 TypeScript 类型声明，以便在他们的 TypeScript 项目中获得类型检查和智能提示。

7. 测试前端代码覆盖率一般有什么手段？【热度: 550】

前端代码的测试覆盖率通常是指衡量在测试过程中有多少代码被执行了的一个指标。测试覆盖率有助于了解测试的全面性，以下是测试前端代码覆盖率常用的手段：

1. 单元测试：

- 使用测试框架（例如 Jest, Mocha, Jasmine 等）编写单元测试。
- 利用测试框架或插件生成覆盖率报告（例如 Istanbul/nyc 工具可以与这些框架集成以生成覆盖率数据）。

2. 集成测试：

- 使用测试工具（比如 Cypress, Selenium 等）编写集成测试来模拟用户操作。
- 通常这些工具也支持收集代码覆盖率信息。
-

3. 手动测试与覆盖率工具结合：

- 在手动测试过程中，可以开启浏览器的覆盖率工具（如 Chrome DevTools 中的 Coverage Tab）记录覆盖率。
- 可以通过浏览器扩展程序或者自动化脚本来启动这些工具。

4. 测试覆盖率服务：

- 使用像 Codecov 或 Coveralls 这样的服务，在 CI/CD 流程中集成覆盖率测试和报告。

8. [Webpack] 如何提取复用代码给多个 entry 使用？【热度：292】

在 Webpack 中提取源码里被多个入口点复用的代码，例如一个 `utils` 文件，可以通过配置 `optimization.splitChunks` 来实现。Webpack 会将这些频繁复用的模块提取出来，打包到一个独立的 chunk 中，使得浏览器可以单独缓存这部分代码，并在多个页面间共享使用，优化加载性能。

使用 `splitChunks` 的基本配置如下：

```
1  module.exports = { // ...其他配置...
2    optimization: {
3      splitChunks: {
4        chunks: "all", // 对所有的 chunk 有效，包括异步和非异步 chunk
5        cacheGroups: {
6          commons: {
7            name: "commons", // 提取出来的文件命名为 'commons.js'
8            chunks: "initial", // 提取出的 chunk 类型, 'initial' 为初始
              chunk, 'async' 为异步 chunk, 'all' 表示全部 chunk
9            minChunks: 2, // 模块被引用 >= 2 次，便分割
10           minSize: 0, // 模块的最小体积
11         },
12       },
13     },
14   },
15   };
```

这个配置的含义是：

- `chunks: 'all'` 指定要优化的 chunk 类型，这里设置为 `all` 代表所有的 chunk，不管是动态还是非动态加载的模块。
- `cacheGroups` 是一个对象，用于定义缓存组，可以继承和/或覆盖 `splitChunks` 的任何选项。每个缓存组可以有自己的配置，将不同的模块提取到不同的文件中。
- `cacheGroups.common` 定义了一个缓存组，专门用于提取 `initial` chunk（最初依赖的模块）中被至少两个 chunk 所共享的模块。
- `name: 'commons'` 为生成的文件定义了一个自定义名称。
- `minChunks: 2` 表示模块至少被两个入口点引用时，才会被提取。

- `minSize: 0` 指定模块的最小体积是 0，即任意大小的模块都被提取。

这会让任何从 `node_modules` 目录导入，并在至少两个入口点中使用的模块，都会被打包到一个名为 `commons.js` 的文件中（当然，实际的文件名会受到 `output` 配置的影响，例如是否包含哈希值等）。

正确配置这些参数后，`utils` 这样的模块就会被自动提取并共享，而不是在每个入口点的 bundle 中重复包含。这样做的好处是，任何更新业务逻辑的时候，只要 `utils` 没有发生变化，用户浏览器上已缓存的 `commons.js` 文件就不需要重新下载。

9. [Webpack] 如何将一些通用的依赖打包成一个独立的 bundle 【热度: 643】

在 Webpack 中，将一些通用的依赖，如 React、React DOM、React Router 等库和框架，打包成一个独立的 bundle，通常是为了长期缓存和减少每次部署更新的下载量。这可以通过 "代码分割" (code splitting) 和 "优化" (optimization) 配置来实现。

以下是 Webpack 中分离通用依赖的几个步骤：

1. 使用 `entry` 来定义不同的入口点: 可以通过配置一个额外的入口来创建一个只包含通用库的 bundle，也就是所谓的 "vendor" bundle。

```
1  module.exports = {
2    entry: {
3      main: "./src/index.js", // 你的应用代码
4      vendor: ["react", "react-dom", "react-router"], // 指定共享库
5    }, // ...
6  };
```

1. 使用 `SplitChunksPlugin`: 这个插件可以将共享代码分割成不同的 chunks，并可以通过配置将其从业务代码中分离出来。在 Webpack 4 及之后的版本中，默认内置了 `optimization.splitChunks`，就是这个插件的配置方法。

```
1  module.exports = { // ...
2    optimization: {
3      splitChunks: {
4        cacheGroups: {
5          vendor: {
6            test: /[\\/]node_modules[\\/]/, // 指定是 node_modules 下的第三方包
7            name: "vendors", // 打包后的文件名，任意命名
8            chunks: "all", // 对所有的 chunk 生效
9          },
10         },
11       },
12     },
13   };
```

```
11     },
12     },
13   };
```

1. 配置 `output`：虽然不是必须的，你还可以在 `output` 中定义 `filename` 和 `chunkFilename`，来控制主入口和非主入口 chunks 的文件名。

```
1  output: {
2    filename: '[name].[contenthash].js',
3    chunkFilename: '[name].[contenthash].js'
4  }
```

通过这样的配置，Webpack 在打包时会自动将 `node_modules` 中的依赖和业务代码分离开来，业务代码会被打包到 `main` chunk 中，而第三方库则会打包到 `vendors` chunk。

10. [Webpack] output 配置里面， chunkFilename 和 filename 区别是什么？【热度: 210】

在 Webpack 中的 `output` 配置对象中，`filename` 和 `chunkFilename` 是用来指定输出文件的命名方式的关键属性。它们之间的区别主要涉及到最终生成的 JavaScript 文件的类型。

1. `filename`: `filename` 属性用于指定输出的 bundle 的名称。当你的应用只有一个入口点时，可以直接使用一个固定名称。如果有多个入口点，那么你可以使用占位符来确保每个文件具有唯一的名称，如使用 `[name]` 来对应每个入口点的名称。`filename` 主要与入口点相关联的那些文件有关。

```
1  output: {
2    filename: "bundle.js"; // 一个固定名称，适用于单入口// 或者
3    filename: "[name].bundle.js"; // 使用占位符，适用于多入口
4  }
```

1. `chunkFilename`: `chunkFilename` 属性用于指定非入口的 chunk（通常是动态加载的模块）的名称。这些 chunk 文件通常是由于代码分割产生的。当使用如 `import()` 这样的动态导入语法时，Webpack 会分割代码到新的 chunk 中，这时候 `chunkFilename` 的命名规则就会被应用。

```
1  output: {
2    chunkFilename: "[name].chunk.js";
3  }
```

这意味着如果你有一个动态加载的模块（例如懒加载模块），Webpack 会使用 `chunkFilename` 的规则来生成这些额外的文件。同样，你也可以在 `chunkFilename` 中使用占位符来保持文件名的唯一性。常用的占位符有 `[id]`，`[name]`，`[chunkhash]` 等。

使用这两个属性使得 Webpack 能够区分出入口文件和其他类型的文件，从而允许开发者更好地控制输出资源的命名和缓存。

11. [Webpack] 多入口打包共享模块【热度: 337】

默认情况下，每个入口 chunk 保存了全部其用的模块(modules)。使用 `dependOn` 选项你可以与另一个入口 chunk 共享模块：

```
1  module.exports = { //...
2    entry: {
3      app: { import: './app.js', dependOn: 'react-vendors' }, "react-vendors":
4        ["react", "react-dom", "prop-types"],
5    },
6  };
```

app 这个 chunk 就不会包含 react-vendors 拥有的模块了。

`dependOn` 选项的也可以为字符串数组：

```
1  module.exports = { //...
2    entry: {
3      moment: { import: "moment-mini", runtime: "runtime" },
4      reactvendors: { import: ["react", "react-dom"], runtime: "runtime" },
5      testapp: { import: './wwwroot/component/TestApp.tsx',
6        dependOn: ["reactvendors", "moment"],
7      },
8    },
9  };
```

此外，你还可以使用数组为每个入口指定多个文件：

```
1  module.exports = { //...
2    entry: {
3      app: { import: ["./app.js", "./app2.js"], dependOn: "react-vendors"
4        }, "react-vendors": ["react", "react-dom", "prop-types"],
5    },
6  };
```

看一个完整案例

```
1  module.exports = {//...
2    entry: {
3      home: "./home.js",
4      shared: ["react", "react-dom", "redux", "react-redux"],
5      catalog: {import: "./catalog.js",
6        filename: "pages/catalog.js",
7        dependOn: "shared",
8        chunkLoading: false, // Disable chunks that are loaded on demand and
        put everything in the main chunk.
9      },
10     personal: {import: "./personal.js",
11       filename: "pages/personal.js",
12       dependOn: "shared",
13       chunkLoading: "jsonp",
14       asyncChunks: true, // Create async chunks that are loaded on demand.
15       layer: "name of layer", // set the layer for an entry point
16     },
17   },
18   };
```

12. [Webpack] 如何使用 ts 来编写配置文件? 【热度: 251】

要使用 [Typescript](#) 来编写 webpack 配置, 你需要先安装必要的依赖, 比如 Typescript 以及其相应的类型声明, 类型声明可以从 [DefinitelyTyped](#) 项目中获取, 依赖安装如下所示:

```
npm install --save-dev typescript ts-node @types/node @types/webpack
```

如果使用版本低于 v4.7.0 的 *webpack-dev-server*, 还需要安装以下依赖

```
npm install --save-dev @types/webpack-dev-server
```

完成依赖安装后便可以开始编写配置文件, 示例如下:

webpack.config.ts

```
1  import path from "path";
2  import webpack from "webpack";
3  // in case you run into any typescript error when configuring devServerimport
   "webpack-dev-server";
4
5  const config: webpack.Configuration = {
6    mode: "production",
7    entry: "./foo.js",
8    output: {
```



```
9     path: path.resolve(__dirname, "dist"),
10     filename: "foo.bundle.js",
11   },
12 };
13
14 export default config;
```

该示例需要 typescript 版本在 2.7 及以上，并在 `tsconfig.json` 文件的 `compilerOptions` 中添加 `esModuleInterop` 和 `allowSyntheticDefaultImports` 两个配置项。

值得注意的是你需要确保 `tsconfig.json` 的 `compilerOptions` 中 `module` 选项的值为 `commonjs`，否则 webpack 的运行会失败报错，因为 `ts-node` 不支持 `commonjs` 以外的其他模块规范。

你可以通过三个途径来完成 `module` 的设置：

- 直接修改 `tsconfig.json` 文件
- 修改 `tsconfig.json` 并且添加 `ts-node` 的设置。
- 使用 `tsconfig-paths`

第一种方法就是打开你的 `tsconfig.json` 文件，找到 `compilerOptions` 的配置，然后设置 `target` 和 `module` 的选项分别为 `"ES5"` 和 `"CommonJs"` (在 `target` 设置为 `es5` 时你也可以不显示编写 `module` 配置)。

第二种方法 就是添加 `ts-node` 设置：

你可以为 `tsc` 保持 `"module": "ESNext"` 配置，如果你是用 webpack 或者其他构建工具的话，为 `ts-node` 设置一个重载（override）。[ts-node 配置项](#)

```
1  {
2    "compilerOptions": {
3      "module": "ESNext"
4    },
5    "ts-node": {
6      "compilerOptions": {
7        "module": "CommonJS"
8      }
9  }
```

第三种方法需要先安装 `tsconfig-paths` 这个 npm 包，如下所示：

```
1  npm install --save-dev tsconfig-paths
```

安装后你可以为 webpack 配置创建一个单独的 TypeScript 配置文件，示例如下：

tsconfig-for-webpack-config.json

```
1  {
2    "compilerOptions": {
3      "module": "commonjs",
4      "target": "es5",
5      "esModuleInterop": true
6    }
7  }
```

提示

ts-node 可以根据 `tsconfig-paths` 提供的环境变量 `process.env.TS_NODE_PROJECT` 来找到 `tsconfig.json` 文件路径。

`process.env.TS_NODE_PROJECT` 变量的设置如下所示：

package.json

```
1  {
2    "scripts": {
3      "build": "cross-env TS_NODE_PROJECT=\"tsconfig-for-webpack-config.json.\n      webpack\"
4    }
5  }
```

之所以要添加 `cross-env`，是因为我们在直接使用 `TS_NODE_PROJECT` 时遇到过 `"TS_NODE_PROJECT" unrecognized command` 报错的反馈，添加 `cross-env` 之后该问题也似乎得到了解决

13. [Webpack] 内部执行原理【热度: 668】

这部分可以直接转官网，官网讲得非常好：

<https://www.webpackjs.com/concepts/under-the-hood/>

14. [Webpack] 为何不支持 CMD 模块化【热度: 255】

1. CMD 是国内产品，webpack 是国外产品，而且 CMD 还没有火起来的时候，就已经被 ESM 替代了

2. CMD 是更加懒惰，是依赖就近，延迟执行。也就是说，在模块中需要用到依赖时，才去引入依赖。这和 Webpack 的理念以及模块收集和打包机制不兼容

CMD (Common Module Definition) 是一种深受国内前端开发者喜爱的模块定义规范，主要被用在了 Sea.js 这个模块加载器中。CMD 是国内开发者提出的规范，它和 AMD 很相似，但是更符合国内开发者的习惯，需要时可以延迟执行。

Webpack 本身是围绕 NPM 生态和标准化模块格式（如 ES Modules 和 CommonJS）构建的，而 NPM 生态主要使用的是 CommonJS 形式。因此，对于大多数使用 NPM 之 Webpack 的用户来说，这些就足够用了。而 ES Modules 作为 JavaScript 官方的模块系统标准，越来越多地在现代应用中被采用。

面对 CMD，Webpack 的社区并没有广泛地采用或者需要支持这种模块定义。CMD 在模块定义时依赖于具体的 API 和加载时机，这和 Webpack 的理念以及模块收集和打包机制不完全兼容。Webpack 鼓励在编译时就确定模块依赖，而 CMD 更倾向于运行时动态确定。

尽管如此，理论上是可以借助一些插件或 loader 来实现对 CMD 模块的支持的，但是官方并没有集成这样的功能，因为需求没有那么大，同时现有的模块加载机制已经可以满足绝大多数场景的需要。随着前端工程化的深入，标准化的模块定义（如 ES Modules）更加受到青睐，而特定的模块定义（如 CMD）则逐渐被边缘化。因此，Webpack 没有默认支持 CMD，也反映了当前前端模块化开发的趋势和实践。

15. [Webpack] 支持哪些模块化加载？【热度: 154】

Webpack 支持以下几种模块化标准：

1. ESM (ECMAScript Modules): 这是 JavaScript ES6 中引入的官方标准模块系统。使用 `import` 和 `export` 语句来导入和导出模块。
2. CommonJS: 主要用于 Node.js，允许使用 `require()` 来加载模块和 `module.exports` 来导出模块。
3. AMD (Asynchronous Module Definition): 用于异步加载模块，并使用 `define` 方法来定义模块。
4. UMD (Universal Module Definition): 结合了 AMD 和 CommonJS 的特点，并支持全局变量定义的方式，使得模块可以在客户端和服务端上运行。

除此之外，Webpack 还可以处理非 JavaScript 文件并将它们视为模块，例如 CSS, LESS, SASS, 图像文件(PNG, JPG, GIF, SVG 等), 字体(OTF, TTF, WOFF, WOFF2, EOT), HTML 以及任何其他类型的文件。这通过使用相应的 loader 来实现，如 `style-loader`, `css-loader`, `file-loader` 等。这些 loader 会将非 JavaScript 文件转换为可以被 Webpack 处理的模块。

16. 前端视角 - 如何保证系统稳定性【热度: 566】

前端视角来做稳定性，本是一个开放性话题，这里没有统一的解法，作者在此提供几个思路和反向：

1. 静态资源多备份（需要有备份）
2. 首屏请求缓存
3. 请求异常报警
4. 页面崩溃报警
5. E2E 定时全量跑用例

17. webpack 的主要配置项有哪些【热度: 766】

Webpack 是一个现代 JavaScript 应用程序的静态模块打包器。配置文件名通常为 `webpack.config.js`，它提供了一种配置 Webpack 的方式。下面是一些主要的 Webpack 配置选项：

1. entry: 入口起点(entry point)指示 webpack 应该使用哪个模块，来作为构建其内部依赖图的开始。可以指定一个或多个入口起点。
2. output: output 属性告诉 webpack 在哪里输出它所创建的 bundles，以及如何命名这些文件，默认值为 `./dist`。
3. module: module 属性用于决定如何处理项目中的不同类型的模块。
 - rules: 配置模块的读取和解析规则，通常用来配置 loader。
4. resolve: 配置模块如何解析。
 - extensions: 自动解析确定的扩展，此选项能够使用户在引入模块时不带扩展。
5. plugins: 插件是用来扩展 webpack 功能的。它们会在构建流程中的特定时机注入运行逻辑来改变构建结果或做你想要的事情。
6. devServer: 通过来自 `webpack-dev-server` 的这些选项能够对开发服务器的行为进行控制。
7. devtool: 此选项控制是否生成，以及如何生成 source map。
8. mode: 通过设置 `development` 或 `production` 之中的一个，来为流程提供相应模式下的内置优化。
9. optimization: 包含一组可用来调整构建输出的选项。
 - splitChunks: 配置模块的拆分，可以将公共的依赖模块提取到已有的入口 chunk 中，或者提取到一个新生成的 chunk。
 - runtimeChunk: 为每个 entry 创建一个运行时文件。
10. performance: 允许 webpack 根据某些参数，控制资产和入口起点的最大大小。
11. externals: 防止将某些 import 包(package)打包到 bundle 中，而是在运行时(runtime)再去从外部获取这些扩展依赖。

每个项目的具体需求不同，Webpack 的配置也会有所不同。这些选项提供了强大的配置能力和灵活性，可以用来定制 Webpack 的打包、加载和转换行为。

18. [webpack] optimize 配置中， 分割代码配置 splitChunks 怎么使用【热度: 546】

在 webpack 中， `splitChunks` 选项是 `optimization` 对象的一个属性，可以用来定义如何分割代码块。默认情况下，webpack 会将所有来自 `node_modules` 的模块分割到一个叫做 `vendors` 的 chunk 中，并且将共享或来自异步请求的模块分割成不同的 chunks。通过配置 `splitChunks` 选项，你可以控制这些行为，创建更细致的代码分割策略。以下是如何使用 `splitChunks` 来优化你的 bundle。

基本配置

```
1  module.exports = {  
2    optimization: {  
3      splitChunks: {  
4        chunks: "all", // 分割所有类型的chunks: 初始和动态加载的chunk  
5      },  
6    },  
7  };
```

在这个配置中， `chunks: 'all'` 指示 webpack 对同步和异步引入的模块都进行分割。webpack 会根据内部的一些默认标准（如模块大小、请求的 chunks 数目等）来决定是否分割一个模块。

基础属性配置

下面的表格详细描述了 `splitChunks` 配置选项及其作用：

配置选项	类型	默认值	说明
chunks	'all', 'async', 'initial'	'async'	设置优化哪些类型的 chunk。
minSize	Number	20000 (20kb)	生成 chunk 的最小体积（以字节为单位）。
maxSize	Number	0	尝试将 chunk 分割成不大于指定体积的块。此选项正在实验中，并可能在将来的

			webpack 版本中更改。
minChunks	Number	1	模块被分享到的最少 chunk 数。
maxAsyncRequests	Number	5	按需加载时的最大并行请求数。
maxInitialRequests	Number	3	一个入口点的最大并行请求数。
automaticNameDelimiter	String	'~'	用于生成名称的分隔符。
name	Boolean or String or Function	TRUE	分割块的名称。
cacheGroups	Object	-	一个对象，它定义了对于.cacheGroups 的子选项，用来控制缓存组聚合和生成的 chunks。
cacheGroups.test	RegExp or Function	-	控制哪些模块被这个缓存组捕捉。
cacheGroups.priority	Number	0	缓存组点击时的优先级，数值越大，优先级越高。
cacheGroups.reuseExistingChunk	Boolean	TRUE	如果当前块包含已从主 bundle 分割的模块，则重用它。
cacheGroups.filename	String or Function	-	允许为生成的 chunk 自定义文件名。

以下是针对上述表格中提及的某些属性的进一步说明：

- `chunks` 选项指定是对哪些 chunks 应用这些优化措施。它可以是三个值之一：'all'会影响所有的 chunks，这使得在异步和非异步 chunks 之间共享模块成为可能；'async' 仅仅影响被异步加载的 chunks；'initial' 仅影响初始加载的 chunks。
- `minSize` 和 `maxSize` 用于控制 webpack 试图以多大的 chunk 为目标。`minSize` 可以避免 chunks 过小，而 `maxSize` 可以帮助进一步分割大的 chunks。
- `cacheGroups` 是配置高度定制化的代码分割策略的地方。默认情况下 webpack 会将来自 `node_modules` 文件夹的代码分割到一个叫做 `vendors` 的 chunk 中，另外 webpack 会将重复引入的代码分割到一个叫做 `default` 的 chunk 中。在这里可以覆盖这些默认设置，或是增加新的缓存组。

使用实例：

```
1  module.exports = { // ...
2    optimization: {
3      splitChunks: {
4        chunks: "all",
5        minSize: 30000, // 最小 30kb
6        maxSize: 0, // 默认无上限
7        minChunks: 1,
8        maxAsyncRequests: 5,
9        maxInitialRequests: 3,
10       automaticNameDelimiter: "~",
11       name: true,
12       cacheGroups: {
13         vendors: {
14           test: /[\\/]node_modules[\\/]/,
15           priority: -10,
16           name(module) { const packageName =
17             module.context.match(/[\\/]node_modules[\\/](.*?)([\\/]|$)/)[1]; return
18             `vendor.${packageName.replace("@", "")}`;
19           },
20           default: {
21             minChunks: 2,
22             priority: -20,
23             reuseExistingChunk: true,
24           },
25         },
26       },
27     },
28   };
29 }
```

高级配置 - 缓存组

缓存组（cacheGroups）能让你对分割出来的 chunks 进一步细分和控制。

```
1  module.exports = {//...
2    optimization: {
3      splitChunks: {
4        chunks: "all",
5        maxInitialRequests: Infinity, // 允许在一个入口处无限多的并行请求
6        minSize: 0, // 生成chunk的最小体积（以字节为单位）
7        cacheGroups: {
8          vendor: {
9            test: /[\\/]node_modules[\\/]/, // 正则表达式，用于测试模块路径，匹配
            node_modules目录下的模块
10           name(module) { // 得到模块名，可能是node_modules包名称的一部分const
            packageName = module.context.match(/[\\/]node_modules[\\/](.*?)([\\/]|$)/
            [1];return `npm.${packageName.replace("@", "")}`; // 创建chunk名
11           },
12         },
13       },
14     },
15   },
16   };
```

这个配置创建了一个缓存组 `vendor`，它会将所有从 `node_modules` 目录导入的模块分割到不同的 chunk 中，并为每个包创建一个以 `npm` 开头的 chunk 名。例如，如果你的应用依赖于 `lodash` 和 `react`，应用中就会有 `npm.lodash` 和 `npm.react` 两个额外的 chunks。

动态导入

当你使用像 `import()` 这样的动态导入语法时，`splitChunks` 插件会自动进行代码分割。

```
1  function getComponent() { // 当我们调用 import() 时，webpack 会对 lodash 进行代码
    分割return import("lodash").then(({ default: _ }) => {
2    const element = document.createElement("div");
3    element.innerHTML = _.join(["Hello", "webpack"], " ");return element;
4  });
5  }
6  getComponent().then((component) => {document.body.appendChild(component);
7  });
```


在这个例子中，`lodash` 会被分成一个单独的 chunk。当 `getComponent` 函数执行并调用 `import()` 时，`lodash` 库会作为一个单独的异步 chunk 加载进来。

通过 `splitChunks` 的适当配置，我们可以大幅度减小初始加载所需的时间，并确保用户只下载当前真正需要的代码，这样就可以加快应用程序的交互速度。

19. [webpack] optimize 配置有哪些作用 【热度: 280】

Webpack 的 `optimize` 选项是在指定 Webpack 配置对象时，用于配置优化选项的一个属性。该属性下包含了一系列用于调整构建输出质量和性能的设置。这里是一些 `optimize` 属性中可能包含的选项：

- `splitChunks`：这用于代码分割，可以将公共的依赖模块提取到已有的入口 chunk 中，或者产生一个新的 chunk。这可以被用来得到更小的 bundle 体积，优化加载时间，或者更好的缓存利用。
- `runtimeChunk`：该选项将 Webpack 的运行时代码分割成一个单独的 chunk。使用这个设置有利于长期缓存，并且当你使用多个入口点时推荐使用。
- `minimize`：当设置为 `true` 时，Webpack 会启动代码压缩。通常，这会使用 `UglifyJSPlugin` 来进行 JavaScript 代码的压缩，但现在通常默认使用更现代的工具如 `TerserPlugin`。
- `minimizer`：当你想要覆盖默认的压缩工具或者提供额外的压缩工具时使用。
- `noEmitOnErrors`（早期版本称为 `NoEmitOnErrorsPlugin`）：启用该选项后，Webpack 编译错误将会导致不生成输出。这确保了不会发出包含错误的 assets。
- `concatenateModules`（早期版本称为 `ModuleConcatenationPlugin`）：这个选项会试图找到模块图中可以安全地连接到单一模块的所有模块，来优化结果的体积。
- `usedExports`（也称为 tree shaking）：该选项用于标记 "tree shaking" 中未被使用的导出，使它们能被压缩工具删除。

在 Webpack 4 及以上版本中，这些优化默认在 `mode` 被设置为 `production` 时生效。通过合理地配置这些选项，开发者可以显著提高应用程序的加载和运行性能。这些优化通常包括减少 bundle 的体积和提高代码的运行时效率。在开发模式下，很多优化默认是关闭的，以提供更快的构建速度和更好的调试体验。

20. [webpack] mode 是做什么用？ 【热度: 475】

在 webpack 中，`mode` 属性用来指定当前的构建环境是：`development`、`production` 或者是 `none`。设置 `mode` 可以使用 webpack 内置的函数，默认值为 `production`。

`mode` 属性的主要作用是：根据当前的构建环境，启用 webpack 内置在该环境下推荐的优化。

mode 的具体作用包括：

1. development

- 主要优化了增量构建速度和开发体验。

- `process.env.NODE_ENV` 的值设为 `development`。
- 启用热替换模块（Hot Module Replacement）。
- 启用开发工具（如调试源码的 source map）以更好地进行调试。

2. production

- 一些处理优化，以提升应用在生产环境的性能。
- `process.env.NODE_ENV` 的值设为 `production`。
- 启用代码压缩（例如 TerserPlugin）。
- 删除 dead code（通过 Tree Shaking）。
- 作用域提升等各种性能优化措施。

3. none

- `mode` 设置为 `none` 则不启用任何默认优化选项，`process.env.NODE_ENV` 也不会被设置，默认为 `production`。

使用方法：

在 webpack 配置文件中，可以直接设置 `mode` 的值：

```
1  module.exports = {  
2    mode: "development", // 'production' 或 'none'// 其他配置...  
3  };
```

或者，在命令行中使用 `--mode` 参数：

```
1  webpack --mode=production
```

设置 `mode` 是告诉 webpack 使用其内部的优化策略，各个模式预定义了一些 webpack 的行为，开发者可以不需要进行详细的配置，也能快速启动一个针对特定环境优化过的构建过程。

21. V8 里面的 JIT 是什么？【热度: 694】

在计算机科学中，JIT 是“Just-In-Time”（即时编译）的缩写，它是一种提高代码执行性能的技术。具体来说，在 V8 引擎（Google Chrome 浏览器和 Node.js 的 JavaScript 引擎）中，JIT 编译器在 JavaScript 代码运行时，将其编译成机器语言，以提高执行速度。

这里简要解释下 JIT 编译器的工作原理：

1. 解释执行：V8 首先通过一个解释器（如 Ignition）来执行 JavaScript 代码。这个过程中，代码不会编译成机器语言，而是逐行解释执行。这样做的优点是启动快，但执行速度较慢。
2. 即时编译：当代码被多次执行时，V8 会认为这部分代码是“热点代码”（Hot Spot），此时 JIT 编译器（如 TurboFan）会介入，将这部分热点代码编译成机器语言。机器语言运行在 CPU 上比解释执行要快得多。
3. 优化与去优化：JIT 编译器会对热点代码进行优化，但有时候它会基于错误的假设做出优化（例如认为某个变量总是某种类型）。如果后来的执行发现这些假设不成立，编译器需要去掉优化（Deoptimize），重新编译。

JIT 编译器的一个关键优点是它能够在不牺牲启动速度的情况下，提供接近于或同等于编译语言的运行速度。这使得像 JavaScript 这样原本被认为执行效率较低的语言能够用于复杂的计算任务和高性能的应用场景。

随着 V8 和其他现代 JavaScript 引擎的不断进步，JIT 编译技术也在持续优化，以提供更快的执行速度和更高的性能。

22. 单元测试中，TDD、BDD、DDD 分别指？【热度: 166】

TDD、BDD 和 DDD 这三个缩写在软件开发中分别代表以下概念：

1. TDD（Test-Driven Development）- 测试驱动开发：

TDD 是一种软件开发过程，其中开发人员首先编写一个小测试用例，然后编写足够的代码来使这个测试通过，最后重构新代码以满足所需的设计标准。这个过程就是一个循环，被称为“红-绿-重构”循环，其中测试先失败（红色），编写代码使其通过（绿色），然后优化代码（重构）。TDD 的焦点在于编写干净的代码和降低未来的缺陷。

2. BDD（Behavior-Driven Development）- 行为驱动开发：

BDD 将 TDD 的基本思想和原则扩展到软件的整个开发生命周期，但其着重点在于软件的行为——即软件应如何表现，而不仅仅是它应该完成什么功能。BDD 强调的是与利益相关者的交流与协作，通过使用通俗易懂的语言来写测试，让非技术人员也能理解测试内容。BDD 鼓励团队成员之间更好地沟通，确保所有人都对软件应有的行为有共同的理解。

3. DDD（Domain-Driven Design）- 领域驱动设计：

DDD 与 TDD 和 BDD 并不是同一类型的概念。DDD 是一种软件设计哲学，强调了在软件项目的设计与开发中应以业务领域（Domain）为中心。它主张将业务领域的专业知识嵌入到软件的设计中，从而使软件能更好地解决业务问题。DDD 通常涉及到丰富的领域模型以及分层的架构设计，以确保业务逻辑清晰和维护性高。

这三个概念在软件开发中都扮演着重要的角色。TDD 和 BDD 都与确保软件质量和满足用户需求有关，而 DDD 则是一种更宏观层面上对软件设计的方法论。

23. husky 作用是啥， 有哪些重要配置 【热度: 192】

Husky 是一个基于 Node 的 Git 钩子管理工具，用于在你的工作流程中强制执行 Git 钩子。Husky 允许你定义脚本，这些脚本会在不同的 Git 生命周期事件触发时自运行，比如在提交、推送或合并前。

使用 Husky 可以：

1. 保证提交质量：Husky 可以在你提交代码之前运行代码校验，确保代码符合项目规范，提高代码质量。
2. 维护代码风格：可以在提交时检查代码风格，确保代码风格一致性。
3. 自动化流程：支持在推送前执行代码部署、测试脚本，让整个开发流程自动化。
4. 预防错误：例如在允许推送到远程仓库之前检查代码中是否有遗留的更改。

Husky 的一些重要配置如下：

1. `npm install husky@latest --save-dev`：安装 husky。
2. `npx husky install`：在新建的项目管理下生成 husky 的配置文件。
3. `npx husky add .husky/*.sh`：添加 Git 钩子脚本，这里的 `*.sh` 是你想触发的钩子点，例如：`pre-commit`、`commit-msg` 等。

Husky 支持的钩子包括：

- `apply-patch-msg`：应用一个补丁到暂存区并生成提交信息时。
- `pre-applypatch`：打补丁前。
- `post-applypatch`：打补丁后。
- `pre-commit`：提交前，常用于检查代码、分析代码风格等。
- `prepare-commit-msg`：提交准备工作完成后，修改提交信息之前运行。
- `commit-msg`：检查提交信息有效性。
- `post-commit`：提交后。
- `pre-rebase`：回滚操作开始前。
- `post-checkout`：检出操作后（如切换分支）。
- `post-merge`：合并和变基操作后。

记得在 `.husky` 文件夹里配置这些钩子脚本，你可以根据项目需求来写自己的 hook 脚本。比如，设置一个 `.husky/pre-commit` 脚本（可能是一个 shell 脚本和 Node.js 脚本的组合），当你尝试提交代码时，Husky 将会运行这个脚本作为 `pre-commit` 钩子。

在一些场景下的 `.husky/pre-commit` 脚本，你可以指定运行如下：

```
1  #!/bin/sh
2  . "$$(dirname -- "$$0")/_/husky.sh"
3  npm run lint 运行 ESLint 检查代码
4  ./node_modules/.bin/prettier 格式化代码
```

以上脚本将确保代码在提交前通过了 linter 检查，并通过 prettier 快速格式化以及 TypeScript 编译。使用的时候，请确认你的项目已经有了 Node.js 环境，并且已经安装了 Husky 和相应的代码检查、格式化工具。

24. Husky 和 lint-staged 有什么区别？【热度: 387】

Husky 和 lint-staged 都是与 Git 钩子 (hooks) 配合使用的 Node.js 库，但它们的用途和工作方式有所不同：

1. Husky:

- Husky 是一个 Git 钩子管理器，它允许你触发自定义脚本在 git 事件发生时运行，如 `pre-commit`，`pre-push`，`post-merge` 等。
- 它的主要目的是自动化你的版本控制工作流程，例如在提交 (commit) 前运行代码检查、格式化代码或执行测试，以确保代码库的质量和一致性。

2. lint-staged:

- lint-staged 是一个运行在 Husky 钩子之上的工具，它专门用于对暂存区 (staged) 文件的检查。
- 当你运行 `git commit` 并且 Husky 触发 `pre-commit` 钩子时，lint-staged 会检查你即将提交的代码（即 `git add` 后的文件列表），并运行你配置好的检查脚本，如代码格式化程序、linter 或其他工具。
- 它的目的是确保在提交之前，只有没有检查错误的代码会被提交。

简而言之，Husky 是一个可以触发多种钩子事件的工具，而 lint-staged 是一种专门用于检查 Git 暂存区文件的工具。它们通常是配合使用的，因为 lint-staged 需要通过 Husky 来触发钩子。在你初始化项目并配置 CI/CD 流程时，通常会同时用到它们。

25. [webpack] 打包时 hash 码是如何生成的【热度: 167】

Webpack 在打包过程中生成 hash 码主要用于缓存和版本管理。主要有三种类型的 hash 码：

1. hash：是和整个项目的构建相关，只要项目文件有修改，整个项目构建的 hash 值就会更改。这意味着任何一个文件的改动都会影响到整体的 hash 值。
2. chunkhash：与 webpack 打包的 chunk 有关，不同的 entry 会生成不同的 chunkhash 值。例如，如果你的配置生成了多个 chunk（例如使用了 code splitting），每个 chunk 的更新只会影响到它自身的 chunkhash。
3. contenthash：根据文件内容来定义 hash，内容不变，则 contenthash 不变。这在使用诸如 CSS 提取到单独文件的插件时特别有用，因此只有当文件的内容实际改变时，浏览器才会重新下载文件。

生成方式：

- hash 和 chunkhash 主要是通过某种 hash 算法（默认 MD5）来对文件名或者 chunk 数据进行编码。
- contenthash 是通过构建时的 webpack 插件（如 mini-css-extract-plugin）来处理的，它会对文件内容进行 hash。

Hash 码的生成可以被 webpack 配置的 hashFunction, hashDigest, hashDigestLength 等选项影响。例如，你可以选择不同的算法如 SHA256 或者 MD5，以及可以决定 hash 值的长度。

在 webpack 的配置文件中，可以通过如下方式设定 hash:

```
1  output: {  
2    filename: '[name].[chunkhash].js',  
3    path: __dirname + '/dist'  
4  }
```

这会将输出的文件名设置为入口名称加上基于每个 chunk 内容的 hash。在使用 `webpack-dev-server` 或者 `webpack --watch` 时，不会生成实际的文件，所以这些 hash 值是在内存中计算并关联的。

26. 如何从 0 到 1 搭建前端基建【热度: 404】

如何从 0 到 1 搭建前端基建

有一个非常经典的文章，直接参考即可：[非大厂的我们，要如何去搞前端基建？](#)

这里简单总结一下文章里面的要点

1. 什么是基建？

2. 为什么要做前端基建？

业务复用；

提升研发效率；

规范研发流程；

团队技术提升；

团队的技术影响力；

开源建设；

3. 前端基建如何推动落地？

- 要合适的同学（资源）
- 要解决的问题（问题）

- 要解决问题方案计划书（方案）
- 要具体执行的步骤（执行）

技术基建四大特性（切记）

- 技术的健全性
- 基建的稳定性
- 研发的效率性
- 业务的体验性

4.前端基建都有什么？

- 前端规范（Standard）
- 前端文档（Document）
- 前端项目模板管理（Templates）
- 前端脚手架（CLI）
- 前端组件库（UI Design）
- 前端响应式设计 or 自适应设计
- 前端工具库（类 Hooks / Utils）
- 前端工具自动化（Tools）
- 接口数据聚合（BFF）
- 前端 SSR 推进
- 前端自动化构建部署（CI/CD）
- 全链路前端监控/数据埋点系统
- 前端可视化平台
- 前端性能优化
- 前端低代码平台搭建
- 微前端（Micro App）

27. [React] 为什么 react 组件， 都必须申明一个 `import React from 'react';` 【热度: 115】

首先要知道一个事情：JSX 是无法直接运行在浏览器环境。

原因

JSX 语法不能被浏览器解析和运行，因此需要插件 `@babel/plugin-transform-react-jsx` 来转换语法，使之能够在浏览器或任何 JavaScript 环境中执行。

所以 React 组件需要引入 `React` 的一个主要原因是：在组件中使用 JSX 时，JSX 语法最终会被 Babel 编译成使用 `React.createElement` 方法的 JavaScript 代码。也就是说，任何使用 JSX 的 React 组件的背后都隐含了 `React.createElement` 的调用。

例如，当你编写如下的 JSX 代码：

```
1  const MyComponent = () => {return <div>Hello, World!</div>;  
2  };
```

Babel 会将这段 JSX 编译为如下的 JavaScript 代码：

```
1  const MyComponent = () => {return React.createElement("div", null, "Hello,  
    World!");  
2  };
```

由于编译后的代码调用了 `React.createElement`，因此你需要在文件顶部导入 `React` 对象才能使用它。即使你在组件中并没有直接使用 `React` 对象，编译后的代码依赖于 `React` 的运行时。

Babel 7.0+ / React 17+，可以不再需要 import React

在 Babel 7.0 版本之后，`@babel/plugin-transform-react-jsx` 插件还支持一个自动模式，它可以自动引入 JSX 转换所需的 `React` 包，无需手动在每个文件中添加 `import React from 'react'`。

注意，随着 React 17 的新 JSX 变换，它们引入了一个新的 JSX 转换方式，这在新的 Babel 插件 `@babel/plugin-transform-react-jsx` 和 `@babel/preset-react` 中得到了支持。这意味着在写 JSX 时，你不再需要导入 `React`。这个插件现在接收一个 `{ runtime: 'automatic' }` 选项来启用这一特性。

举个例子，在使用新的 JSX 转换之后，编译器将会自动引入 JSX 的运行时库，而不是 `React`，例如对于一个使用了新转换的 `MyComponent` 的组件：

- `// React 17及支持新JSX转换的环境，可以不需要显式写这行`
`// import React from 'react';`

```
const MyComponent = () => {return <div>Hello, World!</div>;  
};
```

在新的转换下，你会看到类似 `import { jsx as _jsx } from 'react/jsx-runtime'` 的东西或者类似的别名，被自动插入到转译后的文件中，而不再是直接的 `React.createElement` 调

用。这就是为什么在新版本的 React 中，你可能不再需要手动导入 React 了。

补充一个细节知识点： `plugin-transform-react-jsx` 和 `@babel/preset-react` 是啥关系

它们是包含关系： `@babel/preset-react` 包括了 `@babel/plugin-transform-react-jsx`

`@babel/plugin-transform-react-jsx` 和 `@babel/preset-react` 都是 Babel 插件，它们在处理 React 项目中的 JSX 代码方面有关联，但它们的用途和包含的内容有所不同。

1. `@babel/plugin-transform-react-jsx`:

这是一个特定的 Babel 插件，它的功能就是将 JSX 语法转换为 `React.createElement` 调用。随着 React 17 的更新，它还允许使用新的 JSX 转换，无需导入 React 就可以使用 JSX。这意味着，在文件中不再需要 `import React from 'react'` 语句了，就可以使用 JSX。

2. 这个插件通常用于开发者想要精细控制某个具体转换功能时。如果你只需要转换 JSX 语法，但不需要处理其他与 React 相关的转换或优化，你可能会单独使用这个插件。

3. `@babel/preset-react`:

这是一个 Babel 预设，它是一组 Babel 插件的集合，旨在为 React 项目提供所需的全部 Babel 插件。`@babel/preset-react` 包括了 `@babel/plugin-transform-react-jsx`，但它还包含了其他一些插件，如处理 React 的显示名称的 `@babel/plugin-transform-react-display-name`，以及为开发模式和生产模式添加/删除某些代码的插件。

4. 预设的好处是简化了配置过程。开发者可以在 Babel 的配置中一次性添加 `@babel/preset-react`，而不是单独添加每一个与 React 相关的 Babel 插件。此外，预设将维护这些插件的正确版本和顺序，这有助于避免潜在的配置错误。

在实践中，大多数开发 React 应用的开发者会使用 `@babel/preset-react` 因为它提供了一个即插即用的 Babel 环境，无需担心各个插件的具体细节。但是也有些情况下，为了更细致的优化和控制，开发者可能会选择手动添加特定的插件，包括 `@babel/plugin-transform-react-jsx`。

28. babel 核心库有哪些？【热度: 35】

Babel 是一个 JavaScript 编译器，主要用于将 ES6 及以上版本的代码转换为向后兼容的 JavaScript 语法，以便在当前和旧版浏览器或环境中执行。核心的 Babel 库主要包括：

1. `@babel/core`:

这是 Babel 编译器的核心包，提供了 Babel 的主要转换引擎。它包含了解析、转换和生成代码的主要功能。几乎所有的 Babel 操作都需要这个模块作为基础。

2. `@babel/cli`:

这是 Babel 的命令行接口，通过它可以在终端或命令提示符中运行 Babel。它允许你执行转编译操作，如将 ES6 代码转换为 ES5。

3. @babel/preset-env:

这是一个智能预设，允许你使用最新的 JavaScript，而不必管理语法转换。`@babel/preset-env` 会根据你的目标环境（比如特定版本的浏览器或 Node.js），自动决定使用哪些 Babel 插件和 polyfills。

4. @babel/polyfill (现在已经被废弃，推荐使用 `core-js` 和 `regenerator-runtime`):

早期 Babel 版本中用于模拟完整的 ES2015+环境的包。它的目的是在全局范围内添加填充以模拟较新的环境。从 Babel 7.4.0 开始，建议直接包括 `core-js` 和 `regenerator-runtime`，因为这提供了更好的模块化和按需加载功能。

5. babel-loader:

这是 Babel 的一个 webpack 插件，可以将 Babel 集成到 webpack 构建过程中，使得你可以使用 webpack 来处理和打包使用了新版 JavaScript 语法的文件。

6. @babel/plugin-transform-runtime:

这个插件用于复用 Babel 注入的辅助代码，以节省代码大小，并能够在不污染全局环境的情况下使用新语言特性的 polyfills。

除了这些核心库外，还有许多可用的 Babel 插件，以支持各种 JavaScript 语法和特性（比如装饰器、类属性等）。这些插件可以按需引入，配置在 Babel 的配置文件（通常是 `.babelrc` 或 `babel.config.js`）中。这些插件的命名通常遵循 `@babel/plugin-` 的格式。

29. 在Babel里，stage0、stage1、stage2 和 stage3 分别代表什么含义？

无意中看到别人一个面试问题，个人感觉问这个问题的面试官，不是蠢就是坏。

没有任何面试价值，无法考察候选人水平。

仅仅作作为科普类型参考 - 热度为 0

Babel 是一个流行的 JavaScript 编译器，它允许开发者使用新的语言特性，然后将它们编译成可以在当前和低版本的浏览器或环境中运行的代码。

在 Babel 里，stage0、stage1、stage2 和 stage3 这些术语指的是 ECMAScript 提案的不同阶段。

ECMAScript 是 JavaScript 语言的标准化规范，新的特性进入标准之前会通过几个阶段的提案。

这些阶段表示了一个特性在正式成为 ECMAScript 标准的一部分之前的成熟度。这个过程有一个官方的 5 个阶段流程，即从 Stage 0 (strawman) 到 Stage 4 (finished)。下面是这些阶段的含义：

- Stage 0 - Strawman (稻草人阶段)：初始阶段，任何尚未被 TC39 (ECMAScript 的标准化组织) 官方审议的提案都属于这里。这些都是某个委员或者社区成员提交的想法，还不算是正式的提案。
- Stage 1 - Proposal (提案阶段)：这个阶段的特性是值得进一步探讨的。它们需要有一个形式化的提案和一个负责人。在这个阶段，主要是确定问题和解决方案，以及进行初步探讨。
- Stage 2 - Draft (草案阶段)：一旦一个提案到达这个阶段，它就被认为是初步规格的草案。特性的描述应该足够具体和详细，并且有初步的实现。这个阶段通常需要提案的规格文本和至少一种实验性实现。

- Stage 3 - Candidate（候选阶段）：在候选阶段，提案的规格已经基本完成，并且需要更多的用户反馈来发现潜在问题。通常在这个阶段，实现者和开发者开始在生产环境中尝试使用这些特性，发现问题并提出改善建议。
- Stage 4 - Finished（完成阶段）：当一个提案达到这个阶段，它已经准备好被集成到下一个版本的 ECMAScript 标准中了。这意味着它已经获得了多个独立环境的实现，通过了综合的可行性和稳定性测试，并且已经被 TC39 委员会接受。

开发者们可以根据特性的稳定性和自己的需求，选择使用 Babel 的哪个阶段的预设。然而，请注意，使用较低阶段的提案特性在生产环境中是有风险的，因为它们还没有被完全确定并可能会在将来发生变更。

30. Webpack 项目中通过 script 标签引入资源，在项目中如何处理？【热度: 100】

也是作者无意中看到的一个有意思的问题。

虽然有意思，但是没有任何价值，如果说在项目中遇到过的，而且处理过的同学，肯定知道怎么回答。

但是压根没有碰到过得，就算是你工作十年的老油条也是干望着。所有没有任何面试价值。

故此，可以当做科普来看看就行。

在使用 Webpack 打包的项目中，通常资源（如 JavaScript、CSS、图片等）会被 Webpack 处理，因为 Webpack 的设计初衷就是将所有资源视为模块，并进行有效的管理和打包。但有时候可能需要通过 `<script>` 标签直接引入资源，这通常有两种情况：

1. 在 HTML 文件中直接引入：

可以在项目的 HTML 文件中直接使用 `<script>` 标签来引入外部资源：

```
1  <!-- 若要使用 CDN 上托管的库 --><script src="https://cdn.example.com/library.js">
    </script>
```

1. 这种方法简单直接，但要记住，由于这些资源不会被 Webpack 处理，它们不会被包含在 Webpack 的依赖图中，并且也不会享受到 Webpack 的各种优化。

2. 使用 Webpack 管理：

如果想要 Webpack 来处理这些通过 `<script>` 引入的资源，可以使用几种插件和加载器：

- `html-webpack-plugin` 可以帮助你生成一个 HTML 文件，并在文件中自动引入 Webpack 打包后的 bundles。
- `externals` 配置允许你将一些依赖排除在 Webpack 打包之外，但还是可以通过 `require` 或 `import` 引用它们。
- `script-loader` 可以将第三方全局变量注入的库当作模块来加载使用。

3. 例如，使用 `html-webpack-plugin` 和 `externals`，你可以将一个库配置为 `external`，然后通过 `html-webpack-plugin` 将其引入：

```
1  // webpack.config.js 文件const HtmlWebpackPlugin = require("html-webpack-
  plugin");
2
3  module.exports = {// ...
4    externals: {
5      libraryName: "LibraryGlobalVariable",
6    },
7    plugins: [new HtmlWebpackPlugin({
8      template: "src/index.html",
9      scriptLoading: "blocking", // 或者 'defer'
10    })],
11  ],
12  };
```

1. 然后，在你的 `index.html` 模板文件中可以这样引入资源：

```
1  <script src="https://cdn.example.com/library.js"></script>
```

1. 使用 `externals` 的方法能让你在 Webpack 打包的模块代码中用正常的 `import` 或 `require` 语句来引用那个全局变量：

```
1  // 你的 JavaScript 代码文件中import Library from "LibraryName"; // 虽然定义了
  external, Webpack依然会处理这个import
```

应根据项目需求和现有的架构来决定使用哪种方法。上述两种方法中，第二种可以更好地利用 Webpack 的功能，第一种则更加简单直接。

31. 为什么 Vite 速度比 Webpack 快？【热度: 382】

1、开发模式的差异

在开发环境中，`Webpack` 是先打包再启动开发服务器，而 `Vite` 则是直接启动，然后再按需编译依赖文件。（大家可以启动项目后检查源码 `Sources` 那里看到）

这意味着，当使用 `Webpack` 时，所有的模块都需要在开发前进行打包，这会增加启动时间和构建时间。

而 `Vite` 则采用了不同的策略，它会在请求模块时再进行实时编译，这种按需动态编译的模式极大地缩短了编译时间，特别是在大型项目中，文件数量众多，`Vite` 的优势更为明显。

Webpack启动

Vite启动

2、对ES Modules的支持

现代浏览器本身就支持 `ES Modules`，会 `主动发起` 请求去获取所需文件。`Vite`充分利用了这一点，将开发环境下的模块文件直接作为浏览器要执行的文件，而不是像 `Webpack` 那样 `先打包`，再交给浏览器执行。这种方式减少了中间环节，提高了效率。

什么是ES Modules?

通过使用 `export` 和 `import` 语句，`ES Modules` 允许在浏览器端导入和导出模块。

当使用 `ES Modules` 进行开发时，开发者实际上是在构建一个 `依赖关系图`，不同依赖项之间通过导入语句进行关联。

主流浏览器（除IE外）均支持`ES Modules`，并且可以通过在 `script` 标签中设置 `type="module"` 来加载模块。默认情况下，模块会延迟加载，执行时机在文档解析之后，触发`DOMContentLoaded`事件前。

3、底层语言的差异

`Webpack` 是基于 `Node.js` 构建的，而 `Vite` 则是基于 `esbuild` 进行预构建依赖。`esbuild` 是采用 `Go` 语言编写的，`Go` 语言是 `纳秒` 级别的，而 `Node.js` 是 `毫秒` 级别的。因此，`Vite` 在打包速度上相比`Webpack` 有 `10-100` 倍的提升。

什么是预构建依赖？

预构建依赖通常指的是在项目 `启动或构建` 之前，对项目中所需的依赖项进行预先的 `处理或构建`。这样做的好处在于，当项目实际运行时，可以 `直接使用` 这些已经预构建好的依赖，而无需再进行实时的编译或构建，从而提高了应用程序的运行速度和效率。

4、热更新的处理

在 `Webpack` 中，当一个模块或其依赖的模块内容改变时，需要 `重新编译` 这些模块。

而在 `Vite` 中，当某个模块内容改变时，只需要让浏览器 `重新请求` 该模块即可，这大大减少了热更新的时间。

总结

总的来说，`Vite` 之所以比 `Webpack` 快，主要是因为它采用了 `不同的开发模式`、`充分利用了现代浏览器的 ES Modules 支持`、`使用了更高效的底层语言`，`并优化了热更新的处理`。这些特点使得 `Vite`在大型项目中具有显著的优势，能够快速启动和构建，提高开发效率。

32. web 系统里面，如何对图片进行优化？【热度: 789】

图片作为网页和移动应用中不可或缺的元素之一，对于用户体验和网站性能都有着重要的影响。

加载速度是用户体验的关键因素之一，而大尺寸的图片会增加网页加载时间，导致用户等待时间过长，从而影响用户的满意度和留存率。通过优化图片，我们可以显著减少页面加载时间，提供更快速流畅的使用体验。

图片优化是提升用户体验、提高网站性能、减少流量消耗和增加搜索引擎曝光度的关键因素。为了提供更出色的用户体验，同时也提升网站的性能。总结了一下通用的图片优化首手段。

1. 选择合适的图片格式

以下是对常用的图片格式jpg、png和webp进行深度对比的表格：

特性	JPG	PNG	WebP
压缩算法	有损压缩	无损压缩	有损压缩
透明度	不支持透明度	支持透明度	支持透明度
图片质量	可调整质量	无法调整质量	可调整质量
文件大小	相对较小	相对较大	相对较小
浏览器支持	支持在所有主流浏览器上显示	支持在所有主流浏览器上显示	部分浏览器支持
动画支持	不支持动画	不支持动画	支持动画
兼容性	兼容性较好	兼容性较好	兼容性较差

请注意，这个表格只是对这些格式的一般特征进行了总结，并不代表所有情况。实际情况可能因图像内容、压缩设置和浏览器支持等因素而有所不同。因此，在选择图像格式时，您应根据具体要求和应用场景进行评估和选择。

2. 图片压缩

主要介绍 webpack 对图片进行压缩，可以使用以下步骤：

1. 安装依赖：首先，确保你已经在项目中安装了webpack和相关的loader。可以使用以下命令安装所需的loader：

```
1  npm install --save-dev file-loader image-webpack-loader
```

1. 配置Webpack：在Webpack的配置文件中相关配置。以下是一个简单的示例：

```
1  const path = require('path');
```



```
2
3 module.exports = {
4   entry: 'src/index.js',
5   output: {
6     filename: 'bundle.js',
7     path: path.resolve(__dirname, 'dist')
8   }, module: {
9     rules: [
10      {
11        test: /\. (png|jpe?g|gif)$/i,
12        use: [
13          {
14            loader: 'file-loader',
15            options: {
16              name: '[name].[ext]',
17              outputPath: 'images/'
18            }
19          },
20          {
21            loader: 'image-webpack-loader',
22            options: {
23              mozjpeg: {
24                progressive: true,
25                quality: 65
26              }, // optipng.enabled: false will disable optipng
27              optipng: {
28                enabled: false,
29              },
30              pngquant: {
31                quality: [0.65, 0.90],
32                speed: 4
33              },
34              gifsicle: {
35                interlaced: false,
36              }, // the webp option will enable WEBP
37              webp: {
38                quality: 75
39              }
40            }
41          }
42        ]
43      }
44    ]
45  }
46 };
```

上述配置中，我们使用 `file-loader` 将图片复制到输出目录，并使用 `image-webpack-loader` 对图片进行压缩和优化。

1. 运行Webpack：现在，当你运行Webpack时，它将自动使用 `image-webpack-loader` 对匹配到的图片进行压缩和优化。压缩后的图片将被复制到输出目录中。

2. 雪碧图

Web图片优化的雪碧图（CSS Sprites）是一种将多个小图片合并为一个大图片的技术。通过将多个小图片合并成一张大图片，可以减少浏览器发送的请求次数，从而提高页面加载速度。

雪碧图的原理是通过CSS的 `background-image` 和 `background-position` 属性，将所需的小图片显示在指定的位置上。这样，只需加载一张大图，就可以显示多个小图片，减少了网络请求的数量，提高了页面加载速度。

听上去好像很麻烦，实际上可以使用 webpack 插件 `webpack-spritesmith` 完成自动化处理雪碧图合成，我们在使用过程中正常使用即可。

以下是使用 `webpack-spritesmith` 插件来自动处理雪碧图的步骤：

1. 安装插件：使用npm或yarn安装 `webpack-spritesmith` 插件。

```
1 npm install webpack-spritesmith --save-dev
```

1. 配置Webpack：在Webpack配置文件中，引入 `webpack-spritesmith` 插件，并配置相应的选项。

```
1  const SpritesmithPlugin = require('webpack-spritesmith');
2
3  module.exports = { // ...其他配置
4    plugins: [new SpritesmithPlugin({
5      src: {
6        cwd: path.resolve(__dirname, 'path/to/sprites'), // 需要合并的小图片所在
        的目录
7        glob: '*.png' // 小图片的文件名格式
8      },
9      target: {
10        image: path.resolve(__dirname, 'path/to/output/sprite.png'), // 生成的
        雪碧图的路径和文件名
11        css: path.resolve(__dirname, 'path/to/output/sprite.css') // 生成的CSS
        样式表的路径和文件名
12      },
13      apiOptions: {
14        cssImageRef: 'path/to/output/sprite.png' // CSS样式表中引用雪碧图的路径
15      }
16    })
```



```
17     ]
18 }
```

1. 使用雪碧图：在HTML中，使用生成的CSS样式类来显示相应的小图片。Webpack会自动处理雪碧图的合并和CSS样式的生成。例如：

然后，你可以按照以下方法在CSS中引用雪碧图：

CSS方式：

```
1  div {background: url(path/to/output/sprite.png) no-repeat;
2  }.icon-facebook {/* 设置小图标在雪碧图中的位置和大小 */
3      width: 32px;height: 32px;background-position: 0 0; /* 该小图标在雪碧图中的位置 */
4  }.icon-twitter {width: 32px;height: 32px;background-position: -32px 0; /* 该小图标在雪碧图中的位置 */
5  }.icon-instagram {width: 32px;height: 32px;background-position: -64px 0; /* 该小图标在雪碧图中的位置 */
6  }
```

在HTML中，你可以像下面这样使用对应的CSS类来显示相应的小图标：

```
1  <div class="icon icon-facebook">/div<div class="icon icon-twitter">/div<div
   class="icon icon-instagram">/div
```

这样，Webpack会根据配置自动处理雪碧图，并生成对应的雪碧图和CSS样式表。CSS中的 `background` 属性会引用生成的雪碧图，并通过 `background-position` 来指定显示的小图标在雪碧图中的位置。

确保在CSS中指定了每个小图标在雪碧图中的位置和大小，以便正确显示。

使用Webpack自动处理雪碧图可以简化开发流程，并且可以根据需要自定义配置。`webpack-spritesmith` 是一个常用的Webpack插件，可以帮助自动处理雪碧图。

2. 图标类型资源推荐使用 iconfont

如果你有很多图标类型的图片资源，并且想使用 `iconfont` 来处理这些资源，可以按照以下步骤进行处理：

- 获取图标资源：首先，你需要获取你想要的图标资源。你可以从 `iconfont` 网站或其他图标库中选择和下载符合需求的图标。这个没有啥好说的，直接推荐：<https://www.iconfont.cn/>
- 生成字体文件：接下来，你需要将这些图标转换成字体文件。你可以使用 `iconfont` 提供的在线转换工具，将图标文件上传并生成字体文件（包括 `.ttf`、`.eot`、`.woff` 和 `.svg` 格式）。

- 引入字体文件：将生成的字体文件下载到本地，并在你的项目中引入。通常，你需要在CSS文件中通过 `@font-face` 规则引入字体文件，并为字体定义一个唯一的名称。
- 使用图标：一旦字体文件引入成功，你可以在CSS中通过设置 `content` 属性来使用图标。每个图标都会有一个对应的Unicode代码，你可以在 `iconfont` 提供的网站或字体文件中找到对应图标的Unicode代码，并通过设置 `content` 属性的值为该Unicode代码来使用图标。

以下是一个简单的示例，以帮助你更好地理解：

```
1  @font-face {font-family: 'iconfont';src: url('path/to/iconfont.eot'); /* 引入字体文件 */
2      /* 其他格式的字体文件 */
3  }.icon {font-family: 'iconfont'; /* 使用定义的字体名称 */
4      font-size: 16px; /* 图标大小 */
5      line-height: 1; /* 图标行高 */
6  }.icon-facebook::before {content: '\e001'; /* 使用Unicode代码表示想要显示的图标 */
7  }.icon-twitter::before {content: '\e002'; /* 使用Unicode代码表示想要显示的图标 */
8  }.icon-instagram::before {content: '\e003'; /* 使用Unicode代码表示想要显示的图标 */
9  }
```

在上述示例中，我们首先通过 `@font-face` 引入了字体文件，并为字体定义了一个名称 `iconfont`。然后，我们使用该名称作为 `font-family` 属性的值，以便在 `.icon` 类中使用该字体。最后，我们通过在 `::before` 伪元素中设置 `content` 属性为图标的Unicode代码，来显示相应的图标。

在HTML中，你可以像下面这样使用对应的CSS类来显示相应的图标：

```
1  <span class="icon icon-facebook"></span><span class="icon icon-twitter">
    </span><span class="icon icon-instagram"></span>
```

通过上述步骤，你可以使用 `iconfont` 来处理你的图标资源，并在项目中方便地使用它们。确保在CSS中设置了图标的字体大小和行高，以便正确显示图标。

3. 使用 base64 格式

实际开发过程中，为何会考虑 base64？

使用Base64图片的优势有以下几点：

- 减少HTTP请求数量：通常情况下，每个网页都需要加载多张图片，因此会发送多个HTTP请求来获取这些图片文件。使用Base64图片可以将图片数据嵌入到CSS或HTML文件中，减少了对服务器的请求次数，从而提高网页加载速度。

- 减少图片文件的大小：Base64是一种编码方式，可以将二进制数据转换成文本字符串。通过使用Base64，可以将图片文件转换成文本字符串，并将其嵌入到CSS或HTML文件中。相比于直接引用图片文件，Base64编码的字符串通常会更小，因此可以减少图片文件的大小，从而减少了网页的总体积，加快了网页加载速度。
- 简化部署和维护：将图片数据嵌入到CSS或HTML文件中，可以减少文件的数量和复杂性，使得部署和维护变得更加简单和方便。此外，也不需要处理图片文件的路径和引用相关的问题。
- 实现一些特殊效果：通过Base64图片，可以实现一些特殊的效果，例如页面背景渐变、图标的使用等。这样可以避免使用额外的图片文件，简化了开发过程。

上面虽然说饿了挺多有点，但是劣势也是很明显：

- 增加了文本文件的体积：因为Base64编码将二进制数据转换成文本字符串，所以会增加CSS或HTML文件的体积。在图片较大或数量较多时，这可能会导致文件变得庞大，从而导致网页加载速度变慢。
- 缓存问题：由于Base64图片被嵌入到了CSS或HTML文件中，如果图片内容有更新，那么整个文件都需要重新加载，而无法使用缓存。相比于独立的图片文件，Base64图片对缓存的利用效率较低。

使用Base64图片在一些特定的场景下可以提供一些优势，但也需要权衡其带来的一些缺点。在实际开发中，可以根据具体的需求和情况，选择是否使用Base64图片。所以建议复用性很强, 变更率较低，且小于 10KB 的图片文件，可以考虑 base64

如何使用？有要介绍一下 webpack 插件了： `url-loader` 或 `file-loader`

要使用Webpack将图片自动转换为Base64编码，您需要执行以下步骤：

1. 安装依赖：首先，确保您已经安装了 `url-loader` 或 `file-loader`，它们是Webpack的两个常用的加载器。

```
1 npm install url-loader --save-dev
```

1. 配置Webpack：在Webpack的配置文件中，添加对图片文件的处理规则。您可以在 `module.rules` 数组中添加一个新的规则，以匹配图片文件的后缀。

```
1 module.exports = { // ...module: {
2   rules: [ // ...
3     {
4       test: /\. (png|jpe?g|gif)$/i,
5       use: [
6         {
7           loader: 'url-loader',
8           options: {
9             limit: 8192, // 设置图片大小的阈值，小于该值的图片会被转为Base64
10            outputPath: 'images', // 输出路径
```

```
11         publicPath: 'images', // 资源路径
12     },
13 },
14 ],
15 },
16 ],
17 }, // ...
18 };
```

在上面的示例中，配置了一个处理 `png`、`jpeg`、`jpg` 和 `gif` 格式图片的规则。使用 `url-loader` 加载器，并设置了一些选项，例如 `limit` 限制了图片大小的阈值，小于该值的图片将会被转换为Base64编码。

1. 在代码中引用图片：在您的代码中，可以像引用普通图片一样引用图片文件，Webpack会根据配置自动将其转换为Base64编码。

```
1  import imgSrc from './path/to/image.png';
2
3  const imgElement = document.createElement('img');
4  imgElement.src = imgSrc;
5  document.body.appendChild(imgElement);
```

1. 构建项目：最后，使用Webpack构建项目，它会根据配置自动将符合规则的图片文件转换为Base64编码，并将其嵌入到生成的输出文件中。

```
1  npx webpack
```

这样，Webpack就会自动将图片转换为Base64编码，并将其嵌入到生成的输出文件中。请注意，在使用Base64图片时，需要权衡文件大小和性能，适度使用Base64编码，避免过大的文件导致网页加载变慢。

2. 使用 CDN 加载图片

CND 加载图片优势非常明显：

- 加速网页加载速度：CDN通过将图片资源分布在全球的多个节点上，使用户能从离自己最近的节点获取资源，从而大大减少了网络延迟和加载时间。这可以提高网页的加载速度和用户体验。
- 减轻服务器负载：CDN充当了一个缓冲层，当用户请求图片资源时，CDN会将图片资源从源服务器获取并缓存在节点中，下次再有用户请求同一资源时，CDN会直接从节点返回，减少了对源服务器的请求，分担了服务器的负载。

- 提高并发性能：CDN节点分布在不同地区，用户请求图片资源时可以从离他们最近的节点获取，这可以减少网络拥塞和并发请求，提高了并发性能。
- 节省带宽成本：CDN的节点之间会自动选择最优路径，有效利用了带宽资源，减少了数据传输的成本，尤其在大量图片资源请求时，能够带来显著的成本节省。
- 提供高可用性：CDN通过分布式存储和负载均衡技术，提供了高可用性和容错能力。即使某个节点或源服务器发生故障，CDN会自动切换到其他可用节点，确保用户能够正常访问图片资源。

总之，使用CDN加载图片可以提高网页加载速度、降低服务器负载、提高并发性能、节省带宽成本，并提供高可用性，从而改善用户体验和网站性能。

3. 图片懒加载

图片懒加载是一种在网站或应用中延迟加载图片的技术。它的主要目的是减少页面的初始加载时间，并提高用户的浏览体验。

- 原理：图片懒加载的原理是只在用户需要时加载图片，而不是在页面初始加载时全部加载。这通常通过将图片的真实地址存储在自定义属性（例如 `data-src`）中，而不是在 `src` 属性中。然后，在图片进入浏览器视图时，通过JavaScript动态将 `data-src` 的值赋给 `src` 属性，触发图片的加载。
- 优势：图片懒加载可以显著减少初始页面的加载时间，特别是当页面中有大量图片时。它使页面加载变得更快，提高了用户的浏览体验。此外，懒加载还可以节省带宽和减轻服务器负载，因为只有当图片进入视图时才会加载。
- 实现方法：图片懒加载可以通过纯JavaScript实现，也可以使用现成的JavaScript库，如 `LazyLoad.js`、`Intersection Observer API`等。这些库提供了方便的API和配置选项，可以自定义懒加载的行为和效果。
- 最佳实践：在使用图片懒加载时，可以考虑一些最佳实践。例如，设置一个占位符或加载中的动画，以提供更好的用户体验。另外，确保在不支持JavaScript的情况下仍然可用，并为可访问性提供替代文本（`alt`属性）。此外，对于移动设备，可以考虑使用响应式图片来适应不同的屏幕分辨率。

实现举例：

图片懒加载可以延迟图片的加载，只有当图片即将进入视口范围时才进行加载。这可以大大减轻页面的加载时间，并降低带宽消耗，提高了用户的体验。以下是一些常见的实现方法：

1. Intersection Observer API

`Intersection Observer API` 是一种用于异步检查文档中元素与视口叠加程度的API。可以将其用于检测图片是否已经进入视口，并根据需要进行相应的处理。

```
1 let observer = new IntersectionObserver(function (entries) {
2   entries.forEach(function (entry) {if (entry.isIntersecting) {const
   lazyImage = entry.target;
3     lazyImage.src = lazyImage.dataset.src;
```

```

4      observer.unobserve(lazyImage);
5    }
6  });
7 });
8
9  const lazyImages = [...document.querySelectorAll(".lazy")];
10 lazyImages.forEach(function (image) {
11   observer.observe(image);
12 });

```

1. 自定义监听器

或者，可以通过自定义监听器来实现懒加载。其中，应该避免在滚动事件处理程序中频繁进行图片加载，因为这可能会影响性能。相反，使用自定义监听器只会在滚动停止时进行图片加载。

```

1  function lazyLoad() {const images = document.querySelectorAll(".lazy");const
    scrollTop = window.pageYOffset;
2    images.forEach((img) => {if (img.offsetTop < window.innerHeight +
    scrollTop) {
3      img.src = img.dataset.src;
4      img.classList.remove("lazy");
5    }
6  });
7  }
8
9  let lazyLoadThrottleTimeout;
10 document.addEventListener("scroll", function () {if (lazyLoadThrottleTimeout)
    {
11     clearTimeout(lazyLoadThrottleTimeout);
12   }
13   lazyLoadThrottleTimeout = setTimeout(lazyLoad, 20);
14 });

```

在这个例子中，我们使用了 `setTimeout()` 函数来延迟图片的加载，以避免在滚动事件的频繁触发中对性能的影响。

无论使用哪种方法，都需要为需要懒加载的图片设置占位符，并将未加载的图片路径保存在 `data` 属性中，以便在需要进行加载。这些占位符可以是简单的 `div` 或样式类，用于预留图片的空间，避免页面布局的混乱。

```

1  <!-- 占位符示例 --><div class="lazy-placeholder" style="background-color:
    #ddd;height: 500px;"></div<!-- 图片示例 -->

```

2. 图片预加载

图片预加载是一种在网站或应用中提前加载图片资源的技术。它的主要目的是在用户实际需要加载图片之前，将其提前下载到浏览器缓存中。

图片预加载通常是在页面加载过程中或在特定事件触发前异步加载图片资源。通过使用 `JavaScript`，可以在网页DOM元素中创建一个新的 `Image` 对象，并将要预加载的图片的URL赋值给该对象的 `src` 属性。浏览器在加载过程中会提前下载这些图片，并将其缓存起来，以备将来使用。

图片预加载可以使用原生JavaScript实现，也可以使用现成的JavaScript库，如 `Preload.js`、`LazyLoad.js` 等。这些库提供了方便的API和配置选项，可以灵活地控制预加载的行为和效果。

实现图片预加载可以使用原生JavaScript或使用专门的JavaScript库。下面分别介绍两种方式的实现方法：

1. 使用原生JavaScript实现图片预加载：

```
1  function preloadImage(url) {return new Promise(function(resolve, reject) {var
    img = new Image();
2      img.onload = resolve;
3      img.onerror = reject;
4      img.src = url;
5  });
6  }
7
8  // 调用预加载函数
9  preloadImage('image.jpg')
10 .then(function() {console.log('图片加载成功');// 在此处可以执行加载成功后的操作，例
    如显示图片等
11 })
12 .catch(function() {console.error('图片加载失败');// 在此处可以执行加载失败后的操
    作，例如显示错误信息等
13 });
```

在上述代码中，我们定义了一个 `preloadImage` 函数，它使用 `Image` 对象来加载图片资源。通过 `onload` 事件和 `onerror` 事件来监听图片加载完成和加载错误的情况，并使用Promise对象进行异步处理。

1. 使用JavaScript库实现图片预加载：

使用JavaScript库可以更简便地实现图片预加载，并提供更多的配置选项和功能。以下以Preload.js库为例进行说明：

首先，在HTML文件中引入Preload.js库：


```
1 <script src="preload.js">/script
```

然后，在JavaScript代码中使用Preload.js库来进行图片预加载：

```
1 var preload = new createjs.LoadQueue();
2 preload.on("complete", handleComplete);
3 preload.on("error", handleError);
4 preload.loadFile('image.jpg');
5
6 function handleComplete() {console.log('图片加载成功');} // 在此处可以执行加载成功后的
    操作，例如显示图片等
7 }
8
9 function handleError() {console.error('图片加载失败');} // 在此处可以执行加载失败后的
    操作，例如显示错误信息等
10 }
```

在上述代码中，我们首先创建一个 `LoadQueue` 对象，并使用 `on` 方法来监听加载完成和加载错误的事件。然后使用 `loadFile` 方法来指定要预加载的图片资源的URL。

当图片加载完成时，`handleComplete` 函数会被调用，我们可以在此处执行加载成功后的操作。当图片加载错误时，`handleError` 函数会被调用，我们可以在此处执行加载失败后的操作。

以上是两种常用的实现图片预加载的方法，根据具体需求和项目情况选择合适的方式来实现图片预加载。

2. 响应式加载图片

要在不同分辨率的设备上显示不同尺寸的图片，你可以使用 `<picture>` 元素和 `<source>` 元素来实现响应式图片。以下是一个示例：

```
1 <picture><source media="(min-width: 1200px)" srcset="large-image.jpg"><source
    media="(min-width: 768px)" srcset="medium-image.jpg"><source srcset="small-
    image.jpg">/picture
```

在上面的示例中，`<picture>` 元素内部有多个 `<source>` 元素，每个 `<source>` 元素通过 `srcset` 属性指定了对应分辨率下的图片链接。`media` 属性可以用来指定在哪个分辨率下应用对应的图片。如果没有任何 `<source>` 元素匹配当前设备的分辨率，那么就会使用 `<img>` 元素的 `src` 属性指定的图片链接。

可以根据不同分辨率的设备，提供不同尺寸和质量的图片，以优化用户的视觉体验和页面加载性能。

有可以使用 webpack `responsive-loader` 来实现自动根据设备分辨率加载不同的倍图：

依赖安装:

```
1 npm install responsive-loader sharp --save-dev
```

webpack 配置示范

```
1 module.exports = {
2   entry: {...},
3   output: {...},module: {
4     rules: [
5       {
6         test: /\.?(jpe?g|png|webp)$/i,
7         use: [
8           {
9             loader: "responsive-loader",
10            options: {
11              adapter: require('responsive-loader/sharp'),
12              sizes: [320, 640, 960, 1200, 1800, 2400],
13              placeholder: true,
14              placeholderSize: 20
15            },
16          },
17        ],
18      }
19    ]
20  },
21 }
```

在CSS中使用它(如果使用多个大小, 则只使用第一个调整大小的图像)

```
1 .myImage {background: url('myImage.jpg?size=1140');
2 }@media (max-width: 480px) {.myImage {background: url('myImage.jpg?
  size=480');}}
3 }
```

导入图片到 JS 中:

```
1 import responsiveImage from 'img/myImage.jpg?
  sizes[]=300,sizes[]=600,sizes[]=1024,sizes[]=2048';
```

```

2  import responsiveImageWebp from 'img/myImage.jpg?
   sizes[]=300,sizes[]=600,sizes[]=1024,sizes[]=2048&format=webp';
3
4  // Outputs// responsiveImage.srcSet => '2fefae46cb857bc750fa5e5eed4a0cde-
   300.jpg 300w,2fefae46cb857bc750fa5e5eed4a0cde-600.jpg
   600w,2fefae46cb857bc750fa5e5eed4a0cde-600.jpg 600w ...'//
   responsiveImage.images => [{height: 150, path:
   '2fefae46cb857bc750fa5e5eed4a0cde-300.jpg', width: 300}, {height: 300, path:
   '2fefae46cb857bc750fa5e5eed4a0cde-600.jpg', width: 600} ...]//
   responsiveImage.src => '2fefae46cb857bc750fa5e5eed4a0cde-2048.jpg'//
   responsiveImage.toString() => '2fefae46cb857bc750fa5e5eed4a0cde-2048.jpg'
5  ...
6    <picture><source srcSet={responsiveImageWebp.srcSet} type='image/webp'
   sizes='(min-width: 1024px) 1024px, 100vw' />
7    <img
8      src={responsiveImage.src}
9      srcSet={responsiveImage.srcSet}
10     width={responsiveImage.width}
11     height={responsiveImage.height}
12     sizes='(min-width: 1024px) 1024px, 100vw'
13     loading="lazy"
14   />
15   </picture>
16   ...

```

3. 渐进式加载图片

实现渐进式加载的主要思想是先加载一张较低分辨率的模糊图片，然后逐步加载更高分辨率的图片。

下面是实现渐进式加载图片的一般步骤：

1. 创建一张模糊的低分辨率图片。可以使用图片处理工具将原始图片进行模糊处理，或者使用低分辨率的缩略图作为初始图片。
2. 使用 `<img>` 标签将低分辨率的图片设置为 `src` 属性。这将立即加载并显示这张低分辨率的图片。
3. 在加载低分辨率图片时，同时加载高分辨率的原始图片。可以将高分辨率图片的URL设置为 `data-src` 等自定义属性，或者使用JavaScript动态加载高清图片。
4. 使用JavaScript监听图片的加载事件，在高分辨率图片加载完成后，将其替换低分辨率图片的 `src` 属性，以实现渐进式加载的效果。

下面是一个示例代码，演示了如何实现渐进式加载图片：

```

1  <!-- HTML --><script
2  // JavaScript
3  const image = document.querySelector('img');

```

```
4
5 // 监听高分辨率图片加载完成事件
6 image.addEventListener('load', () => { // 替换低分辨率图片的src属性
7     image.src = image.dataset.src;
8 });
9 </script>
```

在上面的示例中，一开始会显示一张模糊的低分辨率图片，然后在高分辨率图片加载完成后，将其替换为高分辨率图片，实现了渐进式加载的效果。

渐进式加载图片可以减少用户等待时间，提供更好的用户体验。然而，需要注意的是，为了实现渐进式加载，需要额外加载高分辨率的图片，这可能会增加页面加载时间和网络带宽消耗。因此，开发者需要在性能和用户体验之间进行权衡，并根据实际情况进行选择和优化。

33. [webpack] webpack-dev-server 为何不适用于线上环境？【热度: 88】

webpack-dev-server并不适用于线上环境的原因有以下几点：

1. 性能问题：webpack-dev-server是一个开发服务器，它将编译后的文件保存在内存中，并提供给浏览器访问。这种方式在开发阶段可以提供快速的编译和热模块替换，但在线上环境中，内存中保存的文件无法持久化，每次启动服务器都需要重新编译和打包，这会影响性能，并且占用服务器资源。
2. 安全问题：webpack-dev-server并不进行代码压缩和混淆，源代码是以明文形式传输给浏览器的。这将使得攻击者可以很容易地获取到项目的源代码，从而可能导致安全漏洞。
3. 缺少文件输出：webpack-dev-server并没有生成最终的打包文件，它只是将打包后的文件保存在内存中，提供给浏览器访问。这样就无法进行资源的版本管理和持久化存储，无法做到长期的缓存和优化。

34. [webpack] webpack-dev-server 作用是啥？【热度: 387】

webpack-dev-server 是一个开发服务器，它提供了一个快速开发的环境，并且配合Webpack使用。它的作用主要有以下几个方面：

1. 自动编译和打包：webpack-dev-server 可以监听源文件的变化，当文件发生改动时，它会自动重新编译和打包，保证开发过程中始终使用最新的代码。
2. 热模块替换（Hot Module Replacement）：webpack-dev-server 支持热模块替换，即在开发过程中，当某个模块发生变化时，只会重新编译该模块，而不是整个应用，然后将变更的模块替换到浏览器中，从而实现实时更新，无需手动刷新页面。

3. 代理和反向代理：webpack-dev-server 可以配置代理，用于解决前端开发中跨域请求的问题。可以将某些请求转发到其他服务器，或者改变请求的路径。
4. 路由处理：webpack-dev-server 也支持将所有请求重定向到特定的 HTML 文件，用于前端单页应用的路由处理，可以通过配置实现 SPA（Single Page Application）的路由。
5. 静态文件服务：webpack-dev-server 可以将打包后的文件提供给浏览器访问，可以通过配置指定静态文件的路径，并且通过指定的端口提供服务。

总结来说，webpack-dev-server 提供了一个方便的开发环境，可以实时编译和打包代码，并且支持热模块替换，代理和反向代理，路由处理等功能，提高了开发效率和开发体验。

webpack-dev-server 有编译和打包的能力？

webpack-dev-server 本身并没有编译和打包的能力，它是使用 Webpack 来实现编译和打包的。

webpack-dev-server 是基于 Webpack 的一个开发服务器，它通过监听源文件的变化，自动调用 Webpack 进行编译和打包，并将打包后的文件提供给浏览器访问。

这样可以在开发过程中实时更新代码，无需手动进行编译和打包操作。

webpack-dev-server 还支持热模块替换等功能，提供了一个方便的开发环境。

但是需要注意的是，webpack-dev-server 只适用于开发阶段，它并不会生成最终的打包文件，而是将打包后的文件保存在内存中，提供给浏览器访问。

在真正发布项目时，还需要运行 Webpack 的打包命令生成最终的打包文件。

35. vite 和 webpack 在热更新上有啥区别？【热度: 530】

Vite 和 Webpack 在热更新上有一些区别：

1. 模块级别的热更新：Vite 使用浏览器原生的 ES 模块系统，可以实现模块级别的热更新，即只更新修改的模块，而不需要刷新整个页面。这样可以提供更快的发展迭代速度。而在 Webpack 中，热更新是基于文件级别的，需要重新构建并刷新整个页面。
2. 开发环境下的无构建：Vite 在开发环境下不会对代码进行打包构建，而是直接利用浏览器原生的模块导入功能，通过 HTTP 服务器提供模块的即时响应。这样可以避免构建和重新编译的时间，更快地反映出代码的修改。而在 Webpack 中，每次修改代码都需要重新构建和编译，耗费一定的时间。
3. 构建环境下的优化：尽管 Vite 在开发环境下不进行打包构建，但在生产环境下，它会通过预构建的方式生成高性能的静态资源，以提高页面加载速度。而 Webpack 则通过将所有模块打包成 bundle 文件，进行代码压缩和优化，以及使用各种插件和配置来优化构建结果。

总的来说，Vite 在热更新上比 Webpack 更加快速和精细化，能够在开发过程中提供更好的开发体验和更快的反馈速度。但是，Webpack 在构建环境下有更多的优化和功能，适用于更复杂的项目需求。

以下是 Vite 和 Webpack 在热更新方面的对比表格：

特点	Vite	Webpack
实时热更新	支持模块级别的热更新，即只更新修改的模块，无需刷新整个页面	支持文件级别的热更新，修改任何文件都会触发整个应用的重新构建和刷新
构建速度	在开发环境下，利用浏览器原生的模块导入功能，不需要进行打包构建，启动速度更快	需要进行打包构建，每次修改代码都需要重新构建和编译，相对较慢
开发体验	提供更好的开发体验，修改代码后快速反馈，无需等待全量构建	反馈速度较慢，需要等待每次构建和编译完成
适用场景	适用于中小型项目，追求开发效率的前端项目	适用于大型项目，有复杂需求和更多构建优化的前端项目

36. esbuild 和 rollup 都是 vite 的基础依赖， 那么他们有啥不同？ 【热度: 129】

- esbuild 和 Rollup 都是 Vite 的基础依赖，但它们在 Vite 中担负着不同的角色和任务。
1. esbuild：esbuild 是一个快速、可扩展的 JavaScript 打包器，它被用作 Vite 的默认构建工具。esbuild 的主要任务是将源代码转换为浏览器可以理解的代码，同时还支持压缩、代码分割、按需加载等功能。esbuild 利用其高性能的构建能力，实现了快速的开发服务器和热模块替换。
 2. Rollup：Rollup 是一个 JavaScript 模块打包工具，也是 Vite 的另一个基础依赖。在 Vite 中，Rollup 主要用于生产构建阶段。它通过静态分析模块依赖关系，将多个模块打包为一个或多个最终的输出文件。Rollup 支持多种输出格式，如 ES 模块、CommonJS、UMD 等，可以根据项目的需要进行配置。

尽管 esbuild 和 Rollup 都是 Vite 的基础依赖，但它们的分工是不同的。esbuild 用于开发服务器阶段，通过实时编译和提供模块来实现快速的冷启动和热模块替换。而 Rollup 用于生产构建阶段，将源代码打包为最终可发布的文件，以用于部署到生产环境。这样的分工使得 Vite 在开发过程中能够快速响应变化，并在构建过程中生成高效的最终输出文件。

37. vite 编译器有啥特点？【热度: 237】

Vite 是一个基于现代浏览器原生 ES 模块导入功能的开发工具和构建系统。与传统的打包工具相比，Vite 具有以下几个特点：

1. 快速冷启动：Vite 采用了一种新的开发服务器，利用浏览器原生的 ES 模块导入功能，无需提前构建和打包，可以实现快速的冷启动，并在浏览器中按需编译和加载代码。这种特性使得开发时的重新加载速度非常快，提高了开发效率。
2. 按需编译：Vite 通过解析导入的模块路径，只编译当前需要的模块，而不是像传统的打包工具一样对整个项目进行全量编译。这种按需编译的方式减少了不必要的重复编译和构建时间，提高了构建速度。
3. 零配置：Vite 提供了一种零配置的开发体验，无需繁琐的配置文件，可以快速开始项目的开发。Vite 默认支持常见的前端开发场景，如 Vue、React、TypeScript 等，开发者可以通过简单的配置进行个性化定制。
4. 原生 ES 模块支持：Vite 利用浏览器原生的 ES 模块导入功能，可以直接在浏览器中引入 ES 模块，无需经过任何编译和转换，提供了更好的开发体验和更高的性能。
5. 插件化：Vite 的构建系统采用了插件化的架构，开发者可以根据需求选择和配置不同的插件来扩展 Vite 的功能。Vite 提供了丰富的插件生态系统，使得开发者可以定制化地满足项目需求。

38. vite 编译器的组成部分【热度: 335】

Vite 编译器的主要组成部分包括：

1. esbuild：一个快速的 JavaScript 打包器，用于在开发阶段进行实时编译。esbuild 提供了快速的冷启动和热模块替换功能，能够极大地加快开发环境的构建速度。
2. Rollup：一个强大的 JavaScript 模块打包器，在生产构建阶段使用。Rollup 能够将源代码打包为最终可发布的文件，支持代码分割、Tree Shaking 等优化技术，生成更小、更高效的代码包。
3. 前端开发服务器：Vite 还提供了一个内置的开发服务器，用于提供开发环境下的静态文件服务和构建工具集成。这个服务器能够利用 esbuild 实现快速的编译和热模块替换，使开发者在开发过程中可以快速地预览和调试代码。
4. 插件系统：Vite 通过插件系统来扩展其功能。开发者可以编写自定义的插件，用于处理特定的文件类型、引入额外的功能或者定制构建过程。插件系统使得 Vite 能够与各种前端框架和工具集成，并提供更灵活的开发体验。

39. vite 涉及到了哪些底层原理？【热度: 510】

Vite 涉及到以下几个底层原理：

1. ES 模块：Vite 使用了 ES 模块来管理和加载模块。ES 模块是 JavaScript 的标准模块系统，相比于传统的 CommonJS 或 AMD，ES 模块具有更好的静态分析能力和更高的性能。Vite 通过使用浏览器原生的 ES 模块加载器，可以实现按需加载和快速构建。
2. HTTP/2：Vite 借助于现代浏览器的 HTTP/2 支持来实现更高效的资源加载。HTTP/2 支持多路复用，可以同时请求多个资源，避免了传统的 HTTP/1 中的队头阻塞问题，加快了资源加载速度。
3. 编译器：Vite 使用了自定义的编译器来处理开发时的模块解析和转换。它能够识别模块的依赖关系，并将模块转换为浏览器可直接执行的代码。Vite 的编译器支持热模块替换（HMR），可以在代码修改时自动更新浏览器中的页面，提高开发效率。
4. 中间件：Vite 使用了基于 Koa 框架的中间件来处理开发服务器。通过中间件，Vite 可以拦截和处理开发时的 HTTP 请求，并根据请求的路径返回相应的模块文件。中间件还可以处理各种开发时的特殊需求，如代理 API 请求、路由转发等。

Vite 基于 ES 模块、HTTP/2、自定义编译器和中间件等底层原理，实现了快速的模块加载和开发体验。这些原理的运用使得 Vite 在开发环境下能够提供更快的构建速度和更好的开发体验。

40. 页面加载速度提升（性能优化）应该从哪些反向来思考？【热度: 1,099】

页面加载优化

「页面加载链路+流程优化+协作方」的多级分类思考

- 页面启动
 - service worker 缓存重要的静态资源
 - 页面保活
- 资源加载
 - 网络连接
 - NDS
 - 减少 NDS 解析
 - NDA 预解析
 - HTTP
 - 开启 HTTP2 多路复用
 - 优化核心请求链路

- HTML 加载
 - 内容优化
 - 代码压缩
 - 代码精简(tailwindcss)
 - 服务端渲染 SSG
 - 流程优化
 - 服务端渲染 SSR
 - 流式渲染
 - 预渲染
- 静态资源加载
 - 内容优化
 - JS、CSS 代码压缩
 - 均衡资源包体积：复用代码抽离为一份资源打包、同时开启
 - 精简代码
 - 雪碧图
 - 动态图片降质量
 - 动态 polyfill (根据浏览器的支持情况，动态加载需要的 polyfill（填充）脚本)
 - 不常变的资源单独打包
 - 根据浏览器版本打包，高版本浏览器，直接使用 es6 作为输出文件
 - 流程优化
 - 配置前端缓存: 资源、请求
 - 使用 CDN
 - CDN 优化
 - 协调资源加载优先级
 - 动态资源转静态 CDN 链接加载(例如大图片等)
 - 静态资源使用 service worker 离线存储
 - 非首屏资源懒加载
 - 资源和业务请求预加载
 - 微前端加载应用
 - 微组件加载核心模块资源
- 代码执行

- 减少执行
 - 减少重复渲染
 - 大体量计算场景， 尽量使用缓存函数
 - 使用防抖节流
- 速度提升
 - 使用 worker 多线程加速
 - 充分利用异步请求的线下之间来进行核心代码的加载或者执行
 - wasm 处理大量计算场景
 - 渲染高耗时场景， 迁移到 canvas、虚拟 dom 等
 - 动态渲染：动态渲染可视区内容， 例如图片懒加载等；
- 流程优化
 - 非首屏模块， 延迟加载与渲染
 - longtask 任务拆分执行
 - 利用请求闲暇时间， 请求后续页面资源
- 数据获取
 - 内容优化
 - 减少请求、合并请求、BFF
 - 首屏数据使用模板注入到前端应用
 - 流程优化
 - 数据预请求
 - 常量数据缓存
 - 非首屏请求，延迟到首屏加载完成之后请求
 - 请求并行
- 渲染相关
 - 虚拟列表
 - 延迟渲染
 - 减少重绘重排
 - 图片预加载到内存

41. 对比一下 pnpm、npm、yarn 特性【热度: 399】

功能	pnpm	Yarn	npm
工作区支持	✓	✓	✓
隔离的 node_modules	✓ - 默认支持	✓	✓
提升的 node_modules	✓	✓	✓ - 默认支持
自动安装对等依赖	✓	✗	✓
Plug'n'Play	✓	✓ - 默认支持	✗
零安装	✗	✓	✗
修补依赖	✓	✓	✗
管理 Node.js 版本	✓	✗	✗
有一个锁文件	✓ - 使用 pnpm-lock.yaml	✓ - 使用 yarn.lock	✓ - 使用 package-lock.json
覆盖支持	✓	✓ - 通过 resolutions 配置	✓
可寻址存储	✓	✗	✗
动态包执行	✓ - 通过 pnpm dlx	✓ - 通过 yarn dlx	✓ - 通过 npx
副作用缓存	✓	✗	✗
列出许可证	✓ - 通过 pnpm licenses list	✓ - 通过插件	✗

42. [低代码] 代码平台一般渲染是如何设计的？【热度: 399】

渲染设计

渲染核心本质就是：[schema] + [组件] = [页面]

整体架构如下

- 协议层：基于《低代码引擎搭建协议规范》产出的 Schema 作为我们的规范协议。

- 能力层：提供组件、区块、页面等渲染所需的核心能力，包括 Props 解析、样式注入、条件渲染等。
- 适配层：由于我们使用的运行时框架不是统一的，所以统一使用适配层将不同运行框架的差异部分，通过接口对外，让渲染层注册/适配对应所需的方法。能保障渲染层和能力层直接通过适配层连接起来，能起到独立可扩展的作用。
- 渲染层：提供核心的渲染方法，由于不同运行时框架提供的渲染方法是不同的，所以其通过适配层进行注入，只需要提供适配层所需的接口，即可实现渲染。
- 应用层：根据渲染层所提供的方法，可以应用到项目中，根据使用的方法和规模即可实现应用、页面、区块的渲染。

设计模式渲染（Simulator）

设计模式渲染就是将编排生成的《搭建协议》渲染成视图的过程，视图是可以交互的，所以必须要处理好内部数据流、生命周期、事件绑定、国际化等等。

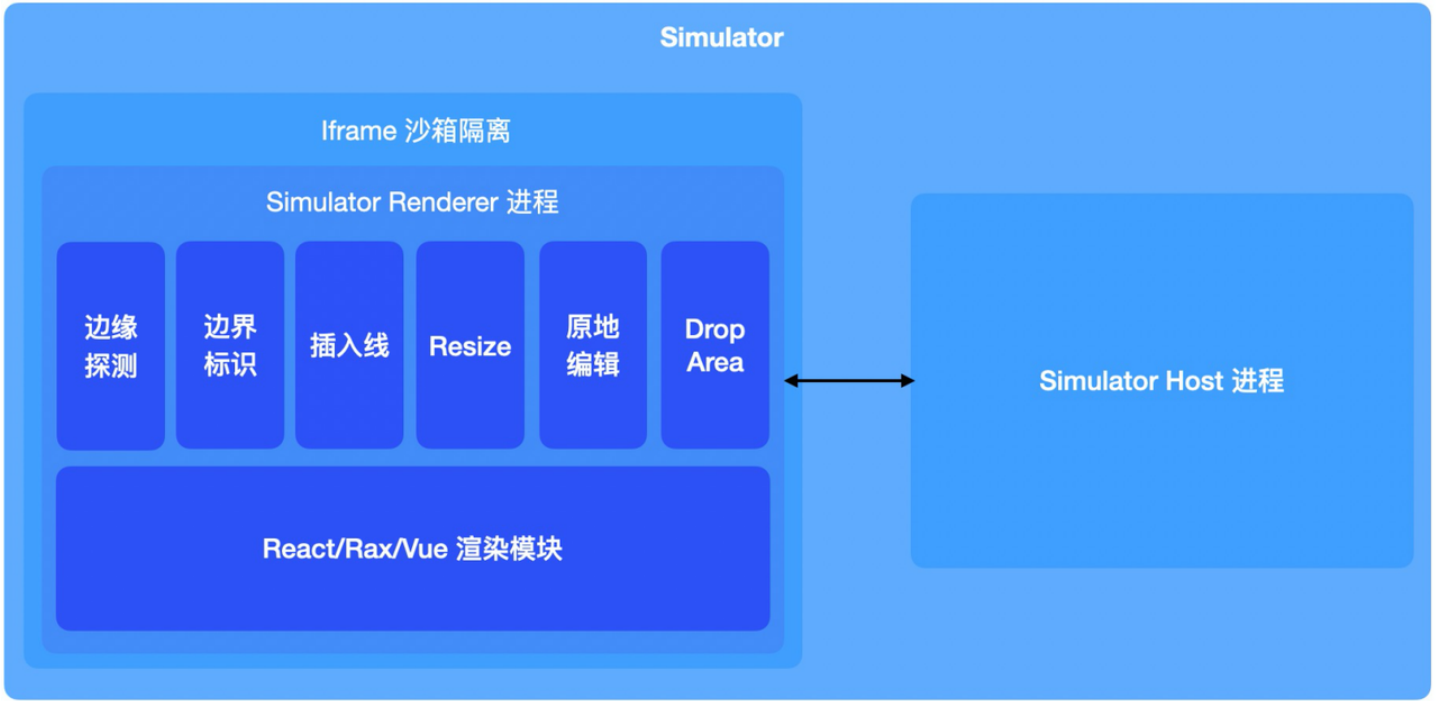
也称为画布的渲染，画布是 UI 编排的核心，它一般融合了页面的渲染以及组件/区块的拖拽、选择、快捷配置。

画布的渲染和预览模式的渲染的区别在于，画布的渲染和设计器之间是有交互的。

所以在这里我们新增了一层 Simulator 作为设计器和渲染的连接器。

Simulator 是将设计器传入的 DocumentModel 和组件/库描述转成相应的 Schema 和 组件类。再调用 Render 层完成渲染。我们这里介绍一下它提供的能力。

- Project：位于顶层的 Project，保留了对所有文档模型的引用，用于管理应用级 Schema 的导入与导出。
- Document：文档模型包括 Simulator 与数据模型两部分。Simulator 通过一份 Simulator Host 协议与数据模型层通信，达到画布上的 UI 操作驱动数据模型变化。通过多文档的设计及多 Tab 交互方式，能够实现同时设计多个页面，以及在一个浏览器标签里进行搭建与配置应用属性。
- Simulator：模拟器主要承载特定运行时环境的页面渲染及与模型层的通信。
- Node：节点模型是对可视化组件/区块的抽象，保留了组件属性集合 Props 的引用，封装了一系列针对组件的 API，比如修改、编辑、保存、拖拽、复制等。
- Props：描述了当前组件所维系的所有可以「设计」的属性，提供一系列操作、遍历和修改属性的方法。同时保持对单个属性 Prop 的引用。
- Prop：属性模型 Prop 与当前可视化组件/区块的某一具体属性想映射，提供了一系列操作属性变更的 API。
- Settings：SettingField 的集合。
- SettingField：它连接属性设置器 Setter 与属性模型 Prop，它是实现多节点属性批处理的关键。
- 通用交互模型：内置了拖拽、活跃追踪、悬停探测、剪贴板、滚动、快捷键绑定。



- 运行时环境：从运行时环境来看，目前我们有 React 生态、Rax 生态。而在对外的历程中，我们也会拥有 Vue 生态、Angular 生态等。
- 布局模式：不同于 C 端营销页的搭建，中后台场景大多是表单、表格，流式布局是主流的选择。对于设计师、产品来说，是需要绝对布局的方式来进行页面研发的。
- 研发场景：从研发场景来看，低代码搭建不仅有页面编排，还有诸如逻辑编排、业务编排的场景。

43. [低代码] 代码平台一般架构设计如何【热度: 517】

关键词：代码平台

代码平台 - 架构综述

分层架构描述

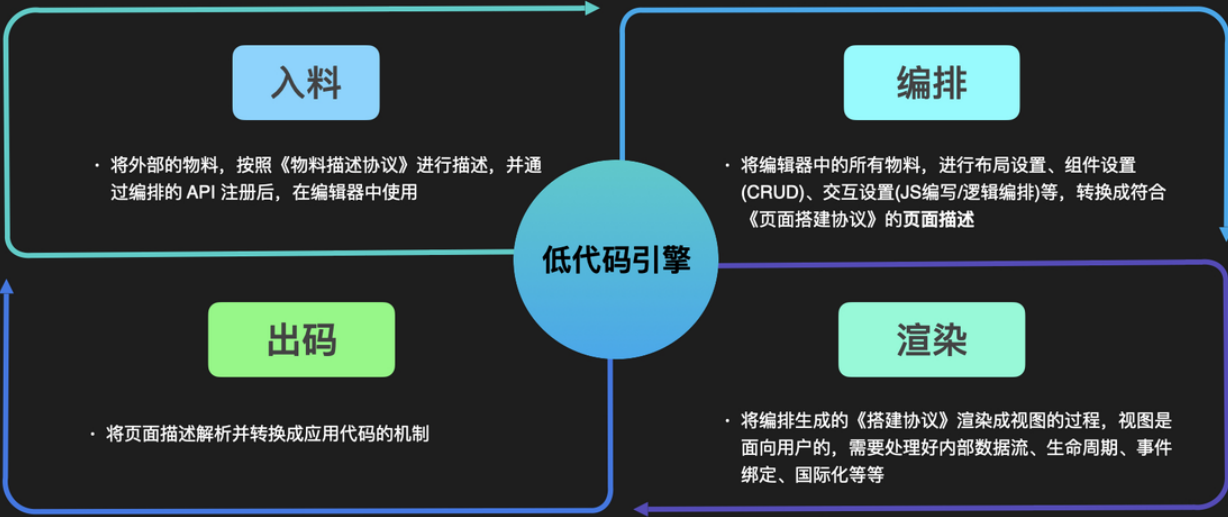


自下而上分别是协议 - 引擎 - 生态 - 平台。

- 底层协议栈定义的是标准，标准的统一让上层产物的互通成为可能。
- 引擎是对协议的实现，同时通过能力的输出，向上支撑生态开放体系，提供各种生态扩展能力。
- 生态就好理解了，是基于引擎核心能力上扩展出来的，比如物料、设置器、插件等，还有工具链支撑开发体系。
- 最后，各个平台基于引擎内核以及生态中的产品组合、衔接形成满足其需求的低代码平台。

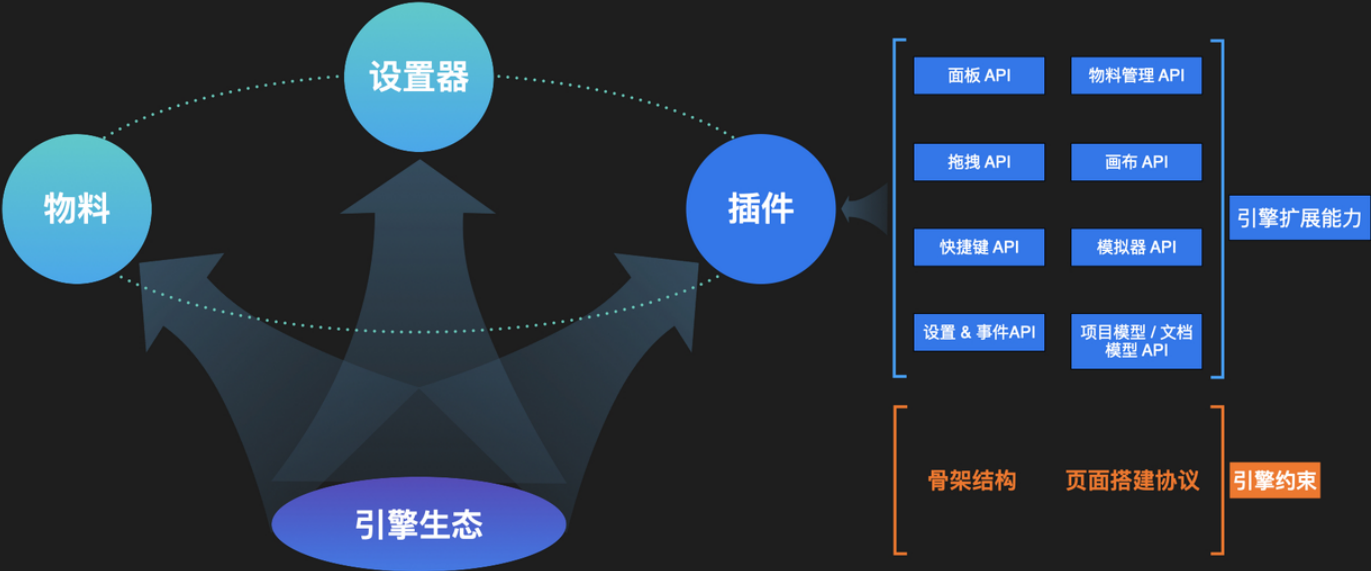
引擎内核简述

阿里低代码引擎的架构 – 引擎内核

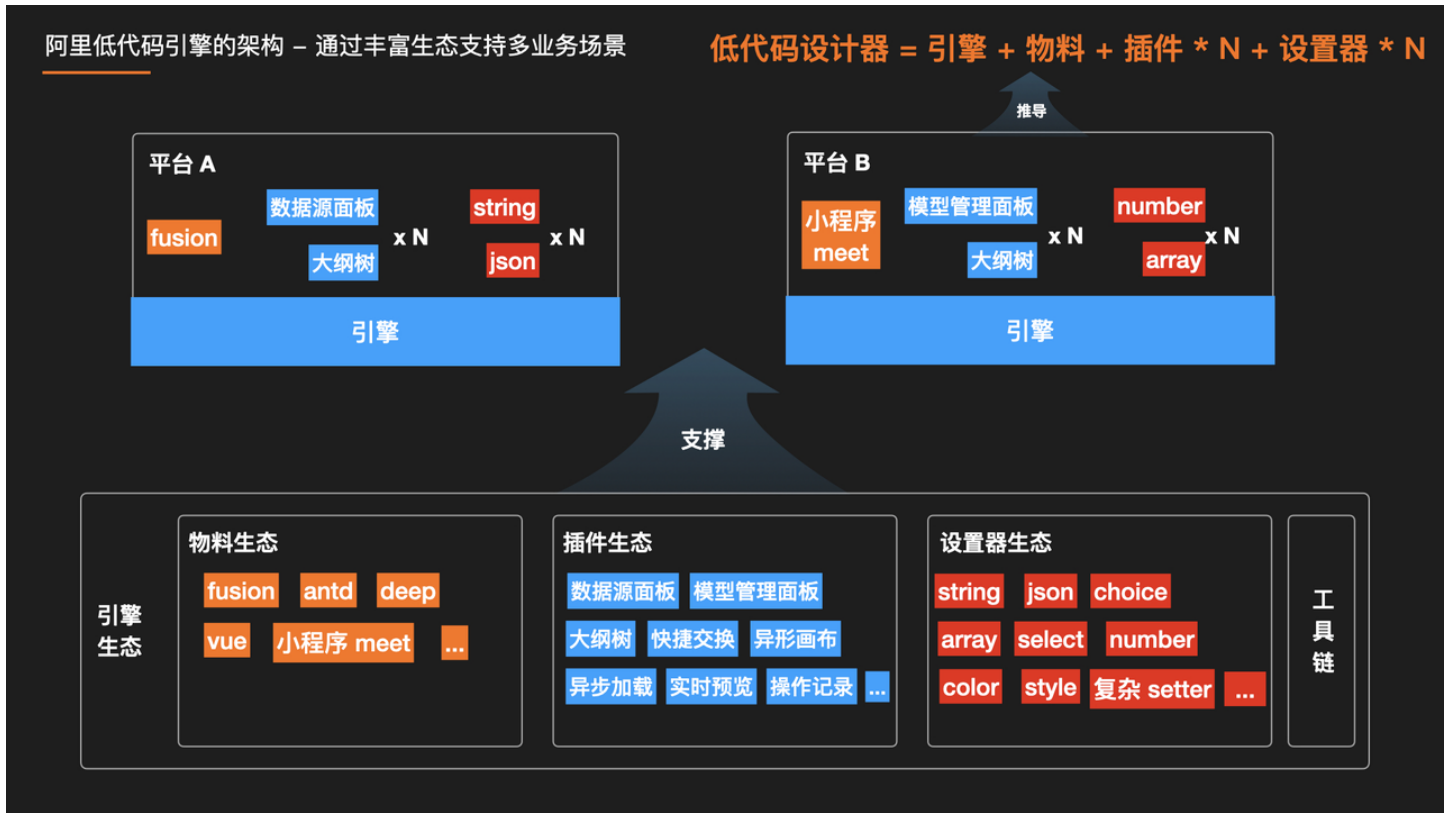


引擎生态简述

阿里低代码引擎的架构 – 引擎生态的设计



调试能力: 工具链 + 调试插件让一切生态元素均可调试, 可组合调, 可线上调



44. [Webpack] Webpack vs Vite的核心差异【热度: 620】

构建速度:

- Webpack: Webpack的构建速度相对较慢，尤其在大型项目中，因为它需要分析整个依赖图，进行多次文件扫描和转译。
- Vite: Vite以开发模式下的极速构建著称。它利用ES模块的特性，只构建正在编辑的文件，而不是整个项目。这使得它在开发环境下几乎是即时的。

开发模式:

- Webpack: Webpack通常使用热模块替换（HMR）来实现快速开发模式，但配置相对复杂。
- Vite: Vite的开发模式非常轻量且快速，支持HMR，但无需额外配置，因为它默认支持。

配置复杂度:

- Webpack: Webpack的配置相对复杂，特别是在处理不同类型的资源和加载器时。
- Vite: Vite鼓励零配置，使得项目起步非常简单，但同时也支持自定义配置，使其适用于复杂项目。

插件生态:

- Webpack: Webpack拥有庞大的插件生态系统，适用于各种不同的需求。
- Vite: Vite也有相当数量的插件，但相对较小，因为它的开发模式和构建方式减少了对一些传统插件的需求。

编译方式:

- Webpack: Webpack使用了多种加载器和插件来处理不同类型的资源，如JavaScript、CSS、图片等。
- Vite: Vite利用ES模块原生支持，使用原生浏览器导入来处理模块，不需要大规模的编译和打包。

应用场景:

- Webpack: 适用于复杂的大型项目，特别是需要大量自定义配置和复杂构建管道的项目。
- Vite: 更适用于小到中型项目，或者需要快速开发原型和小型应用的场景。

打包原理:

- Webpack: Webpack的打包原理是将所有资源打包成一个或多个bundle文件，通常是一个JavaScript文件。
- Vite: Vite的打包原理是保持开发时的模块化结构，使用浏览器原生的导入机制，在生产环境中进行代码分割和优化。

优缺点:

- Webpack:
 - 优点：灵活、强大、适用于复杂场景、庞大的插件生态。
 - 缺点：构建速度较慢、配置复杂、开发体验不如Vite流畅。
- Vite:
 - 优点：极快的开发构建速度、零配置启动、原生ES模块支持、适用于小型项目和快速原型开发。
 - 缺点：插件生态相对较小、不太适用于复杂大型项目。

45. 对 babel 的理解？【热度: 551】

Babel 是一个非常流行的 JavaScript 编译器，用于将最新版本的 ECMAScript 代码转换为向后兼容的 JavaScript 代码，以便在旧版浏览器或环境中运行。

以下是对 Babel 的理解：

1. 语法转换：Babel 可以将使用了最新 ECMAScript 标准的代码转换为被广泛支持的旧版 JavaScript 代码。例如，将使用了箭头函数、解构赋值等语法的代码转换为使用 function 关键字和传统赋值的等效代码。
2. Polyfill：Babel 可以通过添加 Polyfill 来支持新的全局函数、方法和对象，以确保代码在旧版本的浏览器中正常运行。通过使用 Polyfill，Babel 可以在浏览器中模拟缺失的特性，使旧版浏览器能够运行使用了这些特性的代码。

3. 插件系统：Babel 的核心功能可以通过插件进行扩展和定制。Babel 提供了众多的插件，用于实现不同的转换和功能。开发者可以根据自己的需求选择和配置相应的插件，以便将代码转换为特定的目标环境。
4. 预设（Presets）：Babel 提供了预设的概念，可以一次性地配置一组插件，以实现特定的转换目标。预设是一组插件的集合，可以根据应用程序的需要进行选择 and 配置。常见的预设包括 "@babel/preset-env"（根据目标环境自动选择转换规则）和 "@babel/preset-react"（用于处理 React 相关的代码）。
5. 与构建工具的集成：Babel 可以与构建工具（如 webpack、Rollup 等）无缝集成，作为其转换代码的一部分。通过配置构建工具，可以让 Babel 在构建过程中自动处理源代码，将其转换为目标代码。

总之，Babel 是一个功能强大的 JavaScript 编译器，可以将使用最新 ECMAScript 标准的代码转换为向后兼容的 JavaScript 代码，从而在旧版浏览器或环境中运行。通过插件和预设的配置，Babel 提供了高度的灵活性，使开发者能够根据项目需求定制转换规则。

追问：为何要使用 babel

使用 Babel 的主要原因是为了解决 JavaScript 代码的兼容性问题。以下是一些使用 Babel 的主要理由：

- 兼容旧版浏览器：不同的浏览器对 JavaScript 的支持程度不同，特别是旧版浏览器可能不支持最新的 ECMAScript 标准。通过使用 Babel，可以将使用了最新语法的代码转换为等效的旧版 JavaScript 语法，使代码能够在旧版浏览器中正常运行。
- 支持新特性：JavaScript 不断演进，每年都会发布新的 ECMAScript 标准，引入了很多有用的特性和语法糖。使用 Babel 可以提前使用这些新特性，而不用等待浏览器的支持。Babel 可以将这些新特性转换为旧版 JavaScript 语法，以便在当前的浏览器环境中使用。
- 模块化支持：Babel 可以转换模块化导入和导出的语法，使得开发者可以在浏览器中使用模块化的方式组织和管理代码。这样可以提高代码的可维护性和重用性。

46. [Webpack] 有哪些优化项目的手段？【热度: 1,163】

围绕 `webpack` 做性能优化，分为两个方面：`构建时间优化`、`构建体积优化`

构建时间优化

- [缩小范围](#)
- [文件后缀](#)
- [别名](#)
- [缓存](#)
- [并行构建](#)
- [定向查找第三方模块](#)

- 构建结果优化
 - 压缩 js
 - 压缩 css
 - 压缩 html
 - 压缩图片
 - 按需加载
 - preload、prefetch
 - 代码分割
 - tree shaking
 - gzip
 - 作用域提升

缩小范围

我们在使用 loader 时，可以配置 `include`、`exclude` 缩小 loader 对文件的搜索范围，以此来提高构建速率。

像 `/node_modules` 目录下的体积辣么大，又是第三方包的存储目录，直接 `exclude` 掉可以节省一定的时间的。

当然 `exclude` 和 `include` 可以一起配置，大部分情况下都是只需要使用 loader 编译 `src` 目录下的代码

```
1  module.exports = {module: {
2      rules: [
3          {
4              test: /\.([ts|tsx|js|jsx])$/, // 只解析 src 文件夹下的 ts、tsx、
              // js、jsx 文件// include 可以是数组，表示多个文件夹下的模块都要解析
5              include: path.resolve(__dirname, '../src'),
6              use: [ 'thread-loader', 'babel-loader'],
7              //当然也可以配置 exclude，表示 loader 解析时不会编译这部分文件//同
              //样 exclude 也可以是数组
8              exclude: /node_modules/,
9          }
10     ]
11 }
12 }
```

还需注意一个点就是要确保 loader 的 `准确性`，比如不要使用 `less-loader` 去解析 `css` 文件

文件后缀

`resolve.extensions` 是我们常用的一个配置，他可以在导入语句没有带文件后缀时，可以按照配置的列表，自动补上后缀。我们应该根据我们项目中文件的实际使用情况设置后缀列表，将使用频率高的放在前面、同时后缀列表也要尽可能的少，减少没有必要的匹配。同时，我们在源码中写导入语句的时候，尽量带上后缀，避免查找匹配浪费时间。

```
1 module.export = {
2   resolve: {// 按照 tsx、ts、jsx、js 的顺序匹配，若没匹配到则报错
3     extensions: ['.tsx', '.ts', '.jsx', '.js'],
4   }
5 }
```

别名

通过配置 `resolve.alias` 别名的方式，减少引用文件的路径复杂度

```
1 module.exports = {
2   resolve: {
3     alias: {//把 src 文件夹别名为 @//引入 src 下的文件就可以 import xxx from
4       '@/xxx' '@': path.join(__dirname, '../src')
5     }
6   }
7 }
8 // 引入 src 下的某个模块时import XXX from '@/xxx/xxx.tsx'
```

缓存

在优化的方案中，缓存也是其中重要的一环。在构建过程中，开启缓存提升二次打包速度。

在项目中，js 文件是占大头的，当项目越来越大时，如果每次都需要去编译 JS 代码，那么构建的速度肯定会很慢的，所以我们可以配置 `babel-loader` 的缓存配置项 `cacheDirectory` 来缓存没有变过的 js 代码

```
1 module.exports = {module: {
2   rules: [
3     {
4       test: /\.jsx?$/,
5       use: [
6         {
7           loader: 'babel-loader',
```

```

8         options: {
9             cacheDirectory: true,
10        },
11    }
12  ]
13  }
14  ]
15  }
16  }

```

上面的缓存优化只是针对像 `babel-loader` 这样可以配置缓存的 loader，那没有缓存配置的 loader 该怎么使用缓存呢，此时需要 `cache-loader`

```

1  module.exports = {module: {
2    rules: [
3      {
4        test: /\.jsx?$/,
5        use: ['cache-loader', "babel-loader"
6      ],
7    }
8  ]
9  }
10 }

```

编译后同样多一个 `/node_modules/.cache/cache-loader` 缓存目录

当然还有一种方式，`webpack5` 直接提供了 `cache` 配置项，开启后即可缓存

```

1  module.exports = {
2    cache: {
3      type: 'filesystem'
4    }
5  }

```

编译后会多出 `/node_modules/.cache/webpack` 缓存目录

并行构建

首先，运行在 `Node` 里的 `webpack` 是单线程的，所以一次性只能干一件事，那如果利用电脑的多核优势，也能提高构建速度？`thread-loader` 可以开启多进程打包

```

1  module.exports = {module: {

```



```

2     rules: [
3       {
4         test: /\.jsx?$/,
5         use: [// 开启多进程打包。
6           {
7             loader: 'thread-loader',
8             options: {
9               workers: 3 // 开启 3个 进程
10            }
11          },
12          {
13            loader: 'babel-loader',
14          }
15        ]
16      }
17    ]
18  }
19 }

```

放置在这个 `thread-loader` 之后的 loader 就会在一个单独的 worker 池(worker pool) 中运行。每个 worker 都是一个单独的有 600ms 限制的 `node.js` 进程。同时跨进程的数据交换也会被限制。所以建议仅在耗时的 loader 上使用。若项目文件不算多就不要使用，毕竟开启多个线程也会存在性能开销。

定向查找第三方模块

`resolve.modules` 配置用于指定 `webpack` 去哪些目录下寻找第三方模块。默认值是 `['node_modules']`。而在引入模块的时候，会以 `node 核心模块` -----> `node_modules` -----> `node全局模块` 的顺序查找模块。

我们通过配置 `resolve.modules` 指定 `webpack` 搜索第三方模块的范围，提高构建速率

```

1  module.export = {
2    resolve: {
3      modules: [path.resolve(__dirname, 'node_modules')]
4    }
5  }

```

构建结果优化

压缩 js

webpack5的话通过 `terser-webpack-plugin` 来压缩 JS，但在配置了 `mode: production` 时，会默认开启

```

1  const TerserPlugin = require('terser-webpack-plugin');
2
3  module.exports = {
4    optimization: { // 开启压缩
5      minimize: true, // 压缩工具
6      minimizer: [new TerserPlugin({}),
7        ],
8    },
9  }

```

需要注意一个地方：生产环境会默认配置 `terser-webpack-plugin`，所以如果你还有其它压缩插件使用的话需要将 `TerserPlugin` 显示配置或者使用 `...`，否则 `terser-webpack-plugin` 会被覆盖。

```

1  const TerserPlugin = require("terser-webpack-plugin");
2  optimization: {
3    minimize: true,
4    minimizer: [new TerserPlugin({}), // 显示配置// "...", // 或者使用展开符, 启用默认插件// 其它压缩插件new CssMinimizerPlugin(),
5  ],
6  },

```

压缩 css

压缩 css 我们使用 `css-minimizer-webpack-plugin`

同时，应该把 css 提取成单独的文件，使用 `mini-css-extract-plugin`

```

1  const MiniCssExtractPlugin = require("mini-css-extract-plugin");
2  const CssMinimizerPlugin = require("css-minimizer-webpack-plugin");
3
4  module.exports = {module: {
5    rules: [
6      {
7        test: /\.css$/,
8        use: [ // 提取成单独的文件
9          MiniCssExtractPlugin.loader, "css-loader"
10        ],
11        exclude: /node_modules/,
12      },
13    ],
14  },
15  plugins: [new MiniCssExtractPlugin({ // 定义输出文件名和目录

```

```

16     filename: "asset/css/main.css",
17   })
18 ],
19   optimization: {
20     minimize: true,
21     minimizer: [// 压缩 css new CssMinimizerPlugin({}),
22   ],
23   },
24 }

```

压缩 html

压缩 `html` 使用的还是 `html-webpack-plugin` 插件。该插件支持配置一个 `minify` 对象，用来配置压缩 `html`。

```

1  module.export = {
2    plugins: [new HtmlWebpackPlugin({// 动态生成 html 文件
3      template: "./index.html",
4      minify: {// 压缩HTML
5        removeComments: true, // 移除HTML中的注释
6        collapseWhitespace: true, // 删除空白符与换行符
7        minifyCSS: true // 压缩内联css
8      },
9    })
10  ]
11  }

```

压缩图片

可以通过 `image-webpack-loader` 来实现

```

1  module.exports = {module: {
2    rules: [
3      {
4        test: /\..(png|jpg|gif|jpeg|webp|svg)$/,
5        use: ["file-loader",
6          {
7            loader: "image-webpack-loader",
8            options: {
9              mozjpeg: {
10                progressive: true,
11              },
12              optipng: {
13                enabled: false,

```

```

14         },
15         pngquant: {
16             quality: [0.65, 0.9],
17             speed: 4,
18         },
19         gifsicle: {
20             interlaced: false,
21         },
22     },
23 },
24 ],
25     exclude: /node_modules/, //排除 node_modules 目录
26 },
27 ]
28 },
29 }

```

按需加载

很多时候我们不需要一次性加载所有的 JS 文件，而应该在不同阶段去加载所需要的代码。

将路由页面/触发性功能单独打包为一个文件，使用时才加载，好处是减轻首屏渲染的负担。因为项目功能越多其打包体积越大，导致首屏渲染速度越慢。

实际项目中大部分是对懒加载路由，而懒加载路由可以打包到一个 chunk 里面。比如某个列表页和编辑页它们之间存在相互跳转，如果对它们拆分成两个 `import()` js 资源加载模块，在跳转过程中视图会出现白屏切换过程。

因为在跳转期间，浏览器会动态创建 script 标签来加载这个 chunk 文件，在这期间，页面是没有任何内容的。

所以一般会把路由懒加载打包到一个 chunk 里面

```

1  const List = lazyComponent('list', () => import(/* webpackChunkName: "list" */
    '@pages/list'));
2  const Edit = lazyComponent('edit', () => import(/* webpackChunkName: "list" */
    '@pages/edit'));

```

但需要注意一点：动态导入 `import()` 一个模块，这个模块就不能再出现被其他模块使用 `同步 import` 方式导入。

比如，一个路由模块在注册 `<Route />` 时采用动态 `import()` 导入，但在这个模块对外暴露了一些变量方法供其他子模块使用，在这些子模块中使用了同步 ESM module `import` 方式引入，这就造成了 `动态 import()` 的失效。

prload、prefetch

对于某些较大的模块，如果点击时再加载，那可能响应的时间反而延长。我们可以使用

`prefetch`、`preload` 去加载这些模块

`prefetch`：将来可能需要一些模块资源（一般是其他页面的代码），在核心代码加载完成之后带空闲的时候再去加载需要用到的模块代码。

`preload`：当前核心代码加载期间可能需要模块资源（当前页面需要的但暂时还没使用到的），其是和核心代码文件一起去加载的。

只需要通过魔法注释即可实现，以 `prefetch` 为例：

```
1 document.getElementById('btn1').onclick = function() {import(
2   /* webpackChunkName: "btnChunk"
3   /
4   * /* webpackPrefetch: true*/'./module1.js'
5   ).then(fn => fn.default());
6 }
```

这行代码表示在浏览器空闲时加载 `module1.js` 模块，并且单独拆一个 chunk，叫做 `btnChunk`

可以看到，在 `head` 里面，我们的懒加载模块被直接引入了，并且加上了 `rel='prefetch'`。

这样，页面首次加载的时候，浏览器空闲的会后会提前加载 `module1.js`。当我们点击按钮的时候，会直接从缓存中读取该文件，因此速度非常快。

代码分割

在项目中，一般是使用同一套技术栈和公共资源。如果每个页面的代码中都有这些公开资源，就会导致资源的浪费。在每一个页面下都会加载重复的公共资源，一是会浪费用户的流量，二是不利于项目的性能，造成页面加载缓慢，影响用户体验。

一般是把不变的第三方库、一些公共模块（比如 `util.js`）这些单独拆成一个 chunk，在访问页面的时候，就可以一直使用浏览器缓存中的资源

webpack 里面通过 `splitChunks` 来分割代码

```
1 module.exports = {//...
2   optimization: {
3     splitChunks: {
4       chunks: 'async', // 值有 all, async 和 initial
5       minSize: 20000, // 生成 chunk 的最小体积（以 bytes 为单位）。
6       minRemainingSize: 0,
7       minChunks: 1, // 拆分前必须共享模块的最小 chunks 数。
8       maxAsyncRequests: 30, // 按需加载时的最大并行请求数。
```

```

9      maxInitialRequests: 30, // 入口点的最大并行请求数。
10     enforceSizeThreshold: 50000,
11     cacheGroups: {
12       defaultVendors: {
13         test: /[\\/]node_modules[\\/]/, //第三方模块拆出来
14         priority: -10,
15         reuseExistingChunk: true,
16       },
17       util.vendors: {
18         test: /[\\/]utils[\\/]/, //公共模块拆出来
19         minChunks: 2,
20         priority: -20,
21         reuseExistingChunk: true,
22       },
23     },
24   },
25 },
26 };

```

tree shaking

`tree shaking` 在生产模式下已经默认开启了

只是需要注意下面几点：

1. 只对 `ESM` 生效
2. 只能是静态声明和引用的 `ES6` 模块，不能是动态引入和声明的。
3. 只能处理模块级别，不能处理函数级别的冗余。
4. 只能处理 `JS` 相关冗余代码，不能处理 `CSS` 冗余代码。

而可能样式文件里面有些代码我们也没有使用，我们可以通过 `purgecss-webpack-plugin` 插件来对 css 进行 tree shaking

```

1  const path = require("path");
2  const PurgecssPlugin = require("purgecss-webpack-plugin");
3  const glob = require("glob"); // 文件匹配模式 module.exports = { // ...
4    plugins: [
5      ...new PurgeCSSPlugin({
6        paths: glob.sync(`${PATH.src}/**/*`, { nodir: true }),
7      })
8      // Add your plugins here// Learn more about plugins from
8      // https://webpack.js.org/configuration/plugins/
9    ],
10  };

```

gzip

前端除了在打包的时候将无用的代码或者 `console`、注释剔除之外。我们还可以使用 `Gzip` 对资源进行进一步压缩。那么浏览器和服务端是如何通信来支持 `Gzip` 呢？

1. 当用户访问 web 站点的时候，会在 `request header` 中设置 `accept-encoding:gzip`，表明浏览器是否支持 `Gzip`。
2. 服务器在收到请求后，判断如果需要返回 `Gzip` 压缩后的文件那么服务器就会先将我们的 `JS\CSS` 等其他资源文件进行 `Gzip` 压缩后再传输到客户端，同时将 `response headers` 设置 `content-encoding:gzip`。反之，则返回源文件。
3. 浏览器在接收到服务器返回的文件后，判断服务端返回的内容是否为压缩过的内容，是的话则进行解压操作。

一般情况下我们并不会让服务器实时 `Gzip` 压缩，而是利用 `webpack` 提前将静态资源进行 `Gzip` 压缩，然后将 `Gzip` 资源放到服务器，当请求需要的时候直接将 `Gzip` 资源发送给客户端。

我们只需要安装 `compression-webpack-plugin` 并在 `plugins` 配置就可以了

```
1  const CompressionWebpackPlugin = require("compression-webpack-plugin"); // 需
    要安装module.exports = {
2    plugins: [new CompressionWebpackPlugin()
3    ]
4  }
```

作用域提升

`Scope Hoisting` 可以让 `webpack` 打包出来的代码文件体积更小，运行更快。

在开启 `Scope Hoisting` 后，构建后的代码会按照引入顺序放到一个函数作用域里，通过适当重命名某些变量以防止变量名冲突，从而减少函数声明和内存花销。

需要注意：`Scope Hoisting` 需要分析模块之间的依赖关系，所以源码必须采用 ES6 模块化语法

`Scope Hoisting` 是 `webpack` 内置功能，只需要在 `plugins` 里面使用即可，或者直接开启生产环境也可以让作用域提升生效。

```
1  module.exports = { //方式1
2    mode: 'production',
3    //方式2
4    plugins: [ // 开启 Scope Hoisting 功能new
      webpack.optimize.ModuleConcatenationPlugin()
5    ]
6  }
```


47. [Webpack] 全面了解 tree shaking

webpack 如何做 tree shaking

Webpack 通过 tree shaking 技术实现了JavaScript代码的优化和精简。Tree shaking 是指通过静态代码分析，识别和移除未被使用的代码（被称为"dead code"），从而减小最终打包后的文件大小。

下面是Webpack如何进行tree shaking的步骤：

1. 代码标记：在代码中使用ES6模块化语法（如 `import` 和 `export`）来明确指定模块的依赖关系。
2. 代码解析：Webpack会解析整个代码，并构建一个依赖图谱，记录模块之间的依赖关系。
3. 标记未使用代码：在构建依赖图谱的过程中，Webpack会标记那些未被使用的模块，以及这些模块中的未被使用的函数、类、变量等。
4. 无副作用标记：Webpack还会检查模块的副作用（例如对全局变量的修改、网络请求等），并将没有副作用的代码视为可安全移除的。
5. 未使用代码移除：在代码打包阶段，Webpack会根据标记的未使用代码信息，从最终的打包结果中移除这些未被使用的代码。

通过tree shaking，Webpack可以减小打包后的文件大小，提高应用的加载速度和性能。但要注意的是，tree shaking只对ES6模块化的代码有效，对于CommonJS模块化的代码则无法进行优化。另外，有些代码可能由于复杂的依赖关系无法被正确地标记为未使用，这就需要开发者自己进行配置或使用其他工具进行优化。

webpack 处理 tree shaking 配置

要在Webpack中配置tree shaking，需要进行以下步骤：

1. 在 `webpack.config.js` 文件中，将 `mode` 设置为 `production`，这会启用Webpack的优化功能，包括tree shaking。

```
1  module.exports = {  
2    mode: 'production', // 其他配置...  
3  };
```

1. 确保你的代码使用了ES6模块化语法（使用 `import` 和 `export`），以便Webpack能够正确地进行静态代码分析。
2. 确保你的代码库中没有副作用。Webpack会假设没有副作用的代码可以安全地移除。如果你的代码确实有副作用，可以在webpack配置文件中的 `optimization` 选项中设置 `sideEffects` 为 `false` 来告诉Webpack可以安全地进行tree shaking。

```
1  module.exports = {
2    mode: 'production',
3    optimization: {
4      sideEffects: false,
5    }, // 其他配置...
6  };
```

了解一下副作用

在计算机科学中，副作用是指函数或代码的执行对除了返回一个值之外的程序状态产生了可观察的变化。换句话说，副作用是指对外部环境产生了影响或产生了可观察的行为。

以下是一些常见的副作用示例：

- 修改全局变量或外部状态：函数修改了全局变量或外部状态，例如修改了一个共享的数组、对象或文件等。
- 发送网络请求：函数通过网络发送了一个HTTP请求，这会触发网络交互并产生副作用。
- 修改函数参数：函数修改了传入的参数值，这会影响函数外部的变量。
- 控制台打印：函数在执行过程中使用了 `console.log()` 或其他打印语句，这会在控制台中产生可观察到的输出。
- 异步操作：函数中包含了异步操作，例如定时器、Promise或通过回调函数实现的异步操作。

如何申明代码是有副作用

某一些代码是是需要禁止被清理掉，这个时候该如何处理呢？

有几个办法：

方法一：在配置文件中指定副作用

在Webpack配置文件中，可以使用 `sideEffects` 选项来指定哪些文件或模块具有副作用，不允许清理。`sideEffects` 接受一个正则表达式、一个文件名或一个数组。例如：

```
1  module.exports = { //...
2    optimization: {
3      usedExports: true
4    },
5    mode: 'production',
6    sideEffects: ['./src/some-module.js']
7  };
```

在上面的例子中，`sideEffects` 数组中的 `./src/some-module.js` 文件将会被标记为具有副作用，不会被清理。

请注意，为了使 `sideEffects` 选项生效，你需要在配置文件中启用 `optimization.usedExports` 选项，并将 `mode` 设置为 `production`。

方法二：package.json 中配置 sideEffects 属性

可以在 `package.json` 文件中使用 `sideEffects` 字段来申明哪些文件或模块具有副作用，不允许被清理。

1. 如果将 `sideEffects` 设置为布尔值 `false`，表示所有导入的文件都被认为没有副作用，可以被tree shaking清理。这在大多数情况下是默认的行为。

```
1  {
2    "name": "my-package",
3    "version": "1.0.0",
4    "sideEffects": false
5  }
```

1. 如果设置为布尔值 `true`，表示所有导入的文件都被认为有副作用，不会被tree shaking清理。

```
1  {
2    "name": "my-package",
3    "version": "1.0.0",
4    "sideEffects": true
5  }
```

1. 如果将 `sideEffects` 设置为一个数组，数组中的每个元素可以是一个字符串或一个正则表达式，表示具有副作用的文件或模块。

```
1  {
2    "name": "my-package",
3    "version": "1.0.0",
4    "sideEffects": [".src/some-module.js", /\.css$/]
5  }
6  }
```

在上述示例中，`./src/some-module.js` 文件和所有以 `.css` 结尾的文件都被认为有副作用，不会被tree shaking清理。

如果我某一个文件配置了 `sideEffects` 申明该文件有副作用，但是我又想清理其中的某个函数

魔法中的魔法注释: `/*#__PURE__*/`

通过上面的知识，我们知道了，如果是有如果被 `sideEffects` 申明了副作用的文件，是不会被 tree shaking 清理掉的，但是也有例外。

`/*#__PURE__*/` 这个注释的作用是告诉 Webpack 或 Babel 等构建工具，这一行代码是纯粹的，没有副作用，并且可以安全地进行 tree shaking（摇树优化）。

对于一些库或框架，可能会有一些函数或类被导出，但实际上很少被使用，为了让构建工具知道这些代码可以被删除，可以在导出语句上添加 `/*#__PURE__*/` 注释。

例如，假设 `src/myModule.js` 文件有下面的代码：

```
1  export /*#__PURE__*/ function add(a, b) {return a + b;
2  }
3
4  export function subtract(a, b) {return a - b;
5  }
```

且 webpack 已经将 `src/myModule.js` 申明为了有副作用文件

```
1  module.exports = {// ...
2    optimization: {
3      sideEffects: [".src/myModule.js"],
4    },
5  };
```

虽然通过 `sideEffects` 配置申明了 `./src/myModule.js` 文件是有副作用的，但是由于 `add` 方法前面有 `/*#__PURE__*/` 注释标记，意味着这个方法被标记为纯函数，该方法是没有任何副作用。

因此最终通过 `/*#__PURE__*/` 注释标记的 `add` 方法依然可以被 Webpack 的 Tree Shaking 清理。

commonjs 模块就真的不能被 tree shaking 了？

Webpack 曾经不进行对 CommonJs 导出和 `require()` 调用时的导出使用分析。

Webpack 5 增加了对一些 CommonJs 构造的支持，允许消除未使用的 CommonJs 导出，并从 `require()` 调用中跟踪引用的导出名称。

支持以下构造：

- `exports|this|module.exports.xxx = ...`
- `exports|this|module.exports = require("...") (reexport)`

- `exports|this|module.exports.xxx = require("...").xxx (reexport)`
- `Object.defineProperty(exports|this|module.exports, "xxx", ...)`
- `require("abc").xxx`
- `require("abc").xxx()`
- 从 ESM 导入
- `require()` 一个 ESM 模块
- 被标记的导出类型 (对非严格 ESM 导入做特殊处理):
 - `Object.defineProperty(exports|this|module.exports, "__esModule", { value: true|!0 })`
 - `exports|this|module.exports.__esModule = true|!0`

当检测到不可分析的代码时，webpack 会放弃，并且完全不跟踪这些模块的导出信息（出于性能考虑）。

终极必杀问：webpack tree-shaking 在什么情况下会失效

- 动态导入：如果你使用了动态导入（例如使用了 `import()` 或 `require.ensure()`），webpack 无法静态分析模块的导入和导出，因此无法进行 tree-shaking。
- 未使用 ES6 模块语法：tree-shaking 只能对 ES6 模块语法进行优化，如果你的代码中没有使用 ES6 模块语法，webpack 将无法进行 tree-shaking。
- 模块被动态引用或条件引用：如果模块的引用方式是动态的（例如在循环或条件语句中引用），或者通过字符串拼接来引用模块，webpack 无法确定哪些模块会被引用，因此无法进行 tree-shaking。
- 使用了副作用的代码：如果你的代码中包含有副作用的代码（例如在模块的顶级作用域中执行了一些操作），webpack 无法确定哪些代码是无用的，因此无法进行 tree-shaking。

48. [Vue] 响应式为何要从 `Object.defineProperty` 改为 `proxy`? 【热度: 352】

Vue 在早期版本中使用了 `Object.defineProperty` 来实现响应式系统。但是，在 `Object.defineProperty` 中存在一些限制和局限性，导致在某些场景下无法完全满足需求。因此，Vue 在最新的版本中引入了 `Proxy` 来替代 `Object.defineProperty`。

以下是一些 `Proxy` 相对于 `Object.defineProperty` 的优势：

1. 功能更强大：`Proxy` 可以代理整个对象，而 `Object.defineProperty` 只能对已存在的属性进行拦截。使用 `Proxy` 可以在对象级别上进行拦截、代理、验证等操作。
2. 更易于使用和理解：`Proxy` 提供了一组更直观和易于理解的 API，使开发者可以更容易地创建和管理代理。

3. 性能优化：Proxy 针对属性的访问和修改都提供了更佳的性能优化。而 Object.defineProperty 在拦截属性访问和修改时会有一定的性能损耗。
4. 更好的嵌套支持：Proxy 可以代理嵌套对象的属性，而 Object.defineProperty 只能对顶层对象的属性进行拦截。

总的来说，Proxy 相对于 Object.defineProperty 在功能上更强大、使用更便捷、性能更优，并且在更复杂的场景下也能提供更好的支持。因此，Vue 在新版本中选择了使用 Proxy 来实现响应式系统。

49. 在你的项目中，使用过哪些 webpack loader, 说一下他们的作用

Loader 名称	作用
babel-loader	将 ES6+ 代码转换为 ES5 代码，以便在旧版浏览器中运行。
css-loader	解析 CSS 文件，处理 CSS 中的依赖关系，并将 CSS 转换为 JS 模块。
style-loader	将 CSS 代码以内联的方式注入到 HTML 页面中。
file-loader	处理文件资源（如图片、字体等），将文件复制到输出目录，并返回文件路径。
url-loader	与 file-loader 类似，但可以根据文件大小将文件转换为 Data

	URL（base64 格式）或文件路径。
sass-loader	解析 Sass/SCSS 文件，并将其转换为 CSS 代码。
less-loader	解析 Less 文件，并将其转换为 CSS 代码。
	使用
postcss-loader	PostCSS 处理 CSS，可以进行自动添加前缀、压缩、CSS Modules 等操作。
ts-loader	将 TypeScript 代码转换为 JavaScript 代码。
eslint-loader	在构建过程中使用 ESLint 进行代码检查。
stylelint-webpack-plugin	在构建过程中使用 Stylelint 进行 CSS/SCSS 代码检查。
vue-loader	解析 Vue 单文件组件（.vue 文件），并将其转换为 JavaScript 代码。
image-	优化图片资源，包括压

webpack-loader	你，包括压缩、转换格式等操作。
html-loader	解析 HTML 文件，处理其中的引用资源（如图片、字体等），并返回处理后的 HTML 代码。
markdown-loader	将 Markdown 文件转换为 HTML 代码。
json-loader	解析 JSON 文件，并返回解析后的 JavaScript 对象。
eslint-loader	在构建过程中使用 ESLint 进行代码检查。
tslint-loader	在构建过程中使用 TSLint 进行 TypeScript 代码检查。
prettier-loader	在构建过程中使用 Prettier 进行代码格式化。
stylelint-webpack-plugin	在构建过程中使用 Stylelint 进行 CSS/SCSS 代码检查。
mini-css-extract-plugin	提取 CSS 代码到单独的文件，而不是内联到

	JavaScript 代码中。
optimize-css-assets-webpack-plugin	压缩 CSS 代码。
terser-webpack-plugin	压缩 JavaScript 代码。

这些 Loader 可以根据需要配置在 Webpack 的模块规则（`module.rules`）中，以实现对不同类型文件的处理和转换操作。

50. 在 webpack 中，通常用于 css 提取的工具是什么？【热度: 69】

在 webpack 中，通常使用 `mini-css-extract-plugin` 来提取 CSS。它是一个独立的插件，可以将 CSS 从 JavaScript 文件中提取出来，生成独立的 CSS 文件。

使用 `mini-css-extract-plugin` 可以优化代码分离和缓存，以及提高加载速度。

通过配置 webpack 的插件选项，可以将 `mini-css-extract-plugin` 添加到 webpack 构建流程中。在配置中，需要将该插件实例化，并指定输出的 CSS 文件名和路径。

一旦配置完成并运行 webpack 构建，`mini-css-extract-plugin` 就会将 CSS 提取到独立的文件中，而不是将其嵌入到 JavaScript 文件中。

示例代码如下：

```
1  const MiniCssExtractPlugin = require('mini-css-extract-plugin');
2
3  module.exports = { // ...其他配置 module: {
4    rules: [ // ...其他规则
5      {
6        test: /\.css$/,
7        use: [
8          MiniCssExtractPlugin.loader, 'css-loader',
9        ],
10     },
11   ],
12 },
13 plugins: [new MiniCssExtractPlugin({
14   filename: '[name].css',
15   chunkFilename: '[id].css',
16 }),
17 ],
18 };
```

在上述示例中，`MiniCssExtractPlugin.loader` 是用于提取 CSS 的 loader。`css-loader` 用于处理 CSS 文件的导入和解析。

`MiniCssExtractPlugin` 则是插件实例，用于配置输出的 CSS 文件名。

51. 将静态资源缓存在本地的方式有哪些？【热度: 584】

浏览器可以使用以下几种方式将前端静态资源缓存在本地：

1. HTTP缓存：浏览器通过设置HTTP响应头中的Cache-Control或Expires字段来指定资源的缓存策略。常见的缓存策略有：no-cache（每次都请求服务器进行验证）、no-store（不缓存资源）、max-age（设置资源缓存的最大时间）等。浏览器根据这些缓存策略来决定是否将资源缓存在本地。
2. ETag/If-None-Match：服务器可以通过在响应头中添加ETag字段，用于标识资源的版本号。当浏览器再次请求资源时，会将上次请求返回的ETag值通过If-None-Match字段发送给服务器，由服务器判断资源是否发生了变化。如果资源未发生变化，服务器会返回304 Not Modified状态码，浏览器则直接使用本地缓存的资源。
3. Last-Modified/If-Modified-Since：服务器可以通过在响应头中添加Last-Modified字段，用于标识资源的最后修改时间。浏览器再次请求资源时，会将上次请求返回的Last-Modified值通过If-Modified-Since字段发送给服务器。服务器根据资源的最后修改时间判断资源是否发生了变化，如果未发生变化，则返回304 Not Modified状态码，浏览器使用本地缓存的资源。
4. Service Worker缓存：使用Service Worker可以将前端资源缓存在浏览器的Service Worker缓存中。Service Worker是运行在浏览器后台的脚本，它可以拦截和处理网络请求，因此可以将前端资源缓存起来，并在离线状态下提供缓存的资源。
5. LocalStorage或IndexedDB：对于一些小的静态资源，可以将其存储在浏览器的LocalStorage或IndexedDB中。这些存储方式是浏览器提供的本地存储机制，可以将数据以键值对的形式存储在浏览器中，从而实现缓存的效果。

如何将静态资源缓存在 LocalStorage或IndexedDB

以下是一个使用LocalStorage将静态资源缓存的示例代码：

```
1  // 定义一个数组，包含需要缓存的静态资源的URLvar resources =  
    ['https://example.com/css/style.css', 'https://example.com/js/main.js', 'https://  
    /example.com/images/logo.png'  
2  ];  
3  
4  // 遍历资源数组，将资源请求并存储在LocalStorage中  
5  resources.forEach(function(url) { // 发起资源请求  
6      fetch(url)
```

```

7      .then(function(response) {// 检查请求是否成功if (!response.ok) {throw new
Error('Request failed: 'response.status);
8    }// 将响应数据存储在LocalStorage中return response.text();
9    })
10   .then(function(data) {// 将资源数据存储在LocalStorage中, 以URL作为键名
localStorage.setItem(url, data);console.log('Resource cached: 'url);
11   })
12   .catch(function(error) {console.error(error);
13   });
14   });
15   });

```

以下是一个使用IndexedDB将静态资源缓存的示例代码：

```

1  // 打开或创建一个IndexedDB数据库var request = indexedDB.open('myDatabase', 1);
2
3  // 创建或更新数据库的对象存储空间
4  request.onupgradeneeded = function(event) {var db = event.target.result;var
objectStore = db.createObjectStore('resources', { keyPath: 'url' });
5    objectStore.createIndex('url', 'url', { unique: true });
6  };
7
8  // 成功打开数据库后, 将资源请求并存储在IndexedDB中
9  request.onsuccess = function(event) {var db = event.target.result;var
transaction = db.transaction('resources', 'readwrite');var objectStore =
transaction.objectStore('resources');
10    resources.forEach(function(url) {// 发起资源请求
11      fetch(url)
12        .then(function(response) {// 检查请求是否成功if (!response.ok) {throw new
Error('Request failed: 'response.status);
13        }// 将响应数据存储在IndexedDB中return response.blob();
14        })
15        .then(function(data) {// 创建一个资源对象, 以URL作为键名var resource = {
url: url, data: data };// 将资源对象存储在IndexedDB中
16          objectStore.put(resource);console.log('Resource cached: 'url);
17        })
18        .catch(function(error) {console.error(error);
19        });
20    });
21  // 完成事务
22  transaction.oncomplete = function() {console.log('All resources cached in
IndexedDB. ');
23  };
24  transaction.onerror = function(event) {console.error('Transaction error:',
event.target.error);
25  };

```

以上代码仅为示例，实际应用中需要根据具体的需求进行相应的优化和错误处理。

52. SPA首屏加载速度慢的怎么解决【热度: 868】

影响首屏可能得因素

- 网络延时问题
- 资源文件体积是否过大
- 资源是否重复发送请求去加载了
- 加载脚本的时候，渲染内容堵塞了

解决方案

- 减小入口文件积
- 静态资源本地缓存
- UI框架按需加载
- 图片资源的压缩
- 组件重复打包
- 开启 GZip 压缩
- 使用 SSR
- 启用 CDN 加速

53. git 中回滚代码有哪些操作？【热度: 237】

在 Git 中，可以使用以下几种操作来回滚代码：

1. git revert：使用 git revert 命令可以创建一个新的提交来撤销之前的提交。它会创建一个新的提交，恢复之前的更改，从而回滚代码。这种方式比较安全，不会改变历史提交记录。
2. git reset --soft：使用 git reset --soft 命令可以将当前分支的 HEAD 指向指定的提交，同时保留修改。这将撤销指定提交之后的所有提交，但是保留更改的文件在暂存区中，可以用于回滚代码并重新提交。
3. git reset --mixed（默认操作）：使用 git reset --mixed 命令可以将当前分支的 HEAD 指向指定的提交，同时取消之后的提交，但保留修改。这将撤销指定提交之后的所有提交，并重置暂存区的内容，但是不改变工作区的内容，可以用于回滚代码并重新修改后重新提交。
4. git reset --hard：使用 git reset --hard 命令可以将当前分支的 HEAD 指向指定的提交，并丢弃之后的修改。这将撤销指定提交之后的所有提交，并重置工作区和暂存区的内容，慎用，会永久丢失未提交的修改。

需要注意的是，git revert 会保留之前提交的历史记录，而 git reset 则会直接修改历史提交记录。因此，在多人协作或者公共仓库中，建议使用 git revert 来回滚代码，以避免对他人代码的影响。在个人项目或者正在开发中的分支中，可以使用 git reset 更灵活地回滚代码。

54. git reset 作用是啥，有哪些操作？【热度: 275】

git reset 是一个用于撤销提交的命令，可以将当前分支指向某个特定的提交，同时可以选择是否修改工作区和暂存区。

常见的 git reset 操作有以下三种：

1. git reset --soft：将当前分支的 HEAD 指向指定的 commit，并保留之后的修改。这种方式不改变工作区和暂存区的内容，可以用于撤销之前的提交，重新修改后重新提交。
2. git reset --mixed（默认操作）：将当前分支的 HEAD 指向指定的 commit，并取消之后的提交，但保留修改。这种方式会重置暂存区的内容，但不改变工作区的内容，可以用于撤销之前的提交，重新修改后重新提交。
3. git reset --hard：将当前分支的 HEAD 指向指定的 commit，并丢弃之后的修改。这种方式会重置工作区和暂存区的内容，慎用，会永久丢失未提交的修改。

除了上述操作之外，还可以搭配使用 git reset 的其他选项和参数。其中，可以是一个提交的哈希值、分支名或者标签名，用于指定要回退到的提交。

55. 为什么现代前端应用需要打包工具进行打包编译？【热度: 1,588】

现代前端应用需要打包工具进行打包编译的主要原因有以下几点：

1. 模块化管理：现代前端应用通常采用模块化的开发方式，将代码划分为多个模块，每个模块具有独立的功能和依赖关系。打包工具可以将这些模块进行分析，将它们打包成一个或多个静态文件，方便管理和维护。
2. 解决浏览器兼容性问题：不同的浏览器对于 JavaScript 和 CSS 的支持程度不同，而且随着新特性的不断出现，旧版浏览器可能无法完全支持。打包工具可以通过转译、压缩和兼容性处理等手段，将当前前端代码转化为浏览器可识别和运行的代码，解决兼容性问题。
3. 静态资源处理和优化：现代前端应用涉及大量的静态资源，如图片、字体等。打包工具可以对这些资源进行处理和优化，如图片压缩、字体文件打包等，以减小资源文件的体积，提高页面的加载速度和性能。
4. 代码分割和按需加载：打包工具可以将应用程序拆分成多个小块，实现代码分割和按需加载。这样可以实现懒加载，只在需要时加载特定的代码块，提高页面的加载速度。
5. 开发环境支持：打包工具通常提供开发服务器和热模块替换（HMR）等功能，方便开发人员进行开发和调试。开发服务器可以实时预览代码变化，HMR 可以在修改代码后只替换修改的部分，而不是整个页面刷新，提高开发效率。

6. 提升性能：打包工具可以通过代码优化、压缩和混淆等技术手段，减小文件体积，提升应用程序的加载速度和执行效率。
7. 支持多种前端技术：现代前端应用通常使用多种前端技术和语言，如JavaScript、CSS、TypeScript、Sass等。打包工具可以集成这些技术，并提供相应的编译、转译和处理功能，使开发人员能够更轻松地使用这些技术。
8. 自动化工作流程：打包工具可以配合其他构建工具和自动化任务运行器，如Webpack配合Grunt或Gulp，实现自动化的构建和部署流程。这可以减少手动操作，提高开发效率和代码质量。
9. 第三方库管理：现代前端应用通常使用大量的第三方库和框架，这些库可能包含多个文件和依赖关系。打包工具可以自动管理这些库的依赖关系，并将它们打包为单个文件，减少网络请求和提高代码的可维护性。
10. 高度可定制化：打包工具通常提供丰富的插件和配置选项，允许开发人员根据项目需求进行定制。可以灵活配置打包过程中的各种处理和优化方式，以满足项目的具体需求。

总结 - 现代前端应用需要打包工具进行打包编译的原因是为了：实现模块化管理、解决兼容性问题、静态资源处理和优化、代码分割和按需加载、开发环境支持、性能提升、多技术支持、自动化工作流程、第三方库管理和可定制化等方面的需求。

56. 如果用户反馈说感觉网页很卡顿，这个时候该如何排查问题？【热度: 768】

排查网页卡顿问题时，可以按照以下步骤进行处理：

1. 验证用户反馈：首先，确认用户反馈的卡顿问题是否普遍存在，还是个别用户的特殊情况。可以与其他用户进行沟通或观察其他设备上的表现。
2. 检查网络连接：检查用户的网络连接是否稳定。卡顿问题可能由于网络延迟或不稳定导致。可以要求用户进行网络速度测试，或者与用户确认网络连接是否正常。
3. 检查服务器性能：确认服务器是否能够处理用户的请求。可以检查服务器的负载情况、处理请求的时间以及资源使用情况。如果服务器负载过高，可能导致网页卡顿。
4. 检查前端代码：检查网页的前端代码是否存在问题。主要看是否有内存泄露、long task 等情况；
5. 优化网页性能：对于前端代码存在性能问题的情况，可以尝试优化网页性能。例如，压缩和合并CSS和JavaScript文件、减少网络请求次数、使用缓存等方法来提高页面加载速度。
6. 做好性能监控：持续监测网页的性能，并定期更新网页的代码和设计，以提升用户体验。

57. 前端架构和前端工程化有什么区别？

区别

前端架构和前端工程化是两个不同的概念，但它们之间有一些相互关联的特点。

前端架构是指在前端开发中，对整个前端应用程序的组织结构、模块划分、框架选择等方面的设计和规划。前端架构的目标是为了提高代码的可维护性、可扩展性和可重用性，以及优化前端应用程序的性能和用户体验。常见的前端架构包括MVC（Model-View-Controller）、MVVM（Model-View-ViewModel）等。

前端工程化是指使用各种工具、技术和流程对前端开发过程进行管理和优化，以提高开发效率、代码质量和团队协作能力。前端工程化的目标是通过规范化和自动化的方式，解决前端开发中的重复劳动、低效率、代码质量不稳定等问题。前端工程化包括代码管理、代码规范、模块化开发、构建工具、自动化测试、持续集成和部署、性能优化、文档和知识管理等方面。

虽然前端架构和前端工程化是两个不同的概念，但它们之间存在一些相似的目标和方法。前端架构关注的是前端应用程序的结构和设计，而前端工程化关注的是前端开发的流程和工具的使用。前端架构可以通过前端工程化的方式实现，而前端工程化可以提供支持和保障，以实现良好的前端架构。

58. 如何理解前端工程化？

前端工程化是指将前端开发过程中的各种工具、技术和流程进行规范化和自动化，以提高前端开发效率、代码质量和团队协作能力的一种方法。前端工程化主要涵盖以下几个方面：

1. 代码管理：使用版本控制系统（如Git）对前端项目进行代码管理，实现代码的版本控制、协作开发和代码回滚等功能。
2. 代码规范：制定和遵循统一的前端代码规范，包括缩进、命名、注释等，以提高代码的可读性和可维护性。
3. 模块化开发：使用模块化的开发方式，将前端代码拆分为独立的模块，以便于复用和维护。
4. 构建工具：使用构建工具（如Webpack、Gulp）对前端代码进行构建、打包、压缩和优化等处理，提高开发效率和应用性能。
5. 自动化测试：编写自动化测试用例，使用测试工具（如Jest、Mocha）进行自动化测试，以确保代码质量和功能的稳定性。
6. 持续集成和部署：使用持续集成工具（如Jenkins、Travis CI）将代码自动构建、测试和部署到服务器，加快代码交付速度和减少人工操作。
7. 性能优化：使用性能优化工具（如Webpack、Lighthouse）来分析和改进前端应用程序的性能，包括文件加载、资源压缩、缓存策略等方面。
8. 文档和知识管理：编写和维护开发文档、API 文档以及团队知识库，方便团队成员之间的沟通和知识分享。

通过前端工程化的实践，可以减少重复劳动、提高代码质量、加速开发流程和降低维护成本，从而更好地满足项目需求和提升用户体验。

59. 如何理解前端架构？

前端架构是指在开发前端应用程序时，为了提高开发效率、代码可维护性和可扩展性，将前端代码组织和设计的一种架构体系。它涵盖了前端项目的整体结构、代码组织方式、技术选型、模块化开发、数据管理、状态管理、网络请求、路由管理等方面的规划和设计。

前端架构的目标是使前端开发更加规范、高效和可持续。良好的前端架构可以帮助团队成员更好地协作、降低维护成本、提高开发效率、减少代码冗余和bug等。

以下是一些常见的前端架构概念和思想：

1. 分层架构：将前端应用程序划分为不同的层次，如视图层、业务逻辑层、数据层等，以实现各个层次的解耦和职责清晰。
2. 组件化开发：将界面拆分为独立的可复用组件，通过组合不同的组件来构建页面，提高代码的可维护性和可复用性。
3. 模块化开发：将代码按照功能或业务模块进行拆分，每个模块都有独立的职责和功能，便于团队协作和代码维护。
4. 数据管理和状态管理：使用状态管理库（如Redux、Vuex）来管理应用程序的状态和数据流，使得数据的变化和传递更加可控和可预测。
5. 路由管理：使用路由库（如React Router、Vue Router）来管理前端路由，实现页面之间的跳转和导航。
6. 统一风格和规范：制定和遵循统一的编码规范、命名规范、目录结构等，以便于不同开发者之间的协作和代码的统一性。
7. 自动化构建和部署：使用构建工具（如Webpack、Rollup）和自动化部署工具（如Jenkins、Travis CI）来提高开发效率和代码交付速度。
8. 性能优化和项目优化：通过代码分割、懒加载、缓存、CDN 加速、前端性能监控等手段，提升应用程序的性能和用户体验。

前端架构是一种组织和规划前端代码的方法论，旨在提高前端开发的效率和质量，同时也要根据具体项目的需求和规模来选择和适配合适的架构方案。

60. 前端基建设计到哪些方面【热度: 5,782】

前端基建是指在前端开发过程中，为提高开发效率、代码质量和团队协作而构建的一些基础设施和工具。下面是前端基建可以做的一些事情：

1. 脚手架工具

开发和维护一个通用的脚手架工具，可以帮助团队快速初始化项目结构、配置构建工具、集成常用的开发依赖等。

1. 组件库

开发和维护一个内部的组件库，包含常用的UI组件、业务组件等，提供给团队成员复用，减少重复开发的工作量。

1. 构建工具和打包工具

搭建和维护一套完善的构建和打包工具链，包括使用Webpack、Parcel等工具进行代码的压缩、合并、打包等工具，优化前端资源加载和性能。

1. 自动化测试工具

引入自动化测试工具，如Jest、Mocha等，编写和维护测试用例，进行单元测试、集成测试、UI测试等，提高代码质量和可靠性。

1. 文档工具

使用工具如JSDoc、Swagger等，生成项目的API文档、接口文档等，方便团队成员查阅和维护。

1. Git工作流

制定和规范团队的Git工作流程，使用版本控制工具管理代码，方便团队协作和代码回退。

1. 性能监控和优化

引入性能监控工具，如Lighthouse、Web Vitals等，对项目进行性能分析，优化网页加载速度、响应时间等。

1. 工程化规范

制定并推广团队的代码规范、目录结构规范等，提高代码的可读性、可维护性和可扩展性。

1. 持续集成和部署

搭建持续集成和部署系统，如Jenkins、Travis CI等，实现代码的自动构建、测试和部署，提高开发效率和代码质量。

1. 项目文档和知识库

建立一个内部的项目文档和知识库，记录项目的技术细节、开发经验、常见问题等，方便团队成员查阅和学习。

1. 代码质量工具

引入代码质量工具，如ESLint、Prettier等，对代码进行静态分析和格式化，提高代码的一致性和可读性。

1. 国际化支持

为项目添加国际化支持，可以通过引入国际化库，如i18next、vue-i18n等，实现多语言的切换和管理。

1. 错误监控和日志收集

引入错误监控工具，如Sentry、Bugsnag等，实时监控前端错误，并收集错误日志，方便进行问题排查和修复。

1. 前端性能优化工具

使用工具如WebPageTest、Chrome DevTools等，对项目进行性能分析和优化，提高页面加载速度、响应时间等。

1. 缓存管理

考虑合理利用浏览器缓存和服务端缓存，减少网络请求，提升用户访问速度和体验。

1. 移动端适配

针对移动端设备，采用响应式设计或使用CSS媒体查询等技术，实现移动端适配，保证页面在不同尺寸的设备上有良好的显示效果。

1. 安全防护

对项目进行安全审计，使用安全防护工具，如CSP（Content Security Policy）、XSS过滤等，保护网站免受常见的安全攻击。

1. 性能优化指标监控

监控和分析关键的性能指标，如页面加载时间、首次渲染时间、交互响应时间等，以便及时发现和解决性能问题。

1. 前端日志分析

使用日志分析工具，如ELK（Elasticsearch、Logstash、Kibana）等，对前端日志进行收集和分析，了解用户行为和页面异常情况。

1. 跨平台开发

考虑使用跨平台开发框架，如React Native、Flutter等，实现一套代码在多个平台上复用，提高开发效率。

1. 编辑器配置和插件

为团队提供统一的编辑器配置文件，包括代码格式化、语法高亮、代码自动补全等，并推荐常用的编辑器插件，提高开发效率。

1. 文档生成工具

使用工具如Docusaurus、VuePress等，为项目生成漂亮的文档网站，方便团队成员查阅和维护项目文档。

1. Mock数据和接口管理

搭建一个Mock服务器，用于模拟后端接口数据，方便前端开发和测试，同时可以考虑使用接口管理工具，如Swagger等，方便接口的定义和调试。

1. 前端监控和统计

引入前端监控工具，如Google Analytics、百度统计等，收集用户访问数据和行为信息，用于分析和优化用户体验。

1. 移动端调试工具

使用工具如Eruda、VConsole等，帮助在移动端设备上进行调试和错误排查，提高开发效率。

1. 自动化部署

配置自动化部署流程，将项目的代码自动部署到服务器或云平台，减少人工操作，提高发布效率和稳定性。

1. 前端团队协作工具

使用团队协作工具，如GitLab、Bitbucket等，提供代码托管、项目管理、任务分配和团队沟通等功能，增强团队协作效率。

1. 前端培训和知识分享

组织定期的前端培训和技术分享会，让团队成员相互学习和交流，推动技术的共享和提升。

1. 客户端性能优化

针对移动端应用，可以使用工具如React Native Performance、Weex等，进行客户端性能优化，提高应用的响应速度和流畅度。

1. 技术选型和评估

定期评估和研究前端技术的发展趋势，选择适用的技术栈和框架，以保持项目的竞争力和可持续发展。

1. 统一的状态管理

引入状态管理工具，如Redux、Vuex等，帮助团队管理前端应用的状态，提高代码的可维护性和可扩展性。

1. 前端日志记录

引入前端日志记录工具，如log4javascript、logrocket等，记录前端应用的运行日志，方便排查和解决问题。

1. 前端代码扫描

使用静态代码扫描工具，如SonarQube、CodeClimate等，对前端代码进行扫描和分析，发现潜在的问题和漏洞。

1. 前端数据可视化

使用数据可视化工具，如ECharts、Chart.js等，将数据以图表或图形的形式展示，增强数据的可理解性和可视化效果。

1. 前端容灾和故障处理

制定容灾方案和故障处理流程，对前端应用进行监控和预警，及时处理和恢复故障，提高系统的可靠性和稳定性。

1. 前端安全加固

对前端应用进行安全加固，如防止XSS攻击、CSRF攻击、数据加密等，保护用户数据的安全性和隐私。

1. 前端版本管理

建立前端代码的版本管理机制，使用工具如Git、SVN等，管理和追踪代码的变更，方便团队成员之间的协作和版本控制。

1. 前端数据缓存

考虑使用Local Storage、Session Storage等技术，对一些频繁使用的数据进行缓存，提高应用的性能和用户体验。

1. 前端代码分割

使用代码分割技术，如Webpack的动态导入（Dynamic Import），将代码按需加载，减少初始加载的资源大小，提高页面加载速度。

1. 前端性能监测工具

使用性能监测工具，如WebPageTest、GTmetrix等，监测前端应用的性能指标，如页面加载时间、资源加载时间等，进行性能优化。

61. webpack tree-shaking 在什么情况下会失效？【热度：71】

在以下情况下，webpack 的 tree-shaking 可能会失效：

1. 使用了 `sideEffects` 属性：在 webpack 的配置文件中，如果设置了 `sideEffects: false`，则 webpack 会假设所有模块都没有副作用，因此不会进行 tree-shaking。这通常用于避免某些模块被误标记为无用代码而被删除。
2. 动态导入：如果你使用了动态导入（例如使用了 `import()` 或 `require.ensure()`），webpack 无法静态分析模块的导入和导出，因此无法进行 tree-shaking。
3. 使用了 `commonjs` 模块语法：如果你的代码中使用了 `commonjs` 模块语法（例如使用了 `require()` 或 `module.exports`），webpack 无法进行静态分析，因此无法进行 tree-shaking。
4. 未使用 ES6 模块语法：tree-shaking 只能对 ES6 模块语法进行优化，如果你的代码中没有使用 ES6 模块语法，webpack 将无法进行 tree-shaking。
5. 模块被动态引用或条件引用：如果模块的引用方式是动态的（例如在循环或条件语句中引用），或者通过字符串拼接来引用模块，webpack 无法确定哪些模块会被引用，因此无法进行 tree-shaking。
6. 使用了副作用的代码：如果你的代码中包含有副作用的代码（例如在模块的顶级作用域中执行了一些操作），webpack 无法确定哪些代码是无用的，因此无法进行 tree-shaking。

需要注意的是，即使 tree-shaking 可能会失效，webpack 仍然会进行其他优化，例如代码压缩和代码分割等。同时，你可以通过设置 `mode` 为 `production`，来启用 webpack 的优化功能，包括 tree-shaking。

62. [性能] 常见性能指标获取方式? 【热度: 654】

常见性能指标获取方式

指标所反映的用户体验

下表概述了我们的性能指标如何对应到我们的问题之上:

开始了吗?

- 首次绘制、首次内容绘制 First Paint (FP) / First Contentful Paint (FCP)

有用吗?

- 首次有效绘制、主要元素时间点 First Meaningful Paint (FMP) / Hero Element Timing

能用吗?

- 可交互时间点 Time to Interactive (TTI)

好用吗?

- 慢会话 Long Tasks (从技术上来讲应该是: 没有慢会话)

页面何时开始渲染 - FP & FCP

这两个指标, 我们可以通过 `performance.getEntry`、`performance.getEntriesByName`、`performanceObserver` 来获取。

```
1 performance.getEntries().filter(item => item.name === 'first-paint')[0]; // 获取 FP 时间
2 performance.getEntries().filter(item => item.name === 'first-contentful-paint')[0]; // 获取 FCP 时间
3 performance.getEntriesByName('first-paint'); // 获取 FP 时间
4 performance.getEntriesByName('first-contentful-paint'); // 获取 FCP 时间// 也可以通过 performanceObserver 的方式获取
var observer = new PerformanceObserver(function(list, obj) {var entries = list.getEntries();
5     entries.forEach(item => {if (item.name === 'first-paint') {
6         ...
7     }if (item.name === 'first-contentful-paint') {
8         ...
9     }
10 });
11 });
12 observer.observe({type: 'paint'});
```

页面何时渲染主要内容 - FMP & SI & LCP

FMP，是一个已经废弃的性能指标。在实践过程中，由于 FMP 对页面加载的微小差异过于敏感，经常会出现结果不一致的情况。此外，该指标的定义依赖于特定于浏览器的实现细节，这意味着它不能标准化，也不能在所有 Web 浏览器中实现。目前，官方并没有提供有效的获取 FMP 的接口，因此性能分析的时候不再使用这个指标。

SI 和 FMP 一样，官方也没有提供有效的获取接口，只能通过 lighthouse 面板来查看，不作为 Sentry 等工具做性能分析的指标。

LCP，和 FMP 类似，但只聚焦页面首次加载时最大元素的绘制时间点，计算相对简单一些。通过 performanceObserver 这个接口，我们可以获取到 LCP 指标数据。

```
1 new PerformanceObserver((entryList) => {for (const entry of
  entryList.getEntries()) {console.log('LCP candidate:', entry.startTime,
    entry);
2   }
3 }).observe({type: 'largest-contentful-paint', buffered: true});
```

何时可以交互 - TTI & TBT

TTI，**time to interactive**，可交互时间，**lighthouse** 面板中的六大指标之一，用于测量页面从开始加载到主要资源完成渲染，并能够快速、可靠地响应用户输入所需的时间，值越小越好。官方资料: [TTI](#)。

和 FMP、SI 一样，官方并没有提供获取 TTI 的有效接口，只能通过 lighthouse 面板来查看。

计算方式如下：

1. 先进行 First Contentful Paint 首次内容绘制；
2. 沿时间轴正向搜索时长至少为 5 秒的安静窗口，其中，安静窗口的定义为：没有长任务且不超过 2 个正在处理的网络请求；
3. 沿时间轴反向搜索安静窗口之前的最后一个长任务，如果没有找到长任务，则在 FCP 步骤停止执行。
4. TTI 是安静窗口之前最后一个长任务的结束时间（如果没有找到长任务，则与 FCP 值相同）。

理解计算过程如下图：

TTI 表示的是完全可交互的时间，每个 web 系统，对 TTI 时间定义可能并不一定相同，上面只是提供一个计算较为通用的 TTI 的一个方式。

交互是否有延迟 - FID & MPFID & Long Task

FID 和 **MPFID** 可用来衡量用户首次交互延迟的情况，**Long Task** 用来衡量用户在使用应用的过程中遇到的延迟、阻塞情况。

FID，**first input delay**，首次输入延迟，测量从用户第一次与页面交互（例如当他们单击链接、点按按钮或使用由 JavaScript 驱动的自定义控件）直到浏览器对交互作出响应，并实际能够开始

处理事件处理程序所经过的时间。官方资料: FID。

FID 指标的值越小越好。通过 performanceObserver，我们可以获取到 FID 指标数据。

```
1 new PerformanceObserver((entryList) => {for (const entry of
  entryList.getEntries()) {const delay = entry.processingStart -
  entry.startTime;console.log('FID candidate:', delay, entry);
2   }
3 }).observe({type: 'first-input', buffered: true});
```

MPFID，Max Potential First Input Delay，最大潜在首次输入延迟，用于测量用户可能遇到的最坏情况的首次输入延迟。和 FMP 一样，这个指标已经被废弃不再使用。

Long Task，衡量用户在使用过程中遇到的交互延迟、阻塞情况。这个指标，可以告诉我们哪些任务执行耗费了 50ms 或更多时间。

```
1 new PerformanceObserver(function(list) {var perfEntries =
  list.getEntries();for (var i = 0; i < perfEntries.length; i++) {
2     ...
3   }
4 }).observe({type: 'longtask'});
```

页面视觉是否有稳定 - CLS

CLS，Cumulative Layout Shift，累积布局偏移，用于测量整个页面生命周期内发生的所有意外布局偏移中最大一连串的布局偏移情况。官方资料: CLS。

CLS, 值越小，表示页面视觉越稳定。通过 performanceObserver，我们可以获取到 CLS 指标数据。

```
1 new PerformanceObserver(function(list) {var perfEntries =
  list.getEntries();for (var i = 0; i < perfEntries.length; i++) {
2     ...
3   }
4 }).observe({type: 'layout-shift', buffered: true});
```

63. [性能] 衡量页面性能的指标有哪些？【热度: 1,045】

性能的核心问题

- 什么样的性能指标最能度量人的感觉？
- 怎样才能从我们的真实用户中获取这些指标？

- 如何用我们所获取的指标来确定一个页面表现得是否「快」？
- 当我们得知用户所感知的真实性能表现后，我们应该如何做才能避免重蹈覆辙，并在未来提高性能表现？

以用户为中心的性能指标

开始了吗？

页面开始加载了吗？得到了服务端的回应吗？

有用吗？

有足够用户期望看到的内容被渲染出来了吗？

能用吗？

用户能够与我们的页面交互了吗？还是依然在加载？

好用吗？

交互是否流畅、自然、没有延迟与其他的干扰？

首次绘制（First Paint）和首次内容绘制（First Contentful Paint）

首次绘制（FP）和首次内容绘制（FCP）。在浏览器导航并渲染出像素点后，这些指标点立即被标记。这些点对于用户而言十分重要，因为它回答了我们的第一个问题：开始了吗？

FP与FCP的主要区别在于，FP标记浏览器所渲染的任何与导航前内容不同的点，而FCP所标记的是来自于DOM中的内容，可能是文本、图片、SVG，甚至是canvas元素。

首次有效绘制（First Meaningful Paint）和主要元素时间点（Hero Element Timing）

首次有效绘制（FMP）回答了我们的问题：有用吗？对于现存的所有网页而言，我们不能去清晰地界定哪些元素的加载是「有用」的（因此目前尚无规范），

但是对于开发者他们自己而言，他们很容易知道页面的哪些部分对于用户而言是最为有用的。

这些页面中「最重要的部分」通常被称为主要元素。举一些例子，在YouTube的播放页面，播放器就是主要元素。在Twitter中可能是通知的图标，或者是第一条推文。在天气应用中，主要元素应是指定位置的预测信息。在一个新闻站点中，它可能是摘要，或者是第一张插图。

网页中总有一部分内容的重要性大于其余的部分。如果这部分的内容能够很快地被加载出来，用户甚至都不会在意其余部分的加载情况。

可交互时间（TTI）

可交互时间（TTI）标记了你的页面已经呈现了画面，并且能够响应用户输入的时间点。页面不能响应用户输入有以下常见的原因：

- 将被JavaScript所操作的元素还未被加载出来；
- 一些慢会话阻塞了浏览器的主线程（如我们在上一部分所描述的）

TTI所记录实际上是页面的JavaScript完成了初始化，主线程处于空闲的时间点。

long tasks

浏览器像是单线程的。某些情况下，一些任务将可能会花费很长的时间来执行，如果这种情况发生了，主线程阻塞，剩下的任务只能在队列中等待。

用户所感知到的可能是输入的延迟，或者是卡顿一下全部出现。这些是当今网页糟糕体验的主要来源。

Long Tasks API认为任何超过50毫秒的任务都可能存在潜在的问题，并将这些任务揭露给开发者。既然能够满足50毫秒内完成任务，也就能够符合RAIL在100毫秒内相应用户输入的要求。

指标所反映的用户体验

下表概述了我们的性能指标如何对应到我们的问题之上：

开始了吗？

- 首次绘制、首次内容绘制 First Paint (FP) / First Contentful Paint (FCP)

有用吗？

- 首次有效绘制、主要元素时间点 First Meaningful Paint (FMP) / Hero Element Timing

能用吗？

- 可交互时间点 Time to Interactive (TTI)

好用吗？

- 慢会话 Long Tasks (从技术上来讲应该是：没有慢会话)

获取指标

主要依赖浏览器提供的 api

- [PerformanceObserver](#)
- [PerformanceEntry](#)
- [DOMHighResTimeStamp](#)

PerformanceObserver 使用示范

要使用 PerformanceObserver，首先需要创建一个 PerformanceObserver 实例，并指定一个回调函数作为参数。回调函数将在性能事件触发时被调用。然后，通过 PerformanceObserver 的 observe() 方法去监听所关注的性能事件类型。

以下是一个使用 PerformanceObserver 的示例代码：

```
• // 创建回调函数function performanceCallback(list, observer) {  
  list.getEntries().forEach(entry => {console.log(entry.name + ": " + entry.startTime);  
});  
}
```

```
// 创建 PerformanceObserver 实例const observer = new  
PerformanceObserver(performanceCallback);
```

```
// 监听性能事件类型  
observer.observe({ entryTypes: ["measure", "paint"] });
```

在上面的代码中，定义了一个名为 `performanceCallback` 的回调函数，它接收两个参数：`list` 和 `observer`。`list` 参数是一个 `PerformanceEntryList` 对象，包含了所有触发的性能事件，可以通过 `getEntries()` 方法获取详细信息。`observer` 参数表示 `PerformanceObserver` 实例本身。

然后，通过 `new PerformanceObserver(performanceCallback)` 创建了一个 `PerformanceObserver` 实例，并将 `performanceCallback` 作为回调函数传递进去。

最后，通过 `observer.observe({ entryTypes: ["measure", "paint"] })` 方法，指定了要监听的性能事件类型，这里监听了 "measure" 和 "paint" 两种类型的事件。

当指定的性能事件类型发生时，回调函数将被调用，并传递触发事件的 `PerformanceEntry` 对象作为参数。开发者可以在回调函数中进一步处理和分析这些对象，以获取性能数据并进行优化。

需要注意的是，观察者模式是异步的，因此回调函数可能不会立即执行。另外，一旦创建了 `PerformanceObserver` 实例，需要调用其 `disconnect()` 方法来停止监听性能事件，避免内存泄漏。

以上是 `PerformanceObserver` 的基本用法，开发者可以根据实际需求和业务场景来灵活运用。

PerformanceObserver 如何统计FP、FCP

要使用 `PerformanceObserver` 统计 FP（First Paint）和 FCP（First Contentful Paint），可以按照以下步骤进行：

1. 创建 PerformanceObserver 实例，并指定一个回调函数作为参数。

```
1  const observer = new PerformanceObserver((list) => {const entries =  
  list.getEntries();  
2    entries.forEach((entry) => {if (entry.name === 'first-paint')  
    {console.log('FP:', entry.startTime);  
3      } else if (entry.name === 'first-contentful-paint') {console.log('FCP:',  
    entry.startTime);  
4      }  
5    });
```

```
6 });
```

1. 使用 PerformanceObserver 的 observe() 方法监听 'paint' 类型的性能事件。

```
1 observer.observe({ entryTypes: ['paint'] });
```

1. 在回调函数中，通过遍历 PerformanceEntryList 对象的 getEntries() 方法获取所有触发的性能事件，然后根据 entry.name 来判断是否是 FP 或 FCP。
2. 如果是 FP，可以通过 entry.startTime 获取其开始的时间戳，进行相应的处理。同样，如果是 FCP，也可以通过 entry.startTime 获取其开始的时间戳。

完整的示例代码如下：

```
1 const observer = new PerformanceObserver((list) => {const entries =  
  list.getEntries();  
2   entries.forEach((entry) => {if (entry.name === 'first-paint')  
    {console.log('FP:', entry.startTime);  
3     } else if (entry.name === 'first-contentful-paint') {console.log('FCP:',  
      entry.startTime);  
4     }  
5   });  
6 });  
7 observer.observe({ entryTypes: ['paint'] });
```

在上述代码中，创建了一个 PerformanceObserver 实例，并指定一个回调函数。在回调函数中，根据 entry.name 的值判断是否是 FP 或 FCP，并打印出对应的开始时间戳。然后通过调用 observer.observe() 方法监听 'paint' 类型的性能事件。

通过以上步骤，就可以使用 PerformanceObserver 统计 FP 和 FCP，并对这些性能指标进行进一步的处理和分析。

PerformanceObserver 如何统计 long tasks

要使用 PerformanceObserver 统计长任务（Long Tasks），可以按照以下步骤进行：

1. 创建 PerformanceObserver 实例，并指定一个回调函数作为参数。

```
1 const observer = new PerformanceObserver((list) => {const entries =  
  list.getEntries();  
2   entries.forEach((entry) => {console.log('Long Task:', entry);  
3   });
```

```
4 });
```

1. 使用 PerformanceObserver 的 observe() 方法监听 'longtask' 类型的性能事件。

```
1 observer.observe({ entryTypes: ['longtask'] });
```

1. 在回调函数中，通过遍历 PerformanceEntryList 对象的 getEntries() 方法获取所有触发的长任务事件。

2. 可以通过遍历获得的长任务事件数据，并进行进一步的处理和分析。

完整的示例代码如下：

```
1 const observer = new PerformanceObserver((list) => {const entries =  
  list.getEntries();  
2   entries.forEach((entry) => {console.log('Long Task:', entry);  
3   });  
4 });  
5 observer.observe({ entryTypes: ['longtask'] });
```

在上述代码中，创建了一个 PerformanceObserver 实例，并指定一个回调函数。在回调函数中，遍历 PerformanceEntryList 对象的 getEntries() 方法获取所有长任务事件，并打印出相关的长任务数据。

通过以上步骤，就可以使用 PerformanceObserver 统计长任务，并对这些长任务进行进一步的处理和分析。

补充： 页面性能指标有哪些？

以下是常见的页面性能指标，按照阶段顺序进行表述：

阶段	指标名称	描述
导航阶段	DNS 解析时间	浏览器解析域名并获取目标服务器IP地址所花费的时间
导航阶段	TCP 连接时间	浏览器与服务器建立TCP连接所花费的时间

导航阶段	SSL/TLS 握手时间	如果网站启用了 HTTPS，浏览器与服务 器进行 SSL/ TLS 握手所花费的时间
导航阶段	请求时间	浏览器发送 HTTP 请求并等待服务器响应的 时间
导航阶段	首字节时间 (TTFB)	浏览器收到服务器响应的第一个字 节所花费的时间
渲染阶段	DOM 解析时 间	浏览器将 HTML 文档解 析为 DOM 树的时间
渲染阶段	CSS 解析时 间	浏览器将 CSS 样式表 解析为 CSSOM 树的 时间
渲染阶段	首次渲染时 间 (FP)	浏览器将 DOM 树和 CSSOM 树合 并，开始绘 制页面的时 间
渲染阶段	首次内容绘 制时间 (FCP)	页面首次有 可见内容被 绘制的时间
渲染阶段	首次有意义 绘制时间 (FMP)	页面首次有 有意义的内容被绘制的 时间
交互阶段	首次输入延 迟时间 (FID)	用户首次与 页面进行交互（点击按钮、输入框等）后，页 面响应交互

		的时间
交互阶段	首次可交互时间 (TTI)	页面变得完全可交互 (用户可以进行大部分常规操作) 所花费的时间
交互阶段	页面完全加载时间 (Page Load)	页面所有资源 (包括图片、CSS、JavaScript 等) 加载完成、渲染完毕并且可交互的时间
用户体验阶段	页面响应时间	用户发起请求后, 页面完成响应所花费的时间
用户体验阶段	页面加载时间	用户打开页面到页面加载完成所花费的时间
用户体验阶段	页面交互性能	页面响应用户交互 (点击、滚动等) 的流畅程度

请注意, 以上仅为常见的页面性能指标, 并非所有指标都适用于每个网站。具体的指标选择取决于你的应用的特点和需求。

64. 浏览器本身是不支持模块化的, webpack 是如何通过文件打包, 让浏览器可以读取到前端各个模块的代码的? 【热度: 1,153】

浏览器本身不支持模块化的特性, 无法直接读取和执行模块化的代码。Webpack通过使用一种称为"模块化打包"的方式, 将模块化的代码转换为浏览器可以执行的形式。

Webpack使用了一个称为"模块系统"的机制, 通过对模块的依赖关系进行分析, 将所有依赖的模块打包为一个或多个包含所有依赖关系的文件。这些打包后的文件被称为“打包产物”或“bundle”。

在打包过程中，Webpack会根据配置文件中的入口点（entry point）来确定应用程序的起始模块。然后，它将从该模块开始递归地解析所有的依赖关系，包括其他模块或文件。Webpack通过识别模块之间的依赖关系，将它们合并到一个或多个打包产物中。

Webpack还会对打包产物进行一系列的优化，例如代码压缩、拆分和按需加载等，以提高应用程序的性能和加载速度。

当浏览器加载打包产物时，它会执行其中包含的代码，并通过模块系统来解决模块之间的依赖关系。Webpack为浏览器提供了一个称为"运行时"的文件，它是Webpack打包产物的一部分，负责处理模块之间的依赖关系，并提供加载和执行模块的功能。

通过Webpack的打包过程，浏览器可以读取和解析打包产物中的模块化代码，并顺利执行应用程序的逻辑。这样，开发者可以在编写代码时使用模块化的语法，而无需担心浏览器的兼容性问题。

65. 如何搭建一套灰度系统？【热度: 1,226】

这个是一个非常复杂的话题，没法直接给出答案，进提供一些实现的思路：

什么是灰度

灰度系统可以把流量划分成多份，一份走新版本代码，一份走老版本代码。

而且灰度系统支持设置流量的比例，比如可以把走新版本代码的流程设置为 5%，没啥问题再放到 10%，50%，最后放到 100% 全量。

这样可以把出现问题的影响降到最低。

而且灰度系统不止这一个用途，比如产品不确定某些改动是不是要的，就要做 AB 实验，也就是要把流量分成两份，一份走 A 版本代码，一份走 B 版本代码。

实现思路

1. 后端支持：灰度上线需要后端的支持，通过后端的灰度发布控制，可以将不同版本的前端应用分配给不同用户。
2. 搭建网关层：支持一部分用户分发到 A 版本，一部分用户分发到 B 版本（通常使用 nginx 搭建）。
3. 版本管控机制：使用版本控制系统（如Git、package.version、hash version 等）来管理不同版本的前端应用代码。在灰度上线时，可以根据需要切换到特定的版本。
4. 动态路由：通过动态路由配置，将用户请求导向不同版本的前端应用。例如，可以使用Nginx或其他反向代理服务器来实现动态路由。
5. 流量染色：使用Cookie或Session来控制用户的灰度版本访问。可以通过设置不同的Cookie值或Session标记，将用户分配到不同的灰度版本。
6. 更复杂的漏量配置：例如要根据部门、权限、角色等方式来开放灰度；可以使用让用户访问应用的时候，查询其权限和角色，根据权限和角色来分发不同的页面路由。

66. 如何理解研发流程和研发效率，如何保障研发效率

研发流程指的是从需求调研、设计、开发、测试、发布、维护等一系列环节组成的整个研发过程。它是实现软件产品的必经之路，可以帮助团队规范化、标准化研发流程，提高研发效率、降低研发成本、提高产品质量和用户满意度。

研发效率指的是在研发流程中，完成同样的工作所需要的时间和成本，也就是研发效率越高，就可以在同样的时间内完成更多的工作，并在更短的时间内推出产品，提高团队的竞争力和市场占有率。

保障研发效率可以从以下几个方面入手：

1. 优化研发流程：通过优化整个研发流程，缩短产品上线周期，降低研发成本，提高产品质量和用户满意度。
2. 确定明确的目标和任务：团队需要清楚地了解产品的目标和任务，明确每个人的责任和任务，以便更好地完成工作。
3. 提供优秀的工具和环境：为团队提供高效的工具和优秀的开发环境，帮助开发者更好地完成工作，提高工作效率。
4. 加强团队协作和沟通：团队成员之间需要建立良好的沟通和协作机制，通过有效的沟通和协作，提高工作效率和质量。
5. 持续学习和提高技能：团队成员需要不断学习新知识和提高技能，接受新技术，以便更好地完成工作，提高研发效率。

具体讲一下如何优化研发流程

优化研发流程可以从以下几个方面入手：

1. 需求管理：建立明确的需求管理机制，包括需求收集、需求筛选、需求优化、需求变更管理等，以确保需求的准确性、完整性、一致性和可追溯性。
2. 设计管理：建立明确的设计管理机制，包括设计评审、设计文档管理、设计变更管理等，以确保设计的合理性、可行性、可维护性和可扩展性。
3. 开发管理：建立高效的开发管理机制，包括任务分配、代码管理、代码审查、编码规范管理等，以确保开发的效率、质量和一致性。
4. 测试管理：建立严格的测试管理机制，包括测试计划、测试用例管理、测试环境管理、缺陷管理等，以确保产品的质量和稳定性。
5. 发布管理：建立有效的发布管理机制，包括版本控制、发布计划、发布测试、发布文档等，以确保产品的稳定性和用户满意度。
6. 迭代管理：建立迭代管理机制，通过不断的迭代和优化，提高产品的质量和用户满意度，保持团队的创新和活力。
7. 数据分析：建立数据分析机制，通过数据分析和用户反馈，不断优化产品和流程，提高团队的效率和竞争力。

优化研发流程需要建立标准化、规范化的流程和管理机制，不断优化和改善流程，并建立有效的沟通和协作机制，以提高团队的效率和产品质量，满足用户的需求和期望。

如何保证上述机制能够正确推进下去？

为了确保上述机制能够正确推进下去，可以采取以下措施：

1. 建立标准化的流程和管理机制，并将其纳入团队的日常工作流程中，确保每个团队成员都能够遵循。
2. 建立监控和评估机制，对各项机制进行定期的检查、评估和反馈，并根据实际情况进行调整和改进。
3. 建立协作和沟通机制，促进团队成员之间的协作和沟通，确保各项机制能够得到正确的执行和落实。
4. 建立培训和学习机制，定期培训团队成员，提高其专业技能和管理能力，增强其执行机制的能力和信心。
5. 采用项目管理工具和平台，对各项机制进行集中管理和监控，确保团队成员能够及时获取和共享必要的信息和资源。

要保证各项机制能够正确推进下去，需要建立完善的流程和管理机制，并通过监控、评估、协作、沟通、培训和技术支持等措施，确保团队成员能够正确执行和落实机制，从而提高团队的效率和竞争力。

确定明确需求目标和需求内容

首要任务是对需求进行拆解拆分，需要进行需求拆分来更好地管理和实现项目目标。以下是对大型项目进行需求拆分的建议：

对于一个大型项目，需要进行需求拆分来更好地管理和实现项目目标。以下是对大型项目进行需求拆分的建议：

1. 确定项目范围：明确项目的目标和范围，明确项目所需的主要功能和特性。
2. 列出项目需求：将项目所需的各种需求列出来，包括功能需求、性能需求、安全需求、可靠性需求等。
3. 进行需求分类：将需求进行分类，可以按照功能、用户、业务流程等方式进行分类。
4. 制定需求文档：根据需求分类，制定详细的需求文档，包括需求描述、优先级和验收标准等。
5. 制定项目计划：根据需求文档，制定项目计划，包括任务分解、时间安排和资源分配等。
6. 协同开发：在开发过程中，需要协同进行开发，进行需求变更和调整。
7. 进行验收和测试：在项目结束时，进行验收和测试，确保项目满足客户需求和要求。

67. webpack5 Module Federation 了解多少

概念

Webpack 5 的 Module Federation 是一项功能强大的功能，它允许将 JavaScript 应用程序拆分成独立的模块，并在不同的 Webpack 构建中共享这些模块。它解决了多个独立应用程序之间共享代码的问题，使得实现微前端架构变得更加容易。

Module Federation 可以将一个应用程序拆分成多个独立的子应用，每个子应用都可以被独立开发、部署和运行。每个子应用都可以通过配置指定需要共享的模块，然后将这些共享模块以动态方式加载到其他子应用中使用，而无需将这些模块打包进每个子应用的构建文件中。

Module Federation 的核心概念是 "容器" (Container) 和 "远程" (Remote)。容器是一个主应用程序，它可以加载和渲染其他子应用程序，而远程是一个独立的子应用程序，它提供了一些模块供其他子应用程序使用。

Module Federation 提供了一种简单的方式来定义远程模块，并在容器中引用这些远程模块。容器可以从远程加载模块，并通过配置将这些模块暴露给其他子应用程序。这样，子应用程序可以通过远程加载和使用容器中的模块，实现了模块的共享和动态加载。

Module Federation 在实现微前端架构时非常有用，可以将多个独立开发的子应用程序组合成一个整体，并实现共享模块和资源的灵活管理。它提供了一种解决方案，让多个团队可以独立开发和部署自己的子应用程序，同时又能够共享代码和资源，提高开发效率和整体性能。

Webpack 5 的 Module Federation 是一项用于实现微前端架构的功能，它可以将 JavaScript 应用程序拆分成独立的子应用程序，并通过动态加载和共享模块的方式实现子应用程序之间的交互和共享。

使用示范

下面是一个简单的示例，演示如何在 Webpack 5 中使用 Module Federation。

假设我们有两个独立的应用程序：App1 和 App2。我们将使用 Module Federation 将 App2 的模块共享给 App1。

首先，我们需要在 App2 的 Webpack 配置中启用 Module Federation：

```
1  // webpack.config.js (App2) const { ModuleFederationPlugin } =  
    require('webpack');  
2  
3  module.exports = { // ...其他配置  
4    plugins: [new ModuleFederationPlugin({  
5      name: 'app2',  
6      filename: 'remoteEntry.js',  
7      exposes: { './Button': './src/Button', // 暴露 App2 的 Button 模块  
8    }},  
9    ]},  
10  ],  
11  };
```

接下来，我们需要在 App1 的 Webpack 配置中配置远程加载 App2 的模块：

```

1  // webpack.config.js (App1)const { ModuleFederationPlugin } =
    require('webpack');
2
3  module.exports = { // ...其他配置
4    plugins: [new ModuleFederationPlugin({
5      name: 'app1',
6      remotes: {
7        app2: 'app2@http://localhost:3002/remoteEntry.js', // 远程加载 App2 的
        模块
8      },
9    }],
10  ],
11  };

```

在 App1 中，我们可以像使用本地模块一样使用 App2 的模块：

```

1  // App1import React from 'react';
2  import ReactDOM from 'react-dom';
3  import App2Button from 'app2/Button'; // 远程加载 App2 的 Button 模块
4  ReactDOM.render(<App2Button />, document.getElementById('root'));

```

在上面的示例中，我们通过 Module Federation 将 App2 的 Button 模块暴露给了 App1，然后在 App1 中可以直接通过 `import` 语句引入并使用。

需要注意的是，App1 需要在 `remotes` 配置中指定远程加载的模块，其中 `app2` 是一个远程模块的名称，而 `http://localhost:3002/remoteEntry.js` 是 App2 构建输出的远程入口文件。

这只是一个简单的示例，实际使用中可能涉及更复杂的配置和场景。但通过以上配置，我们可以实现在不同应用程序之间共享模块，并通过动态加载的方式使用远程模块。

68. source map 了解多少【热度: 396】

Source Map（源映射）作用

Source Map（源映射）是一种文件，用于将压缩、混淆或编译后的代码映射回原始的源代码，以便在调试过程中能够直接查看和调试源代码。它提供了压缩文件和源文件之间的映射关系，包括每个压缩文件中的代码位置、原始文件的路径和行号等信息。

Source Map的主要作用如下：

1. 调试：在开发过程中，源代码经常会被压缩、合并或转换为其他形式的代码，这使得在调试时直接查看和调试源代码变得困难。Source Map提供了一种方式，通过将压缩代码映射回源代码，开发者可以在调试器中直接查看和调试原始的、易于理解的源代码。

2. 错误追踪：当发生错误或异常时，浏览器或运行环境会提供错误信息，其中包含了压缩后的代码行号和列号。Source Map可以将这些行号和列号映射回源代码的行号和列号，帮助开发者定位和追踪错误。
3. 性能分析：Source Map可以提供压缩文件中每个代码片段对应的原始文件位置信息，这对于性能分析工具来说非常有用。性能分析工具可以使用Source Map来将性能数据映射回源代码，以便更准确地分析和优化代码性能。

Source Map的原理是通过在压缩文件中添加特定的注释或者生成独立的.map文件来存储映射关系。在调试过程中，浏览器或调试器会读取Source Map，并根据其中的映射关系将压缩代码中的行号、列号等信息映射回源代码的对应位置。

Source Map（源映射）实现原理

Source Map 的实现原理可以简单描述如下：

1. 生成 Source Map：在代码的压缩、混淆或编译过程中，生成器会创建一个 Source Map 对象，并收集相关的映射信息。这些信息包括原始文件路径、行号、列号以及对应的压缩文件中的位置信息。
2. 生成编码字符串：将收集到的映射信息使用 VLQ（Variable Length Quantity）编码进行压缩，将数字转换为可变长度的 Base64 编码字符串。VLQ 编码能够通过特定的规则将数字转换为可变长度的字符串，以减小 Source Map 的体积。
3. 关联 Source Map：在生成的压缩文件中，通过注释或独立的 .map 文件将 Source Map 关联到压缩文件。注释方式可以通过特定的注释语法将编码字符串直接嵌入到压缩文件中，而独立的 .map 文件则将编码字符串保存在一个独立的文件中。
4. 调试时使用 Source Map：在调试过程中，当开发者需要查看或调试源代码时，浏览器或调试工具会加载关联的 Source Map 文件，根据映射关系将压缩文件中的位置信息映射回源代码的对应位置。

通过这种方式，Source Map 实现了将压缩后的代码映射回原始源代码的功能，使得在调试、错误追踪和性能分析时能够更方便地操作和理解源代码。实际上，Source Map 的实现会有更多的细节和规范，但以上是其基本的实现原理概述。

69. 如何定制前端项目代码规范【热度: 1,155】

当按照上述步骤定制前端代码规范时，可以按照以下详细步骤执行：

1. 选择代码规范工具：
 - 研究可用的代码规范工具，如 ESLint、Prettier 等。
 - 比较各工具的功能、灵活性和社区支持，并选择最适合你团队和项目的工具。
2. 定义规范：
 - 针对项目的需求和团队的编码风格，制定代码规范的具体规则和约定。

- 参考行业内的代码规范，如 Airbnb JavaScript Style Guide、Google JavaScript Style Guide 等，以获取最佳实践和通用规则的参考。
- 考虑以下方面进行规范定义：
 - 缩进和空格：确定使用的缩进大小、空格还是制表符等。
 - 命名约定：定义变量、函数、类、文件等的命名约定。
 - 代码风格：确定代码的风格规则，如花括号的位置、换行符的使用等。
 - 语法约定：定义应该使用的语言特性和语法约定，如使用严格模式、避免使用特定的语言功能等。

3. 配置规范工具：

- 创建代码规范工具的配置文件，如 `.eslintrc.js` 或 `.prettierrc`。
- 在配置文件中指定所选规范工具的规则和选项，根据定义的规范进行配置。
- 根据需要，可以启用或禁用不同的规则，并进行其他自定义配置。

4. 集成到开发环境：

- 集成代码规范工具到开发环境，以实现自动检测和修复代码规范问题。
- 针对使用的编辑器或集成开发环境（IDE），安装相应的插件或扩展来支持代码规范检查和格式化。
- 配置构建工具（如 webpack）或版本控制系统（如 Git）的钩子，以在代码提交前运行代码规范检查。

5. 告知团队：

- 与团队成员分享定制的代码规范，并解释规范的目的和重要性。
- 提供规范的文档或指南，以便团队成员参考和遵循。
- 组织一个会议或培训，介绍代码规范并解答团队成员的疑问。

6. 定期审查和更新：

- 定期审查代码规范的有效性，并根据实际需求进行更新和调整。
- 接收团队成员的反馈和建议，以改进和优化代码规范。
- 在项目演进和技术发展的过程中，适时地更新代码规范以适应变化的需求。

以上步骤是一个通用的指南，你可以根据自己的团队和项目的特定要求进行调整和执行。此外，团队中的讨论和协商也是非常重要的，确保所有成员都参与到代码规范的制定和执行中。

70. 如何编写一个 babel 插件【热度: 1,062】

编写一个 babel 插件的基本步骤

编写一个 Babel 插件可以让你自定义转换、分析或操作 JavaScript 代码。下面是编写 Babel 插件的基本步骤：

1. 安装 Babel：首先，确保你已经安装了 Babel 的相关工具和依赖。可以使用 npm 或 yarn 安装 `@babel/core`、`@babel/preset-env` 和 `@babel/plugin-syntax-plugin-name`。
2. 创建插件文件：在项目中创建一个新的 JavaScript 文件，用于编写自定义插件的代码。命名约定是以 `babel-plugin-` 开头，例如 `babel-plugin-custom-plugin.js`。
3. 导出插件函数：在插件文件中，导出一个函数作为你的插件。这个函数将接收一个 Babel 的 `babel` 对象作为参数，包含了一些 Babel 的工具方法，如 `types` 和 `template`。

```
1 module.exports = function(babel) {// 插件代码
2   };
```

1. 实现插件逻辑：在插件函数内部，实现你的插件逻辑。可以使用 `babel.types` 对象提供的方法来操作抽象语法树（AST）节点，例如 `babel.types.VariableDeclaration`、`babel.types.CallExpression` 等。

```
1 module.exports = function(babel) {const { types: t } = babel;
2   return {
3     visitor: {
4       Identifier(path) {// 对每个 Identifier 节点进行处理const name =
        path.node.name;
5         path.node.name = name.toUpperCase();
6       }
7     }
8   };
9   };
```

1. 导出插件配置：为了让 Babel 可以识别你的插件，需要在插件函数中返回一个配置对象，其中 `visitor` 属性指定了你的插件要处理的 AST 节点类型和对应的处理函数。

```
1 module.exports = function(babel) {const { types: t } = babel;
2   return {
3     visitor: {// ...
4   }
5   };
6   };
```

1. 配置 Babel：在项目的 `.babelrc` 或 `babel.config.js` 文件中，将你的插件添加到 Babel 的插件列表中。

```
1  {
2    "plugins": ["babel-plugin-custom-plugin"]
3  }
```

1. 使用插件：运行 Babel，它将根据你的配置和代码中的语法，应用插件并对代码进行转换。

以上是编写 Babel 插件的基本步骤，可以根据具体需求和场景，实现各种自定义的转换、分析和操作逻辑。

babel 编写插件的时候有哪些核心方法

在编写 Babel 插件时，可以使用以下核心方法来操作抽象语法树（AST）节点：

1. `types` 对象：Babel 的 `types` 对象是你在插件中最常用的工具之一。它提供了一系列用于创建、访问和操作 AST 节点的方法。
 - `types.identifier(name)`：创建一个标识符节点，表示一个变量或函数的名称。
 - `types.stringLiteral(value)`：创建一个字符串字面量节点，表示一个字符串值。
 - `types.numericLiteral(value)`：创建一个数值字面量节点，表示一个数字值。
 - `types.booleanLiteral(value)`：创建一个布尔字面量节点，表示一个布尔值。
 - `types.objectExpression(properties)`：创建一个对象表达式节点，表示一个对象字面量。
 - `types.arrayExpression(elements)`：创建一个数组表达式节点，表示一个数组字面量。
 - `types.callExpression(callee, arguments)`：创建一个函数调用表达式节点，表示一个函数的调用。
 - `types.memberExpression(object, property)`：创建一个成员表达式节点，表示一个对象的成员访问。
2. 这些方法可以帮助你构建新的 AST 节点或访问现有的 AST 节点。
3. `path` 对象：Babel 的 `path` 对象代表 AST 中的一个路径，你可以通过该对象访问和操作 AST 节点。在插件的处理函数中，你将会经常使用 `path` 对象。
 - `path.node`：访问当前路径对应的节点。
 - `path.parent`：访问当前路径的父路径。
 - `path.scope`：访问当前路径的作用域。
 - `path.traverse(visitor)`：遍历当前路径的子路径，使用指定的访问者函数。

- `path.replaceWith(node)` : 替换当前路径的节点。
 - `path.remove()` : 移除当前路径的节点。
4. 这些方法可以帮助你遍历 AST 树时对节点进行修改、替换或删除。
 5. `traverse` 方法: `babel-traverse` 是 Babel 提供的一个独立的模块, 用于遍历和操作 AST。在插件中, 你可以使用 `traverse` 方法来遍历 AST 树并应用你的插件逻辑。
 - `traverse(ast, visitor)` : 使用指定的访问者函数遍历给定的 AST 树。
 6. `visitor` 是一个对象, 其中包含了处理不同类型节点的方法。通过在 `visitor` 对象中定义相应类型节点的处理函数, 你可以在遍历过程中针对特定类型的节点执行你的插件逻辑。
 7. `babel.template` 方法: `babel-template` 是 Babel 提供的一个独立模块, 用于根据字符串模板生成 AST 节点。你可以使用 `babel.template` 方法来创建包含特定模板结构的 AST 节点。
 - `babel.template(code, options)` : 根据指定的代码模板生成 AST 节点。
 8. `code` 参数是一个包含要生成的代码模板的字符串, 而 `options` 参数可以指定一些配置选项, 如 `preserveComments` 来保留注释。该方法将返回一个函数, 调用该函数并传入替换模板中的变量值, 即可生成对应的 AST 节点。
 9. 通过使用 `babel.template` 方法, 你可以更方便地创建复杂的 AST 节点结构, 尤其在需要生成大量相似结构的节点时非常有用。
 10. `babel.transform` 方法: `babel-transform` 是 Babel 提供的一个独立模块, 用于将 JavaScript 代码转换为 AST 或将 AST 转换回 JavaScript 代码。在编写插件时, 你可以使用 `babel.transform` 方法来进行代码转换操作。
 - `babel.transform(code, options)` : 将指定的代码转换为 AST 或将 AST 转换回代码。
 11. `code` 参数是一个包含要转换的 JavaScript 代码的字符串, 而 `options` 参数可以指定一些配置选项, 如 `plugins` 来指定要应用的插件。该方法将返回一个包含 `ast` 和 `code` 属性的对象, `ast` 属性表示生成的 AST 树, `code` 属性表示转换后的代码。
 12. 通过使用 `babel.transform` 方法, 你可以在插件内部对代码进行转换和处理, 将代码转换为 AST 进行修改, 然后再将修改后的 AST 转换回代码。

编写一个去除代码里面 `console.log` 的 babel 插件

以下是一个简单的 Babel 插件示例, 用于去除代码中的 `console.log` 语句:

```
1  // babel-plugin-remove-console.js
2  module.exports = function ({ types: t }) {
3    return {
4      visitor: {
5        CallExpression(path) {
6          const { callee } = path.node;
7          // 如果函数调用的名称是 console.log
8          if (
9            t.isMemberExpression(callee) &&
10            t.isIdentifier(callee.object, { name: 'console' }) &&
```

```
7         t.isIdentifier(callee.property, { name: 'log' })
8     ) { // 移除该函数调用
9         path.remove();
10    }
11 }
12 }
13 };
14 };
```

该插件会遍历代码中的函数调用表达式，如果发现是 `console.log`，则会移除该函数调用。

要使用该插件，可以在项目中安装并配置它。例如，创建一个 `.babelrc` 文件，并将该插件添加到 Babel 的插件列表中：

```
1  {
2    "plugins": ["../path/to/babel-plugin-remove-console.js"]
3  }
```

然后运行 Babel 命令或构建工具，它将应用该插件，并从代码中去除所有的 `console.log` 语句。

请注意，这只是一个简单的示例插件，仅适用于演示目的。在实际开发中，你可能需要更复杂的逻辑来处理不同的情况和要求。

71. SSR 了解多少【热度: 486】

SSR 原理是啥

服务器端渲染（Server-Side Rendering，SSR）是一种前端渲染方式，其核心原理是在服务器端将动态生成的 HTML 页面发送给客户端，以便客户端在接收到页面时直接渲染显示，而不是在客户端使用 JavaScript 动态生成页面。

核心原理如下：

1. 客户端发起请求：当用户访问一个 SSR 应用的页面时，客户端会向服务器发起请求。
2. 服务器处理请求：服务器接收到请求后，根据请求的路径和参数，获取对应的数据。
3. 数据获取和页面渲染：在服务器端，通过调用后端数据接口或其他数据源获取页面所需的数据。获取到数据后，服务器使用模板引擎或渲染框架将数据填充到页面模板中，生成完整的 HTML 页面。
4. HTML 页面返回给客户端：服务器将生成的 HTML 页面作为响应返回给客户端。
5. 客户端渲染：客户端接收到服务器返回的 HTML 页面后，直接渲染显示页面内容。由于服务器已经将数据填充到了页面中，客户端无需再进行数据获取和页面渲染的过程，提升了页面的加载速度和用户体验。

SSR 的核心原理是在服务器端生成完整的 HTML 页面，并将其发送给客户端，使客户端能够更快地显示页面内容。相比于传统的客户端渲染（CSR），SSR 可以改善首次加载时的白屏时间和搜索引擎抓取等方面的问题。同时，SSR 也可以更好地支持 SEO（搜索引擎优化）和提供更好的性能体验给用户。

实现方案

前端实现服务器端渲染（SSR）的方案有以下几种：

1. 基于 Node.js 的框架：使用 Node.js 的框架（如Express、Koa、Nest.js等）来构建服务器端应用程序，并在服务器端进行页面渲染。通过在服务器上运行 JavaScript 代码，将渲染好的页面直接返回给客户端。
2. 框架提供的 SSR 功能：一些前端框架（如Next.js、Nuxt.js、Angular Universal等）提供了内置的服务器端渲染功能，可以更方便地实现 SSR。这些框架会负责处理路由、数据预取和页面渲染等工作，并将渲染好的页面返回给客户端。
3. 预渲染：使用预渲染技术将静态页面提前生成，并部署到服务器上。在用户请求页面时，直接返回预渲染好的 HTML 页面，然后再由客户端接管页面的交互。这种方式适用于内容不经常变动或不需要动态数据的页面。
4. 后端代理：通过将前端应用程序的请求代理到服务器端，然后在服务器端进行页面渲染，并将渲染好的页面返回给客户端。这种方式适用于在现有的后端服务中添加 SSR 功能，而无需重写整个应用程序。

需要根据具体的项目需求、技术栈和框架选择合适的 SSR 实现方案。每种方案都有其优点和限制，综合考虑性能、开发体验、部署成本和维护复杂度等因素来做出决策。

72. Babel Polyfill 了解多少【热度: 200】

Babel Polyfill 作用是啥

Babel Polyfill 的作用是在旧版本浏览器中提供对新的JavaScript特性和API的支持。当使用Babel进行代码转换时，它只会转换语法，而不会转换新的API和全局对象（如Promise、Map、Set等）。

旧版本的浏览器可能不支持这些新的API和全局对象，因此在运行使用这些特性的代码时会抛出错误。为了解决这个问题，可以使用Babel Polyfill来填充缺失的功能，以确保代码在旧版本浏览器中正常运行。

Babel Polyfill通过修改全局对象和原型链，添加缺失的方法和属性，使得代码能够在不支持这些功能的浏览器中运行。它会检测当前环境的特性支持情况，并根据需要自动加载所需的Polyfill代码。

使用Babel Polyfill可以让开发人员在编写代码时不必过多考虑浏览器的兼容性，而专注于使用最新的JavaScript特性和API。它提供了一种简单方便的方式来填充浏览器的功能差异，确保代码在各种浏览器环境中具有一致的行为。

如何使用

要使用 Babel Polyfill，需要按照以下步骤进行设置：

1. 安装依赖：首先，确保你的项目已经安装了 Babel 相关的依赖包。这包括 `@babel/core`、`@babel/preset-env` 和 `@babel/polyfill`。你可以使用 npm 或者 yarn 进行安装：

```
1 npm install --save-dev @babel/core @babel/preset-env @babel/polyfill
```

1. 配置 Babel：在项目根目录下创建一个 `.babelrc` 文件，并添加以下配置：

```
1 {  
2   "presets": ["@babel/preset-env"]  
3 }
```

这样的配置将告诉 Babel 使用 `@babel/preset-env` 预设来进行转换。

1. 导入 Polyfill：在你的入口文件（通常是项目的主 JavaScript 文件）中导入 Babel Polyfill。你可以使用 import 语句或者 require 来导入 Polyfill：

使用 import（适用于 ES6 模块）：

```
1 import '@babel/polyfill';
```

使用 require（适用于 CommonJS 模块）：

```
1 require('@babel/polyfill');
```

导入 Polyfill 的位置很重要，通常应该在你的应用程序代码之前导入，以确保 Polyfill 在应用程序代码之前被加载和执行。

1. 配置目标浏览器：为了让 Babel Polyfill 根据目标浏览器进行特性填充，你可以在 `.babelrc` 文件中的 `@babel/preset-env` 配置中指定目标浏览器的选项。例如，你可以在配置中添加 `targets` 属性：

```
1 {  
2   "presets": [  
3     "@babel/preset-env",  
4     {  
5       "targets": {
```

```
6         "browsers": ["last 2 versions", "ie >= 11"]}]
7     ]
8 ]
9 }
```

这样，Polyfill 将根据所选的目标浏览器填充相应的功能。

完成以上步骤后，Babel Polyfill 将根据配置在目标浏览器中填充所需的功能，以确保你的代码在旧版本浏览器中正常运行。请注意，Polyfill 会增加你的应用程序的大小，因此请考虑仅填充所需的功能，以减小文件大小并优化性能。

按需加载 Polyfill

Babel Polyfill 默认会填充所有缺失的功能，但如果你只需要按需加载特定功能，可以使用 core-js 库的按需加载特性。下面是按需加载 Babel Polyfill 的步骤：

1. 安装依赖：确保你的项目已经安装了必要的依赖。除了之前提到的 Babel 相关依赖外，你还需要安装 `core-js`。

```
1 npm install --save-dev @babel/core @babel/preset-env core-js
```

1. 配置 Babel：在 `.babelrc` 文件中，添加以下配置：

```
1 {
2   "presets": [
3     ["@babel/preset-env",
4     {
5       "useBuiltIns": "usage",
6       "corejs": 3
7     }
8   ]
9 ]
10 }
```

`useBuiltIns` 选项设置为 `"usage"` 表示按需加载特性，而 `"corejs": 3` 指定了使用的 `core-js` 版本。

1. 导入 Polyfill：在需要使用特定功能的文件中，按需导入所需的 Polyfill。例如，如果你需要填充 `Promise` 和 `Array.prototype.includes`，你可以按如下方式导入：

```
1 import 'core-js/features/promise';
```



```
2 import 'core-js/features/array/includes';
```

这样只会加载和填充所需的功能，而不会加载整个 Polyfill 库。你可以根据具体的功能需求进行按需导入。

请注意，使用按需加载的方式可以减小应用程序的文件大小，并且只填充需要的功能，但需要确保在使用相关功能之前已经导入了相应的 Polyfill。

73. 幽灵依赖 是什么

"幽灵依赖" (Ghost Dependency) 是指在项目的 `node_modules` 目录中存在但未被实际使用的依赖包。

在使用 npm 或者其他包管理工具安装依赖包时，有时会出现安装了一些不需要的或者不正确的依赖包的情况。这些依赖包在项目中没有被显式地引用或使用，但仍然存在于 `node_modules` 目录中，占用了项目的存储空间。

幽灵依赖可能会产生以下问题：

1. 占用存储空间：未使用的依赖包会增加项目的体积，占用存储空间。对于大型项目或频繁部署的项目来说，这可能会造成不必要的存储资源浪费。
2. 增加构建时间：未使用的依赖包可能会增加构建过程中的解析和处理时间，导致构建过程变慢。这会影响开发人员的开发效率和项目的部署速度。
3. 潜在的安全风险：未使用的依赖包可能包含漏洞或安全风险，但由于没有使用，可能没有及时更新或修复这些问题，增加了项目的安全隐患。

为了解决幽灵依赖的问题，可以采取以下措施：

1. 定期检查依赖：定期检查项目的依赖，识别和删除未使用的依赖包。可以使用工具如 `npm-check-unused`、`depcheck` 等来帮助检测和清理未使用的依赖。
2. 精简依赖：审查项目的依赖关系，仅安装和保留必要的依赖包。避免过度依赖，只安装项目所需的模块，减少项目体积和构建时间。
3. 更新依赖包：确保项目中使用的依赖包都是最新版本，并及时更新已知的安全漏洞和问题。这可以通过定期检查依赖包的更新和使用工具如 `npm audit` 来实现。

通过处理幽灵依赖，可以提高项目的整洁性、性能和安全性，并减少不必要的开销和风险。

pnpm 是如何解决幽灵依赖问题的

pnpm 是一个基于 npm 的包管理工具，它采用了一种称为"快速硬链接 (Fast Hard Links)"的机制来解决幽灵依赖问题。

传统的 npm 或 yarn 安装依赖时，每个项目都会在 `node_modules` 目录下创建依赖包的副本。这导致了大量的重复文件，尤其是对于多个项目都使用同一依赖包时。

而 pnpm 通过使用快速硬链接机制，在全局的存储位置（默认为 `~/.pnpm-store`）只保存一份依赖包，而不是为每个项目都复制一份。这样就避免了幽灵依赖问题，减少了存储空间的占用。

当使用 pnpm 安装依赖时，它会在项目的 `node_modules` 目录下创建一个 `.modules.yaml` 文件，记录项目所需的依赖包和版本信息。实际的依赖包文件通过硬链接指向全局存储位置中的依赖包。这意味着不同项目之间可以共享相同的依赖包，但每个项目都拥有自己的依赖版本。

通过这种方式，pnpm 解决了幽灵依赖的问题，同时减少了存储空间的使用。它还具有一些其他的优点，如更快的安装速度、更少的网络传输和更好的缓存利用率。

需要注意的是，pnpm 仍然会将项目中的所有依赖安装在 `node_modules` 目录下，但它使用硬链接的方式避免了重复文件的复制，从而解决了幽灵依赖问题。

73. 如何将JavaScript代码解析成抽象语法树(AST)【热度: 1,169】

如何将JavaScript代码解析成抽象语法树

要将JavaScript代码解析成抽象语法树（Abstract Syntax Tree, AST），你可以使用工具或库来实现。以下是几种常用的方法：

1. Esprima: Esprima 是一个流行的JavaScript解析器，它可以将JavaScript代码解析成AST。你可以使用它的 JavaScript API 来将代码解析成AST对象。

```
1  const esprima = require('esprima');
2  const code = 'var x = 5;';
3  const ast = esprima.parseScript(code);
4  console.log(ast);
```

1. Acorn: Acorn 是另一个广泛使用的JavaScript解析器，它也可以将JavaScript代码解析成AST。你可以使用它的 JavaScript API 来解析代码并获取AST对象。

```
1  const acorn = require('acorn');
2  const code = 'var x = 5;';
3  const ast = acorn.parse(code, { ecmaVersion: 2020 });
4  console.log(ast);
```

1. Babel: Babel 是一个功能强大的JavaScript编译器，它可以将代码转换为AST，并提供了丰富的插件系统，用于转换和操作AST。你可以使用 Babel 的 API 来解析代码并获取AST对象。

```
1  const babelParser = require('@babel/parser');
```

```
2  const code = 'const x = 5;';
3  const ast = babelParser.parse(code, { sourceType: 'module' });
4  console.log(ast);
```

这些工具和库都可以将JavaScript代码解析成AST对象，从而使你能够对代码进行进一步的分析、转换或处理。你可以根据自己的需求选择其中之一，并根据其文档了解更多关于解析选项和AST节点的信息。

JavaScript代码解析成抽象语法树的原理是什么

JavaScript代码解析成抽象语法树（Abstract Syntax Tree, AST）的过程涉及以下几个主要步骤：

1. 词法分析（Lexical Analysis）：词法分析器（Lexer）将源代码拆分成词法单元（tokens），比如变量名、关键字、操作符、标点符号等。它根据一组定义好的规则（词法规范）来识别和分类这些词法单元。
2. 语法分析（Syntax Analysis）：语法分析器（Parser）接收词法分析器生成的词法单元，并根据语法规则构建AST。语法分析器使用上下文无关文法（Context-Free Grammar）来定义语言的语法规则，它通过递归下降、LR(1) 等算法来处理这些规则，以确定输入是否符合语法规则并生成相应的AST。
3. 构建AST：在语法分析的过程中，语法分析器根据语法规则构建AST。AST是一个树状结构，其中每个节点表示源代码中的一个语法结构，如表达式、语句、函数等。不同节点类型代表不同的语法结构，它们之间通过父子关系和兄弟关系来表示源代码的层次结构和逻辑关联。
4. 后续处理：生成AST后，可以进行进一步的处理和分析。这可能包括语义分析、类型推断、符号解析、代码优化等。这些步骤可以根据具体的需求和工具进行。

总结：将JavaScript代码解析成AST的过程是通过词法分析器将源代码拆分成词法单元，然后语法分析器根据语法规则构建AST。AST提供了对代码结构的抽象表示，便于进一步分析、转换和操作代码。

74. npm script 生命周期有哪些？【热度: 519】

安装和卸载

脚本名称	阶段	描述	执行时机
preinstall	pre	在 npm install 执行前运行，用于执行一些安装前的准备工作，例如检查依赖项或设置环境变量。	安装前
install, postinstall	install	在模块安装后执行，通常用于构建项目或者为其生成某些必须的文件，例如安装完毕后自动编译 TypeScript、ES6 等。	安装后
preuninstall	pre	在 npm uninstall 执行前运行，用于执行一些卸载前的准备工作。	卸载前
uninstall	post	在 npm uninstall 执行后运行，用于清理卸载后的一些操作。	卸载后
postuninstall	post	在 npm uninstall 执行后运行，用于执行一些卸载后的操作。	卸载后

发布和更新版本

脚本名称	阶段	描述	执行时机
------	----	----	------

脚本名称	阶段	描述	执行时机
prepublish	pre	在 publish（npm发布）执行前，运行 npm pack。	发布前
prepare	pre	在包被发布前或安装前执行，可以用来设置编译或验证文件的操作。	发布前、安装前
prepublishOnly	pre	在 npm publish 执行前运行，用于确保在 publish 命令执行时不会意外发布不必要的文件。	发布前
prepack	pre	在 npm pack（打包命令）执行前运行，用于执行一些打包前的准备工作。	打包前
postpack	post	在 npm pack 执行后运行，用于清理和重置打包相关的操作。	打包后
publish	post	在包被成功发布后执行。	发布后
postpublish	post	在包被成功发布后执行，用于执行一些发布后的操作。	发布后
		在项目版本号更新	更新版本号

preversion	pre	(npm version) 之前执行。	更新版本号前
version	post	在 npm version 执行后执行，用于执行一些版本更新后的操作。	更新版本号后
postversion	post	在项目版本号更新 (npm version) 之后执行。	更新版本号后

测试和运行

脚本名称	阶段	描述	执行时机
pretest	pre	在 npm test 执行前执行，用于执行某些测试前的准备工作。	测试前
test	test	执行 npm run test 命令时执行。通常用于执行单元测试，并返回任何错误状态。	默认测试阶段
posttest	post	在 npm test 执行后执行，用于执行某些测试后的操作。	测试后
prestart	pre	在 npm start 执行前运行，用于执行某些启动进程前的准备工作。	启动前
		执行 npm	

start	start	start 命令时执行，通常用于启动 Web 服务器、Node 服务器、实时编译器等。	默认启动阶段
poststart	post	在 npm start 执行后执行，用于执行某些启动进程后的操作。	启动后
prerestart	pre	在 npm restart 执行前执行，用于执行一些重新启动进程前的准备工作。	重新启动前
restart	stop/start	执行 npm restart 命令时执行，通常用于停止正在运行的 Node 服务器、Web 服务器等，然后以更新的源码重新启动服务。	默认重新启动阶段，但是该命令会触发停止和启动两个标准阶段
postrestart	post	在 npm restart 执行后执行，用于执行一些重新启动进程后的操作。	重新启动后

其他生命周期

脚本名称	阶段	描述	执行时机
prestop	pre	在 npm stop 执行前运行，用于执行某些停止进程前的准备工作。	停止前
stop	stop	执行 npm stop 命令时执行，通常用于停止正在运行的 Web 服务器、Node 服务器、实时编译器等。	默认停止阶段
poststop	post	在 npm stop 执行后执行，用于执行某些停止进程后的操作。	停止后

75. [React] react 是如何实现页面的快速响应？【热度：696】

react 是如何实现快速响应的？

我们日常使用App，浏览网页时，有两类场景会制约快速响应：

当遇到大计算量的操作或者设备性能不足使页面掉帧，导致卡顿。

发送网络请求后，由于需要等待数据返回才能进一步操作导致不能快速响应。

这两类场景可以概括为：

- CPU的瓶颈
- IO的瓶颈

CPU的瓶颈

主流浏览器刷新频率为60Hz，即每（1000ms / 60Hz）16.6ms浏览器刷新一次。

我们知道，JS可以操作DOM，GUI渲染线程与JS线程是互斥的。所以JS脚本执行和浏览器布局、绘制不能同时执行。

在每16.6ms时间内，需要完成如下工作： JS脚本执行 ----- 样式布局 ----- 样式绘制

当JS执行时间过长，超出了16.6ms，这次刷新就没有时间执行样式布局和样式绘制了。

比如我们可以通过一个循环，渲染列表 3000 个组件，那么这种渲染时间，就肯定是远超过 16.6 ms 的，页面就会感觉到卡顿。

如何解决这个问题呢？

答案是：在浏览器每一帧的时间中，预留一些时间给JS线程，React利用这部分时间更新组件（可以看到，在源码中，预留的初始时间是5ms）。

当预留的时间不够用时，React将线程控制权交还给浏览器使其有时间渲染UI，React则等待下一帧时间到来继续被中断的工作。

这种将长任务分拆到每一帧中，像蚂蚁搬家一样一次执行一小段任务的操作，被称为时间切片（time slice）

所以，解决CPU瓶颈的关键是实现时间切片，而时间切片的关键是：将同步的更新变为可中断的异步更新。

IO的瓶颈

网络延迟是前端开发者无法解决的。如何在网络延迟客观存在的情况下，减少用户对网络延迟的感知？

简单点儿来说，就是在点击页面跳转是时候提前去加载下一个页面的内容。或者在当前页面 hold .5s 左右时间，利用这个时间去加载下一个页面的内容。

从而达到下一个页面的快速交互

React实现了 `Suspense` 功能及配套的 hook——`useDeferredValue`。

而在源码内部，为了支持这些特性，同样需要将同步的更新变为可中断的异步更新。

76. npm 包管理了解多少？【热度: 1,321】

npm 是如何进行依赖管理的？

npm 是通过 package.json 文件来进行依赖管理的。当在项目中使用第三方库时，我们可以在 package.json 中添加对应的依赖项及版本号，npm 会根据 package.json 中的依赖关系，自动安装相应的依赖包及其依赖项。当我们执行 npm install 命令时，npm 会自动根据 package.json 中的依赖信息进行依赖包的安装。

npm 的依赖管理还涉及到依赖的版本控制，可以在 package.json 中指定对应的版本号，常见的版本号控制符号有：

- `^` (caret) : 匹配到次要版本号（第二个数字）的最新版本。例如，`^1.2.3` 表示安装 1.2.x 的最新版本（除了 1.3.0）。
- `~` (tilde) : 匹配到修订版本号（第三个数字）的最新版本。例如，`~1.2.3` 表示安装 1.2.3 到 1.2.x 的最新版本（除了 1.3.0）。
- `*` : 匹配到最新的版本。
- `=` : 匹配到大于或等于指定版本的最新版本。
- `<`、`<=`、`>` : 匹配到小于、小于等于或大于指定版本的最新版本。

在 npm 的依赖管理中，还有两种类型的依赖关系：生产依赖和开发依赖。生产依赖是指在应用程序运行时必须要加载的依赖，开发依赖是指在应用程序开发过程中使用的依赖。在 `package.json` 中，生产依赖使用 `dependencies` 字段，开发依赖使用 `devDependencies` 字段。这样可以项目更加清晰地管理其依赖关系。

npm 有缓存包的能力吗？

npm 有缓存包的能力。当你第一次使用 npm 安装一个包时，npm 会自动将该包缓存在本地。这样，当你下次需要安装相同版本的该包时，npm 就不必重新从网络上下载该包，而是直接使用缓存中的包。这样可以提高包的下载速度，节省网络带宽。

npm 的缓存位于本地文件系统中的隐藏目录。默认情况下，缓存位于当前用户的主目录下的 `.npm` 目录中。你可以使用以下命令查看 npm 缓存的路径：

```
1 npm config get cache
```

你也可以通过 `npm cache` 命令来管理 npm 缓存，例如清空缓存：

```
1 npm cache clean
```

或者查看缓存的统计信息：

```
1 npm cache ls
```

npm 是如何使用缓存中的包的？

使用缓存中的包可以通过以下两种方式实现：

- 使用 `npm ci` 命令

`npm ci` 命令会首先检查 `package-lock.json` 或 `npm-shrinkwrap.json` 文件，以确保安装的依赖与锁定的版本一致。然后，它会在 `node_modules` 目录下安装依赖，如果缓存中存在符合要求的包，`npm ci` 会直接从缓存中复制到 `node_modules` 目录下，而不需要重新下载和编译。

- 手动指定缓存路径

如果需要手动使用缓存中的包，可以在 `npm install` 命令中指定缓存路径，例如：

```
1 npm install --cache /path/to/npm-cache
```

然后，执行 `npm install` 命令时，`npm` 会尝试从指定的缓存路径中获取包，如果找到匹配的包，就会直接复制到 `node_modules` 目录下。

需要注意的是，手动指定缓存路径的方式可能会导致不同的项目之间共用缓存，因此需要确保缓存路径的唯一性。

77. npx 了解多少？【热度: 290】

npx 是什么

npx 是一个由 Node.js 官方提供的用于快速执行 npm 包中的可执行文件的工具。它可以帮助我们在不全局安装某些包的情况下，直接运行该包提供的命令行工具。npx 会在执行时，检查本地项目中是否安装了对应的依赖，如果没有安装则会自动下载安装，并执行命令。如果本地已经存在该依赖，则直接执行命令。

使用 npx 时，可以在命令行中输入要执行的包名加上其参数，例如：

```
1 npx create-react-app my-app
```

以上命令会在本地下载并运行 `create-react-app` 包中的可执行文件，创建一个名为 `my-app` 的 React 应用程序。

npx 会把远端的包下载到本地吗？

npx 不会像 `npm` 或 `yarn` 一样将包下载到本地的 `node_modules` 目录中。相反，它会在执行命令时，在本地缓存中寻找并下载包，然后执行该包中的命令。这样可以避免在开发过程中在全局安装大量的包，同时也可以确保使用的是最新版本的包。

npx 执行完成之后，下载的包是否会被删除？

是的，npx 会在执行完命令后删除下载的包。这是因为 npx 会在执行命令之前，将需要执行的包下载到一个临时目录中，并在执行完毕后删除该目录。这样可以避免在本地留下不必要的依赖包。如果需要保留依赖包，可以使用 `--no-cleanup` 选项来禁止删除下载的包。

78. npm lock 文件了解多少？【热度: 258】

作用

npm lock 文件（如 package-lock.json 或 yarn.lock）的作用是确保在不同机器上或在不同时间安装相同的依赖包时，获得相同的版本，以避免由于版本不一致而产生的问题。在安装依赖包时，npm lock 文件会锁定当前的依赖树，并记录每个依赖包的确切版本号和依赖关系。这样，在重新安装依赖包时，npm 将使用 lock 文件中记录的版本和依赖关系来安装依赖包，而不是根据 package.json 文件中的符号依赖去解析版本。这确保了依赖包版本的一致性。

生成原理

生成 npm lock 文件的原理如下：

- 当我们使用 npm install 或 npm ci 安装依赖包时，npm 会检查项目中的 package.json 文件，并根据其中的依赖包信息，生成一个 node_modules 目录用来存储这些依赖包。
- 在生成 node_modules 目录时，npm 会生成一个 npm-shrinkwrap.json 或 package-lock.json 文件，用来记录所有已经安装的依赖包的精确版本信息和依赖关系。这些信息是根据 package.json 文件和 node_modules 目录中实际安装的依赖包信息计算出来的。
- 在以后的安装过程中，npm 会先检查是否存在 npm-shrinkwrap.json 或 package-lock.json 文件，如果存在，就使用其中的依赖包版本信息来安装依赖包，而不是根据 package.json 文件中的信息重新计算依赖包版本。这样就可以确保每次安装时都使用相同的依赖包版本，避免了版本不一致导致的问题。

npm-shrinkwrap.json 是什么文件？

npm-shrinkwrap.json 文件是 Node.js 包管理工具 npm 生成的一份锁定文件，用于锁定项目依赖包的版本，确保团队成员在使用同一版本的依赖包，以避免在不同环境下因版本不一致而导致的问题。

与 package-lock.json 文件类似，npm-shrinkwrap.json 文件可以在项目中确保依赖包版本的一致性，但它与 package-lock.json 文件不同之处在于，它能够锁定所有的依赖包版本，包括间接依赖的包版本，而 package-lock.json 文件只会锁定直接依赖包的版本。

同时，使用 npm-shrinkwrap.json 文件也需要注意，在项目开发过程中，如果需要升级依赖包版本，需要手动更新 npm-shrinkwrap.json 文件中的对应依赖包版本号。

如何启用 npm-shrinkwrap.json

在项目根目录下使用以下命令可以生成 npm-shrinkwrap.json 文件：

```
1 npm shrinkwrap
```

如果需要在安装新的包时同时更新 npm-shrinkwrap.json 文件，可以使用以下命令：

```
1 npm shrinkwrap --dev
```

这个命令会把 devDependencies 也包括在生成的 npm-shrinkwrap.json 文件中。

79. 语义化版本 SemVer (Semantic Versioning) 了解多少?

Semantic Versioning (语义化版本) 是一种为软件组件定义版本号的规范。它使用 “major.minor.patch” 的格式来表示版本号。其中：

- Major (主版本号)：当你做了不兼容的 API 修改时，你需要更新主版本号。
- Minor (次版本号)：当你做了向下兼容的功能性新增时，你需要更新次版本号。
- Patch (修订号)：当你做了向下兼容的问题修正时，你需要更新修订号。

Semantic Versioning 的目的是为了让软件版本号的变化具有可读性和可预测性，这样用户就可以通过版本号来了解软件包的更新内容和影响。

版本更新

在升级版本时，常常使用一些符号来指定允许升级的范围，其中包括 ^ 和 ~ 等。

- ^ 表示向后兼容地升级版本号，只允许升级到次版本号或修订版本号，不允许升级到主版本号。
- ~ 表示只允许升级到修订版本号，不允许升级到次版本号或主版本号。

例如，对于版本号为 1.2.3：

- ^1.2.3 允许升级到 1.3.0、2.0.0 等次版本或主版本号的版本，但不允许升级到 1.2.4。
- ~1.2.3 只允许升级到 1.2.4、1.2.5 等修订版本号的版本，但不允许升级到 1.3.0、2.0.0 等更高的版本。

80. npm script 了解多少？【热度: 364】

什么是 npm script

npm 允许在 package.json 文件里面，使用 scripts 字段定义脚本命令。

```
1 {  
2   // ..."scripts": {"build": "node build.js"  
3 }  
4 }
```

上面代码是 `package.json` 文件的一个片段，里面的 `scripts` 字段是一个对象。它的每一个属性，对应一段脚本。比如，`build` 命令对应的脚本是 `node build.js`。

命令行下使用 `npm run` 命令，就可以执行这段脚本。

\$ npm run build

等同于执行\$ node build.js

这些定义在 `package.json` 里面的脚本，就称为 npm 脚本。它的优点很多。

- 项目的相关脚本，可以集中在一个地方。
- 不同项目的脚本命令，只要功能相同，就可以有同样的对外接口。
- 用户不需要知道怎么测试你的项目，只要运行 `npm run test` 即可。
- 可以利用 npm 提供的很多辅助功能。

查看当前项目的所有 npm 脚本命令，可以使用不带任何参数的 `npm run` 命令。

```
1 $ npm run
```

原理

`npm run` 实际上是 `npm run-script` 命令的简写

- 从 `package.json` 文件中读取 `scripts` 对象里面的全部配置；
- 以传给 `npm run` 的第一个参数作为键，如 `dev`，在 `scripts` 对象里面获取对应的值作为接下来要执行的命令，如果没找到直接报错；

每当执行 `npm run`，就会自动新建一个 Shell，在这个 Shell 里面执行指定的脚本命令。因此，只要是 Shell（一般是 Bash）可以运行的命令，就可以写在 npm 脚本里面。

比较特别的是，`npm run` 新建的这个 Shell，会将当前目录的 `node_modules/.bin` 子目录加入 `PATH` 变量，执行结束后，再将 `PATH` 变量恢复原样。

这意味着，当前目录的 `node_modules/.bin` 子目录里面的所有脚本，都可以直接用脚本名调用，而不必加上路径。比如，当前项目的依赖里面有 Mocha，只要直接写 `mocha test` 就可以了。

```
1 "test": "mocha test"
```

通配符

由于 npm 脚本就是 Shell 脚本，因为可以使用 Shell 通配符。


```
1  "lint": "jshint *.js""lint": "jshint **/*.js"
```

上面代码中，*表示任意文件名，**表示任意一层子目录。

如果要将通配符传入原始命令，防止被 Shell 转义，要将星号转义。

```
1  "test": "tap test/*.js"
```

传参

给 npm script 传递参数 给 npm script 传递参数 eslint 内置了代码风格自动修复模式，只需给它传入 -fix 参数即可，在 scripts 中声明检查代码命令的同时你可能也需要声明修复代码的命令，面对这种需求，大多数同学可能会忍不住复制粘贴，如下：

- @@ -5,6 +5,7 @@
 "lint:js": "eslint *.js",
 "lint:js:fix": "eslint *.js --fix",

在 lint:js 命令比较短的时候复制粘贴的方法简单粗暴有效，但是当 lint:js 命令变的很长之后，难免后续会有人改了 lint:js 而忘记修改 lint:js:fix（别问我为啥，我就是踩着坑过来的），更健壮的做法是，在运行 npm script 时给定额外的参数，代码修改如下：

- @@ -5,6 +5,7 @@
 "lint:js": "eslint *.js",
 "lint:js:fix": "npm run lint:js -- --fix",

要格外注意 --fix 参数前面的 -- 分隔符，意指要给 npm run lint:js 实际指向的命令传递额外的参数。

注释

```
1  "test": "# 运行所有代码检查和单元测试 .    npm-run-all --parallel lint:* mocha"
```

或者在单独的文件中可以自由给它添加注释

日志

```
1  npm run test --loglevel silent
2  npm run test --silent
3  npm run test -s
```

这个日志级别，只有命令本身的输出，读起来非常的简洁

```
1  npm run test --loglevel verbose
2  npm run test --verbose
3  npm run test -d
```

这个日志级别，详细打印出了每个步骤的参数、返回值

执行顺序

如果 npm 脚本里面需要执行多个任务，那么需要明确它们的执行顺序。

如果是并行执行（即同时的平行执行），可以使用&符号。

```
1  $ npm run script1.js & npm run script2.js
```

如果是继发执行（即只有前一个任务成功，才执行下一个任务），可以使用&&符号。

```
1  $ npm run script1.js && npm run script2.js
```

这两个符号是 Bash 的功能。此外，还可以使用 node 的任务管理模块：`npm-run-all`、`script-runner`

- `// 串行`

```
{  
  "test": "npm run lint:js && npm run lint:css && npm run lint:json && npm run lint:markdown"  
  "test": "npm-run-all lint:js lint:css lint:json lint:markdown"  
},  
// 并行 --parallel  
{  
  "test": "npm-run-all lint:*"  
  "test": "npm-run-all --parallel lint:* mocha"  
}
```

默认值

一般来说，npm 脚本由用户提供。但是，npm 对两个脚本提供了默认值。也就是说，这两个脚本不用定义，就可以直接使用。

```
1  "start": "node server.js",  
2  "install": "node-gyp rebuild"
```

上面代码中，npm run start的默认值是node server.js，前提是项目根目录下有server.js这个脚本；npm run install的默认值是node-gyp rebuild，前提是项目根目录下有binding.gyp文件。

钩子

npm 脚本有pre和post两个钩子。举例来说，build脚本命令的钩子就是prebuild和postbuild。

```
1  "prebuild": "echo I run before the build script",  
2  "build": "cross-env NODE_ENV=production webpack",  
3  "postbuild": "echo I run after the build script"
```

用户执行npm run build的时候，会自动按照下面的顺序执行。

npm run prebuild && npm run build && npm run postbuild 因此，可以在这两个钩子里面，完成一些准备工作和清理工作。下面是一个例子。

```
1  "clean": "rimraf ./dist && mkdir dist",  
2  "prebuild": "npm run clean",
```

```
3  "build": "cross-env NODE_ENV=production webpack"
```

npm 默认提供下面这些钩子。

```
1  prepublish, postpublish
2  preinstall, postinstall
3  preuninstall, postuninstall
4  preversion, postversion
5  pretest, posttest
6  prestop, poststop
7  prestart, poststart
8  prerestart, postrestart
```

自定义的脚本命令也可以加上pre和post钩子。比如，myscript这个脚本命令，也有premyscript和postmyscript钩子。不过，双重的pre和post无效，比如prepretest和postposttest是无效的。

npm 提供一个npm_lifecycle_event变量，返回当前正在运行的脚本名称，比如pretest、test、posttest等等。所以，可以利用这个变量，在同一个脚本文件里面，为不同的npm scripts命令编写代码。请看下面的例子。

```
1  const TARGET = process.env.npm_lifecycle_event;
2
3  if (TARGET === 'test') {console.log(`Running the test task!`);
4  }
5
6  if (TARGET === 'pretest') {console.log(`Running the pretest task!`);
7  }
8
9  if (TARGET === 'posttest') {console.log(`Running the posttest task!`);
10 }
```

注意，prepublish这个钩子不仅会在npm publish命令之前运行，还会在npm install（不带任何参数）命令之前运行。这种行为很容易让用户感到困惑，所以 npm 4 引入了一个新的钩子prepare，行为等同于prepublish，而从 npm 5 开始，prepublish将只在npm publish命令之前运行。

简写形式

四个常用的 npm 脚本有简写形式。

```
1  npm start是npm run start
2  npm stop是npm run stop的简写
3  npm test是npm run test的简写
4  npm restart是npm run stop && npm run restart && npm run start的简写
```

npm start、npm stop和npm restart都比较好理解，而npm restart是一个复合命令，实际上会执行三个脚本命令：stop、restart、start。具体的执行顺序如下。

```
1  prerestart
2  prestop
3  stop
4  poststop
5  restart
6  prestart
7  start
8  poststart
9  postrestart
```

变量

npm 脚本有一个非常强大的功能，就是可以使用 npm 的内部变量。

运行 `npm run env` 能拿到完整的变量列表

使用 `npm run env | grep npm_package | sort` 拿到部分排序后的环境变量

通过npm_package_前缀，npm 脚本可以拿到package.json里面的字段。比如，下面是一个package.json。

```
1  {
2    "name": "foo",
3    "version": "1.2.5",
4    "config" : { "port" : "8080" },
5    "scripts" : { "start" : "node server.js" }
6  }
```

那么，变量npm_package_name返回foo，变量npm_package_version返回1.2.5。

```
1  // view.js
2  console.log(process.env.npm_package_name); // foo
```

```
3 console.log(process.env.npm_package_version); // 1.2.5
```

上面代码中，我们通过环境变量`process.env`对象，拿到`package.json`的字段值。如果是 Bash 脚本，可以用 `$npm_package_name` 和 `$npm_package_version` 取到这两个值。

```
$npm_package_scripts_start
```

结合 npm script 和 git-hooks

Git 在代码版本管理之外，也提供了类似 npm script 里 pre、post 的钩子机制，叫做 Git Hooks，钩子机制能让我们在代码 commit、push 之前（后）做自己想做的事情。

前端社区里有多种结合 npm script 和 git-hooks 的方案，比如 `pre-commit`、`husky`，相比较而言 `husky` 更好用，它支持更多的 Git Hooks 种类，再结合 `lint-staged` 使用就更好了。

[了解更多](#)

常用脚本示例

```
1 // 删除目录"clean": "rimraf dist/*",
2
3 // 本地搭建一个 HTTP 服务"serve": "http-server -p 9090 dist/",
4
5 // 打开浏览器"open:dev": "opener http://localhost:9090",
6
7 // 实时刷新"livereload": "live-reload --port 9091 dist/",
8
9 // 构建 HTML 文件"build:html": "jade index.jade > dist/index.html",
10
11 // 只要 CSS 文件有变动，就重新执行构建"watch:css": "watch 'npm run build:css'
    assets/styles/",
12 // 只要 HTML 文件有变动，就重新执行构建
13 "watch:html": "watch 'npm run build:html' assets/html",
14 // 部署到 Amazon S3
15 "deploy:prod": "s3-cli sync ./dist/ s3://example-com/prod-site/",// 构建
    favicon"build:favicon": "node scripts/favicon.js",
```

81. package.json 配置了解多少？【热度: 747】

`package.json` 常见配置分类：

- 描述配置

- 文件配置
- 脚本配置
- 依赖配置
- 发布配置
- 系统配置
- 第三方配置

`package.json` 作用：存储一切与项目相关的配置，例如项目基本信息、外界访问项目的方式、项目内置脚本、项目依赖等。

描述配置

主要是项目的基本信息，包括名称，版本，描述，仓库，作者等，部分会展示在 npm 官网上。

```
1  {"name": "react", // 项目名称 or npm包名 "version": "18.2.0", // 版本号, 开源项目的
   // 版本号通常遵循 semver 语义化规范 "repository": {"type": "git", "url":
   // "https://github.com/facebook/react.git", "directory": "packages/react"
2   }, // 项目的仓库地址及版本控制信息 "description": "React is a JavaScript
   // library for building user interfaces.", // 项目描述 (展示于 npm 官
   // 网) "keywords":
   // ["ant", "component", "components", "design", "framework", "frontend", "react", "react
   // -component", "ui"
3   ], // 项目技术关键词 "homepage": "https://reactjs.org/", // 项目的主页链接, 通常
   // 是项目 github 链接, 项目官网或文档首页 "bugs":
   // "https://github.com/vuejs/core/issues", // 项目 bug 反馈地址, 通常是 github
   // issue 页面的链接 "author": "Li jiaxun", // 作者信息 "private": true, // 私有项目,
   // 若为 true 则无法发布到 npm 官网上
4 }
```

文件配置

包括项目所包含的文件，以及入口等信息。

```
1  { // 🔥@files: 指定需要跟随一起发布的内容, 控制 npm 包的大小。// 发布时默认会包括
   // package.json, license, README 和 main 字段里指定的文件。忽略 node_modules,
   // lockfile 等文件。在此基础上, 可以指定更多需要一起发布的内容。(单独的文件/整个文件夹/使
   // 用通配符匹配到的文件)// 一般情况下, files 里会指定构建出来的产物以及类型文件, 而 src,
   // test 等目录下的文件不需要跟随发布。"files": ["filename.js", "directory/", "glob/*.
   // {js,json}"
2   }, // 🔥@type: 'module' => 用 ESM 解释 .js 文件(此时访问 CJS 模块文件需要 .cjs 后
   // 缀); 反之同理。"type": "module", // 🔥@main: 项目入口文件。// if "type: 'module'"
   // => 指向 ESM 模块规范的项目入口文件 else => CommonJS 模块规范的项目入口文件。"main":
```



```

"./index.cjs",// @browser: web端项目入口文件路径。该路径下文件不允许在 server 端使用。
"browser": "./browser/index.js",// 🔥@module: ESM 规范模块的项目入口文件。
"module": "./index.js",// 🔥@exports: 配置不同环境对应的模块入口文件(优先级最高 > main)。// 作用1: 以别名形式封装包的子路径。"import
packageA/dist/css/index.css" => "import packageA/style"// 作用2: 以 '.' 为别名时, 表示模块主入口, 可以看做是 "main"/"module" 等字段功能的集合。// 作用3: 设置模块访问权限。exports 限制使用者不可以访问未在"exports"中定义的任何其他路径。// 作用4: 提供了项目(包)多入口访问的途径。例如下面的 './docs' 和 './components'
"exports": {".": {
  "require": "./index.cjs", "import": "./index.js"
}, "./docs": {
  "require": "./docs/index.cjs", "import": "./docs/index.js"
}, "./components": {
  "require": "./components/index.cjs", "import":
  "./components/index.js"
}, "./style": "./dist/css/index.css"
},
// 🔥@workspaces: 项目的工作区配置, 用于在本地的根目录下管理多个子项目。
// 可以自动地在 npm install 时将 workspaces 下面的包, 软链到根目录的
node_modules 中, 不用手动执行 npm link 操作。
// 通常子项目都会平铺管理在 packages 目录下, "packages/*" 表示将该路径下所有子项目的
node_modules 软链到根目录。
"workspaces": [
  "packages/*",
],
}

```

当一个项目同时定义了 main, browser 和 module, 像 webpack, rollup 等构建工具会感知这些字段, 并会根据环境以及不同的模块规范来进行不同的入口文件查找。

exports 字段详解

参考 [阮一峰](#)

exports 字段的优先级高于 main 字段。它有多种用法。

1. 子目录别名

package.json 文件的 exports 字段可以指定脚本或子目录的别名, 此时它的前缀的参照拼接路径是包名。

```

1 // ./node_modules/es-module-package/package.json
2 {"exports": {"./submodule": "./src/submodule.js"
3   }
4 }

```

上面的代码指定 src/submodule.js 别名为 submodule, 然后就可以从别名加载这个文件。

```
1 import submodule from 'es-module-package/submodule';
2 // 加载 ./node_modules/es-module-package/src/submodule.js
```

如果没有指定别名，就不能用“模块+脚本名”这种形式加载脚本。

```
1 // 报错import submodule from 'es-module-package/private-module.js';
2
3 // 不报错import submodule from './node_modules/es-module-package/private-
  module.js';
```

1. main 的别名

`exports` 字段的别名如果是 `.`，就代表模块的主入口，优先级高于 `main` 字段，并且可以直接简写成 `exports` 字段的值。

```
1 {"exports": {".": "./main.js"}}
2 }
3 }
4
5 // 等同于
6 {"exports": "./main.js"}
7 }
```

由于 `exports` 字段只有支持 ES6 的 Node.js 才认识，所以可以同时添加 `main` 字段来兼容旧版本的 Node.js。

```
1 {
2   "main": "./main-legacy.cjs",
3   "exports": {
4     ".": "./main-modern.cjs"
5   }
6 }
```

上面代码中，老版本的 Node.js（不支持 ES6 模块）的入口文件是 `main-legacy.cjs`，新版本的 Node.js 的入口文件是 `main-modern.cjs`。

1. 条件加载

利用 `.` 这个别名，可以为 ES6 模块和 CommonJS 指定不同的入口。

```
1  {
2    "type": "module",
3    "exports": {
4      ".": {
5        "require": "./main.cjs",
6        "default": "./main.js"
7      }
8    }
}
```

上面代码中，别名 `.` 的 `require` 条件指定 `require()` 命令的入口文件（即 CommonJS 的入口），`default` 条件指定其他情况的入口，此处配置了 `type: 'module'`，因此默认命中 ESM 模块规范。

脚本配置

```
1  {
2    // npm run {scripts} / yarn {scripts} 等命令行方式启动预设的脚本 "scripts":
    {"build": "webpack"}
3  },
4    // 设置 scripts 里的脚本在运行时的参数 "config": {"port": "3001"}
5  },
6  }
```

依赖配置

项目依赖其他包引用的相关信息。

```
1  { // 项目生产环境(运行时)下需要用到的依赖 // 使用 npm install xxx 或则 npm install
    xxx --save 时，会被自动插入到该字段中。 "dependencies": {"react":
    "^18.2.0", "react-dom": "^18.2.0"}
2  }, // 项目开发环境需要用到而运行时不需要的依赖，用于辅助开发 // 使用 npm install xxx
    -D 或者 npm install xxx --save-dev 时，会被自动插入到该字段中。 "devDependencies":
    {"webpack": "^5.69.0"}
3  }, // 同伴依赖 // 一种特殊的依赖，不会被自动安装，通常用于表示与另一个包的依赖与兼容性关
    系来警示使用者。 // 比如我们安装 A，A 的正常使用依赖 B@2.x 版本，那么 B@2.x 就应该被列
    在 A 的 peerDependencies 下，表示“如果你使用我，那么你也需要安装 B，并且至少是 2.x 版
    本”。 // 比如 React 组件库 Ant Design，它的 package.json 里 peerDependencies 为 //
    表示如果你使用 Ant Design，那么你的项目也应该安装 react 和 react-dom，并且版本需要大
    于等于 16.9.0。 "peerDependencies": {"react": ">=16.9.0", "react-dom": ">=16.9.0"}
4  }, // optionalDependencies // 可选依赖，顾名思义，表示依赖是可选的，它不会阻塞主功能
    的使用，安装或者引入失败也无妨。这类依赖如果安装失败，那么 npm 的整个安装过程也是成功
    的。 // 比如我们使用 colors 这个包来对 console.log 打印的信息进行着色来增强和区分提示，
    但它并不是必需的，所以可以将其加入到 optionalDependencies，并且在运行时处理引入失败的
```

```

逻辑。// 使用 npm install xxx -O 或者 npm install xxx --save-optional 时, 依赖会
被自动插入到该字段中。"optionalDependencies": {"colors": "^1.4.0"
5   },// peerDependenciesMeta// 同伴依赖也可以使用 peerDependenciesMeta 将其指定为可
选的。"peerDependencies": {"colors": "^1.4.0"
6   }, "peerDependenciesMeta": {"colors": {"optional": true
7   }
8   },// bundleDependencies// 打包依赖。它的值是一个数组, 在发布包时,
bundleDependencies 里面的依赖都会被一起打包。// 比如指定 react 和 react-dom 为打包
依赖: // 在执行 npm pack 打包生成 tgz 压缩包中, 将出现 node_modules 并包含 react
和 react-dom。// 需要注意的是, 这个字段数组中的值必须是在 dependencies,
devDependencies 两个里面声明过的依赖才行。// 普通依赖通常从 npm registry 安装, 但当
你想用一个不在 npm registry 里的包, 或者一个被修改过的第三方包时, 打包依赖会比普通依赖
更好用。"bundleDependencies": ["react", "react-dom"
9   ],// overrides// overrides 可以重写项目依赖的依赖, 及其依赖树下某个依赖的版本号, 进
行包的替换。// 比如某个依赖 A, 由于一些原因它依赖的包 foo@1.0.0 需要替换, 我们可以使用
overrides 修改 foo 的版本号: "overrides": {"foo": "1.1.0-patch"
10  }
11 }

```

发布配置

主要是和项目发布相关的配置。

`private`

如果是私有项目, 不希望发布到公共 `npm` 仓库上, 可以将 `private` 设为 `true`。

```
1  "private": true
```

`publishConfig`

顾名思义, `publishConfig` 就是 `npm` 包发布时使用的配置。

比如在安装依赖时指定了 `registry` 为 `taobao` 镜像源, 但发布时希望在公网发布, 就可以指定 `publishConfig.registry`。

```
1  "publishConfig": {"registry": "https://registry.npmjs.org/"
2  }
```

系统配置

和项目关联的系统配置, 比如 `node` 版本或操作系统兼容性之类。这些要求只会起到提示警告的作用, 即使用户的环境不符合要求, 也不影响安装依赖包。

engines

一些项目由于兼容性问题会对 node 或者包管理器有特定的版本号要求，比如：

```
1  "engines": {"node": ">=14 <16", "pnpm": ">7"}
2  }
```

要求 node 版本大于等于 14 且小于 16，同时 pnpm 版本号需要大于 7。

os

在 linux 上能正常运行的项目可能在 windows 上会出现异常，使用 os 字段可以指定项目对操作系统的兼容性要求。

```
1  "os": ["darwin", "linux"]
```

cpu

指定项目只能在特定的 CPU 体系上运行。

```
1  "cpu": ["x64", "ia32"]
```

第三方配置

一些第三方库或应用在进行某些内部处理时会依赖这些字段，使用它们时需要安装对应的第三方库。

```
1  { // 其他工具访问本项目 ts 类型定义时的入口文件 "types": "./index.d.ts", // npm 上所有
    // 的文件都开启 CDN 服务 "unpkg": "dist/vue.global.js", // 设置项目的浏览器兼容情况，
    // babel 和 autoprefixer 等工具会使用该配置对代码进行转换 "browserslist": [ ">
    // 1%", "last 2 versions"
2  ], // 用于 webpack 的 tree-shaking 优化，指定路径下的文件不参与 tree-shaking 并始
    // 终保留。 "sideEffects": ["dist/*", "es/**/style/*", "lib/**/style/*", "*.less"
3  ]
4  }
```

82. [webpack] module、chunk、bundle 的区别【热度：136】

- 手写下一个一个的文件，它们无论是 ESM 还是 commonJS 或是 AMD，他们都是 module

- 当我们写的 `module` 源文件传到 `webpack` 进行打包时，`webpack` 会根据文件引用关系生成 `chunk` 文件，`webpack` 会对这个 `chunk` 文件进行一些操作
- `webpack` 处理好 `chunk` 文件后，最后会输出 `bundle` 文件，这个 `bundle` 文件包含了经过加载和编译的最终源文件，可以直接在浏览器中运行。

一般来说一个 `chunk` 对应一个 `bundle`，比如上图中的 `utils.js -> chunks 1 -> utils.bundle.js`

但也有例外，比如说上图中，我就用 `MiniCssExtractPlugin` 从 `chunks 0` 中抽离出了 `index.bundle.css` 文件

总结：

`module`，`chunk` 和 `bundle` 其实就是同一份逻辑代码在不同转换场景下的取了三个名字：

我们直接写出来的是 `module`，`webpack` 处理时是 `chunk`，最后生成浏览器可以直接运行的 `bundle`。

83. [webpack] 分包的方式有哪些？

在 Webpack 中，可以通过以下方式进行分包

1. 通过 `entry` 属性指定入口文件，在打包时，Webpack 会按照入口文件生成一个 `chunk`，每个 `chunk` 包含了一组代码块，最终生成一个或多个 `bundle`。
2. 通过 `SplitChunksPlugin` 插件对公共依赖进行分割。该插件会把公共依赖提取出来，形成一个或多个独立的 `chunk`，以便在多个 `bundle` 中共享。
3. 使用动态导入（Dynamic Import）技术进行按需加载。在代码中使用 `import()` 或 `require.ensure()`，Webpack 会将这些代码块按照配置的策略进行分割，生成一个或多个独立的 `chunk`。
4. 使用 `DllPlugin` 插件将一些不经常变化的代码提取出来，形成一个动态链接库（DLL）。在打包时，可以直接引用这个 DLL，而不必重复打包。
5. 通过 `externals` 属性将一些模块声明为外部依赖。在打包时，Webpack 会跳过这些模块的打包过程，而是在运行时从外部环境中加载。

SplitChunksPlugin 是怎么对公共依赖进行分割的？

Webpack 的 `SplitChunksPlugin` 插件是用来对公共依赖进行分割的，其原理是将公共模块提取出来，形成一个或多个共享块，并在需要时动态加载。这个插件会分析模块之间的依赖关系，将具有相同引用模块的代码块进行提取，以便于实现缓存和更快的加载。

`SplitChunksPlugin` 的默认配置包括以下三个块：

1. 通过异步加载（异步块）生成的代码块
2. 与初始块（即入口点）共享的代码块
3. 在两个或更多块之间共享的代码块

其中，第二个块是最常见的，它包括从多个入口点引用的公共模块。我们可以通过以下示例来说明：

```
1  // webpack.config.jsmodule.exports = {
2    entry: {
3      main: './src/main.js',
4      other: './src/other.js'
5    },
6    optimization: {
7      splitChunks: {
8        cacheGroups: {
9          commons: {
10             test: /[\\/]node_modules[\\/]/,
11             name: 'vendors',
12             chunks: 'all'
13           }
14         }
15       }
16     }
17   };
```

在上面的配置中，我们定义了两个入口点 `main` 和 `other`，并将 `SplitChunksPlugin` 配置为将从 `node_modules` 目录中引用的代码提取到名为 `vendors` 的代码块中。这个代码块将被引用到我们的入口点中，以便于减少加载时间和提高性能。

当我们打包时，Webpack 将分析这两个入口点，并发现它们都引用了 `node_modules` 目录下的依赖，于是将这些依赖提取到 `vendors` 代码块中。我们可以在 `main.js` 和 `other.js` 中看到这样的代码：

```
1  import 'lodash'; // 公共模块被引用
```

通过这种方式，Webpack 能够将公共代码提取到单独的代码块中，以便于更高效地加载和缓存。

webpack DllPlugin 是如何提取依赖的？

Webpack `DllPlugin` 是用于将某些库（例如第三方库）提前编译和打包，以便在开发和生产构建过程中减少依赖项的分析时间，提高打包速度的插件。

具体来说，`DllPlugin` 分为两个步骤：第一步是生成一个包含库的映射文件（`manifest.json`），第二步是在开发或生产构建中使用该映射文件来引入预编译的库。

`DllPlugin` 的基本原理是将库单独编译成一个或多个 `dll` 文件（动态链接库），同时生成一个 `manifest.json` 文件记录这些库文件的名称、路径等信息。在开发或生产构建过程中，通过引用

manifest.json 文件，将已经预编译好的库文件引入到项目中。这样可以避免在每次打包时都去分析和打包第三方库，提高构建速度。

下面是一个简单的例子：

在 webpack.config.dll.js 中定义打包的库：

```
1  luaCopy codeconst path = require('path');
2  const webpack = require('webpack');
3
4  module.exports = {
5    mode: 'production',
6    entry: {
7      vendor: ['react', 'react-dom', 'lodash']
8    },
9    output: {
10     path: path.join(__dirname, 'dist'),
11     filename: '[name].dll.js',
12     library: '[name]'
13   },
14   plugins: [
15     new webpack.DllPlugin({
16       name: '[name]',
17       path: path.join(__dirname, 'dist', '[name].manifest.json')
18     })
19   ]
20   };
```

然后运行 `webpack --config webpack.config.dll.js`，会生成 `vendor.dll.js` 和 `vendor.manifest.json` 文件。

在开发或生产环境中，通过引入生成的 manifest.json 文件，将预编译好的库文件引入项目中：

```
1  luaCopy codeconst path = require('path');
2  const webpack = require('webpack');
3
4  module.exports = {
5    mode: 'production',
6    entry: {
7      app: './src/index.js'
8    },
9    output: {
10     path: path.join(__dirname, 'dist'),
11     filename: '[name].js'
12   },
```

```
13     plugins: [  
14         new webpack.DllReferencePlugin({  
15             manifest: require('./dist/vendor.manifest.json')  
16         })  
17     ]  
18 };
```

这样，在打包过程中，webpack 会自动将 `vendor.dll.js` 中包含的第三方库从项目中分离出来，而不需要重复打包和分析这些库。

84. [webpack] externals 作用是啥？

`webpack` 中的 `externals` 配置项用于指定在打包时需要排除掉的模块，这些模块会被视为外部依赖，即不会被打包进最终的输出文件中，而是通过其他方式引入。

使用 `externals` 配置项可以使得打包后的代码文件更小，同时也可以从外部获取依赖，例如通过 CDN、全局变量或者通过 `require` 的方式等。

举个例子，假设我们需要在项目中引入 `jquery` 库，但我们并不想在打包的过程中将其打包进最终的输出文件中，而是从外部引入。我们可以通过以下的配置来实现：

```
1  module.exports = { // ...  
2      externals: {  
3          jquery: 'jQuery'  
4      }  
5  };
```

这里的 `externals` 配置项告诉 `webpack` 在打包时忽略 `jquery` 模块的引用，而在代码运行时，我们需要手动将 `jquery` 通过 `script` 标签引入，并将其暴露在全局变量 `jQuery` 下，例如：

```
1  <script src="https://cdn.bootcdn.net/ajax/libs/jquery/3.6.0/jquery.min.js"  
    </script><script  
2      window.jQuery = jQuery;  
3  </script>
```

这样在代码中引入 `jquery` 模块时，`webpack` 就会将其作为外部依赖进行处理，而不是将其打包进输出文件中。

需要注意的是，使用 `externals` 配置项需要谨慎，因为如果在运行时无法正确获取到指定的外部依赖，就会导致代码运行出错。

85. [webpack] 异步加载原理是啥

Webpack 异步加载模块的方式主要有以下几种

1. 使用动态 import: 使用 ES6 的 `import()` 语法，动态加载模块。

```
1 import('./path/to/module')
2   .then(module => {
3     // do something with module
4   })
5   .catch(error => {
6     // handle error
7   });
```

1. 使用 `require.ensure`: 异步加载模块并将其放置到指定的 chunk 中。

```
1 require.ensure(['./path/to/module'], function(require) {const module =
  require('./path/to/module'); // do something with module
2 });
```

1. 使用 `bundle-loader`: 将模块放置到一个单独的文件中，按需加载。

```
1 const load = require('bundle-loader!./path/to/module');
2 load(function(module) { // do something with module
3 });
```

1. 使用 webpack 的 `require.ensure` API

```
1 require.ensure([], function(require) { // require dependenciesvar foo =
  require("./foo"); // ...
2 });
```

1. 使用 webpack 的 `import` 动态导入

```
1 import("./dynamic-module.js").then(module => {
2   // do something with module
3 });
```

这些方式都可以在 Webpack 中使用，具体使用哪种方式，取决于具体的场景和需求。

动态加载的原理

在 Webpack 中，异步加载组件的原理是利用动态导入（Dynamic import）特性。使用动态导入可以将模块的加载从编译时刻延迟到运行时刻。

具体来说，当 Webpack 打包代码时，遇到动态导入语句时不会将其打包进入主文件，而是将其单独打包为一个新的文件。在运行时，当代码需要加载该组件时，会通过网络请求动态加载该文件。

这样做的好处是可以减小主文件的体积，从而加快页面的加载速度，并且也可以提高代码的灵活性和可维护性。同时，Webpack 还可以对动态加载的文件进行代码分割和按需加载，进一步优化页面的性能。

在使用动态导入时，需要注意一些细节。例如，在支持动态导入的浏览器中，需要使用 `import()` 函数进行动态导入；而在不支持动态导入的浏览器中，需要使用 Webpack 提供的 `require.ensure` 或 `require.include` 等方法进行模块的异步加载。同时，还需要注意动态导入的兼容性和性能问题。

86. [webpack] 核心库 - tapable 的设计思路与实现原理是什么？

Webpack Tapable 的设计思路

Webpack Tapable 的设计思路主要基于观察者模式（Observer Pattern）和发布-订阅模式（Publish-Subscribe Pattern），用于解耦各个插件之间的依赖关系，让插件能够独立作用于特定的钩子（Hook），从而实现可扩展性和灵活性。

具体来说，Tapable 采用了钩子（Hook）的概念，每个钩子对应一组事件，Webpack 在不同的时刻触发这些钩子，插件可以注册自己的事件处理函数到对应的钩子上，以实现各种功能。

为了避免插件之间的耦合，Tapable 将事件处理函数按照钩子类型分为同步钩子（Sync Hook）、异步钩子（Async Hook）、单向异步钩子（Async Parallel Hook）和多向异步钩子（Async Series Hook）四种类型。这样，不同类型的钩子对应着不同的事件处理顺序和调用方式，插件在注册自己的事件处理函数时，可以选择不同的钩子类型来适应不同的应用场景。

除此之外，Tapable 还提供了一些辅助方法和工具函数，用于方便地创建和管理钩子、向钩子注册事件处理函数、调用钩子的事件处理函数等。这些工具函数的设计思路也遵循了解耦、简单易用的原则，为插件开发提供了很大的便利性。

Tapable 的使用

Webpack Tapable 的使用分为三个步骤：

1. 定义一个新的 Tapable 实例：在 Webpack 插件中定义一个新的 Tapable 实例，并定义需要监听的事件。

```

1  const { SyncHook } = require('tapable');
2
3  class MyPlugin {
4    constructor() {this.hooks = {
5      beforeRun: new SyncHook(['compiler']),
6      done: new SyncHook(['stats'])
7    }
8  }
9  apply(compiler) {this.hooks.beforeRun.tap('MyPlugin', compiler =>
10    {console.log('Webpack is starting to run...');
11    this.hooks.done.tap('MyPlugin', stats => {console.log('Webpack has finished
12    running.')});
13  }
14  }

```

1. 触发事件：在 Webpack 的编译过程中，调用 Tapable 实例的触发方法，触发事件。

```

1  compiler.hooks.beforeRun.call(compiler);
2  // Webpack is starting to run...
3  compiler.run((err, stats) => {if (err) {console.error(err);return;
4  }
5  console.log(stats);
6  compiler.hooks.done.call(stats);// Webpack has finished running.
7  });

```

1. 注册插件：在 Webpack 的配置文件中，将插件实例注册到 Webpack 中。

```

1  const MyPlugin = require('./my-plugin');
2
3  module.exports = {
4    plugins: [new MyPlugin()]
5  };

```

以上是使用 Tapable 的基本流程，通过 Tapable 可以监听到编译过程中的各个事件，并对编译过程进行修改，从而实现各种插件。以下是一些常见的 Tapable 类型和用法：

- SyncHook：同步 Hook，按照注册的顺序同步执行所有回调函数。

```

1  const { SyncHook } = require('tapable');

```

```

2
3  const hook = new SyncHook(['arg1', 'arg2']);
4  hook.tap('MyPlugin', (arg1, arg2) => {console.log(`Hook is triggered with
arguments:
5  ${arg1}, $
6  {arg2}`)});
7  });
8  hook.tap('MyPlugin', (arg1, arg2) => {console.log('Second callback is
called')});
9  });
10 hook.call('Hello', 'world');
11 // Hook is triggered with arguments: Hello, world// Second callback is called

```

- AsyncParallelHook: 异步 Hook，按照注册的顺序异步执行所有回调函数，不关心回调函数的返回值。

```

1  const { AsyncParallelHook } = require('tapable');
2
3  const hook = new AsyncParallelHook(['arg1', 'arg2']);
4  hook.tap('MyPlugin', (arg1, arg2, callback) => {
5    setTimeout(() => {console.log(`Hook is triggered with arguments:
6    ${arg1}, $
7    {arg2}`)});
8    callback();
9  }, 1000);
10 });
11 hook.tap('MyPlugin', (arg1, arg2, callback) => {
12   setTimeout(() => {console.log('Second callback is called')});
13   callback();
14 }, 500)
15 })

```

Tapable 是如何实现的？代码简单实现一下？

Webpack Tapable 是基于发布-订阅模式的一个插件系统，它提供了一组钩子函数，让插件可以在相应的时机执行自己的逻辑。

下面是一个简单的自定义 Tapable 的实现：

```

1  class Tapable {
2    constructor() {this.hooks = {}};
3  }
4  // 注册事件监听函数

```

```

5     tap(name, callback) {if (!this.hooks[name]) {this.hooks[name] = [];}
6         }this.hooks[name].push(callback);
7     }
8     // 触发事件
9     call(name, ...args) {const callbacks = this.hooks[name];if (callbacks &&
    callbacks.length) {
10         callbacks.forEach((callback) => callback(...args));
11     }
12 }
13 }

```

在这个例子中，我们定义了一个 `Tapable` 类，它有一个 `hooks` 对象属性，用于存储各个事件对应的监听函数。然后我们定义了 `tap` 方法，用于注册事件监听函数，以及 `call` 方法，用于触发事件。

下面是一个使用自定义 `Tapable` 的例子：

```

1     const tapable = new Tapable();
2     tapable.tap('event1', (arg1, arg2) => {console.log('event1 is triggered with
    arguments:', arg1, arg2);
3 });
4     tapable.tap('event2', (arg1, arg2) => {console.log('event2 is triggered with
    arguments:', arg1, arg2);
5 });
6     tapable.call('event1', 'hello', 'world');
7     tapable.call('event2', 'foo', 'bar');

```

在这个例子中，我们定义了两个事件 `event1` 和 `event2`，并为它们注册了监听函数。当我们调用 `call` 方法触发事件时，注册的监听函数就会依次执行。

这个自定义 `Tapable` 的实现虽然简单，但它体现了 `Tapable` 的设计思路和核心功能。在实际使用中，`Webpack` 的 `Tapable` 提供了更多的功能和钩子，可以满足不同场景的需求。

87. [webpack] 构建流程是怎么样的？

`webpack` 的运行流程是一个串行的过程，它的工作流程就是将各个插件串联起来

在运行过程中会广播事件，插件只需要监听它所关心的事件，就能加入到这条 `webpack` 机制中，去改变 `webpack` 的运作，使得整个系统扩展性良好

从启动到结束会依次执行以下三大步骤：

- 初始化流程：从配置文件和 `Shell` 语句中读取与合并参数，并初始化需要使用的插件和配置插件等执行环境所需要的参数

- 编译构建流程：从 Entry 发出，针对每个 Module 串行调用对应的 Loader 去翻译文件内容，再找到该 Module 依赖的 Module，递归地进行编译处理
- 输出流程：对编译后的 Module 组合成 Chunk，把 Chunk 转换成文件，输出到文件系统

初始化流程

从配置文件和 `Shell` 语句中读取与合并参数，得出最终的参数

配置文件默认下为 `webpack.config.js`，也或者通过命令的形式指定配置文件，主要作用是用于激活 `webpack` 的加载项和插件

`webpack` 将 `webpack.config.js` 中的各个配置项拷贝到 `options` 对象中，并加载用户配置的 `plugins`

完成上述步骤之后，则开始初始化 `Compiler` 编译对象，该对象掌控者 `webpack` 声明周期，不执行具体的任务，只是进行一些调度工作

```
1  class Compiler extends Tapable {
2      constructor(context) {
3          super(); this.hooks = {
4              beforeCompile: new AsyncSeriesHook(["params"]),
5              compile: new SyncHook(["params"]),
6              afterCompile: new AsyncSeriesHook(["compilation"]),
7              make: new AsyncParallelHook(["compilation"]),
8              entryOption: new SyncBailHook(["context", "entry"])// 定义了很多不同
                类型的钩子
9          };// ...
10     }
11 }
12
13 function webpack(options) {var compiler = new Compiler();
14     ...// 检查options,若watch字段为true,则开启watch线程return compiler;
15 }
16 ...
```

`Compiler` 对象继承自 `Tapable`，初始化时定义了很多钩子函数

编译构建流程

根据配置中的 `entry` 找出所有的入口文件

```
1  module.exports = {
2      entry: './src/file.js'
3  }
```


初始化完成后会调用 `Compiler` 的 `run` 来真正启动 `webpack` 编译构建流程，主要流程如下：

- `compile` 开始编译
- `make` 从入口点分析模块及其依赖的模块，创建这些模块对象
- `build-module` 构建模块
- `seal` 封装构建结果
- `emit` 把各个chunk输出到结果文件

compile 编译

执行了 `run` 方法后，首先会触发 `compile`，主要是构建一个 `Compilation` 对象

该对象是编译阶段的主要执行者，主要会依次下述流程：执行模块创建、依赖收集、分块、打包等主要任务的对象

make 编译模块

当完成了上述的 `compilation` 对象后，就开始从 `Entry` 入口文件开始读取，主要执行 `_addModuleChain()` 函数，如下：

```
1  _addModuleChain(context, dependency, onModule, callback) {
2      ...// 根据依赖查找对应的工厂函数const Dep = /** @type {DepConstructor} */
      (dependency.constructor);const moduleFactory =
      this.dependencyFactories.get(Dep);
3  // 调用工厂函数NormalModuleFactory的create来生成一个空的NormalModule对象
4      moduleFactory.create({
5          dependencies: [dependency]
6          ...
7      }, (err, module) => {
8          ...const afterBuild = () => {this.processModuleDependencies(module, err
=> {if (err) return callback(err);
9              callback(null, module);
10             });
11         };
12         this.buildModule(module, false, null, null, err => {
13             ...
14             afterBuild();
15         })
16     })
17 }
```

过程如下：

`_addModuleChain` 中接收参数 `dependency` 传入的入口依赖，使用对应的工厂函数 `NormalModuleFactory.create` 方法生成一个空的 `module` 对象

回调中会把此 `module` 存入 `compilation.modules` 对象和 `dependencies.module` 对象中，由于是入口文件，也会存入 `compilation.entries` 中

随后执行 `buildModule` 进入真正的构建模块 `module` 内容的过程

build module 完成模块编译

这里主要调用配置的 `loaders`，将我们的模块转成标准的 `JS` 模块

在用 `Loader` 对一个模块转换完后，使用 `acorn` 解析转换后的内容，输出对应的抽象语法树（AST），以方便 `Webpack` 后面对代码的分析

从配置的入口模块开始，分析其 `AST`，当遇到 `require` 等导入其它模块语句时，便将其加入到依赖的模块列表，同时对新找出的依赖模块递归分析，最终搞清所有模块的依赖关系

输出流程

seal 输出资源

`seal` 方法主要是要生成 `chunks`，对 `chunks` 进行一系列的优化操作，并生成要输出的代码

`webpack` 中的 `chunk`，可以理解为配置在 `entry` 中的模块，或者是动态引入的模块

根据入口和模块之间的依赖关系，组装成一个个包含多个模块的 `Chunk`，再把每个 `Chunk` 转换成一个单独的文件加入到输出列表

emit 输出完成

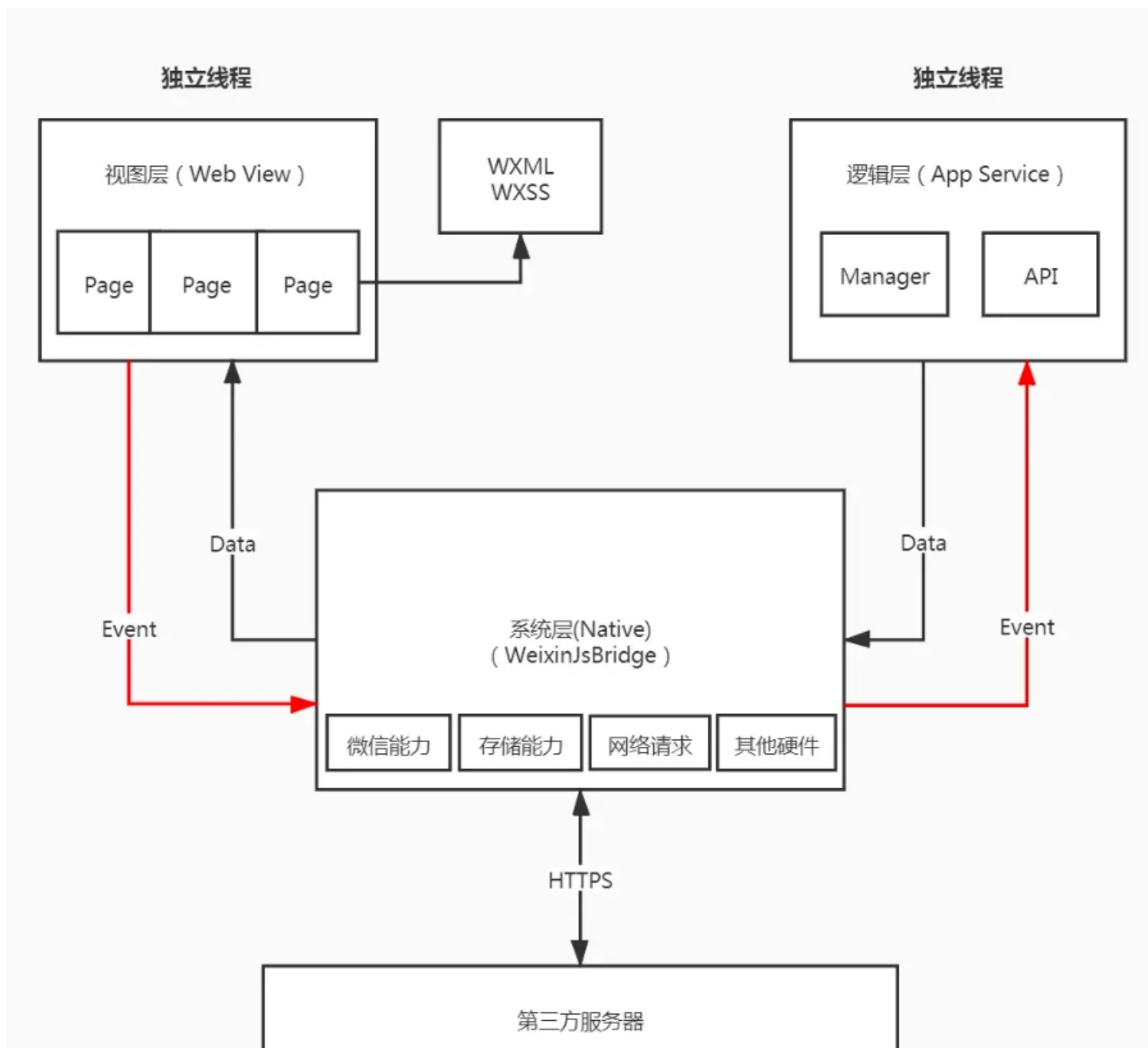
在确定好输出内容后，根据配置确定输出的路径和文件名

```
1  output: {  
2    path: path.resolve(__dirname, 'build'),  
3    filename: '[name].js'  
4  }
```

在 `Compiler` 开始生成文件前，钩子 `emit` 会被执行，这是我们修改最终文件的最后一个机会
从而 `webpack` 整个打包过程则结束了

88. 小程序的大概原理？

具体流程可以看下面这个图：



要了解小程序架构原理，要从以下几个方面入手探索

- 宿主环境
- 执行环境
- 小程序整体架构
- 运行机制
- 更新机制
- 数据通信机制
- 登录机制
- 性能方向问题
- JSCore

89. [Webpack] webpack热更新原理是什么？

Hot Module Replacement，简称 HMR，无需完全刷新整个页面的同时，更新模块。HMR 的好处，在日常开发工作中体会颇深：节省宝贵的开发时间、提升开发体验。

刷新我们一般分为两种：

- 一种是页面刷新，不保留页面状态，就是简单粗暴，直接 `window.location.reload()`。
- 另一种是基于 WDS (Webpack-dev-server) 的模块热替换，只需要局部刷新页面上发生变化的模块，同时可以保留当前的页面状态，比如复选框的选中状态、输入框的输入等。

HMR 作为一个 Webpack 内置的功能，可以通过 `HotModuleReplacementPlugin` 或 `--hot` 开启。那么，HMR 到底是怎么实现热更新的呢？下面让我们了解一下吧！

1. webpack-dev-server启动本地服务

我们根据 `webpack-dev-server` 的 `package.json` 中的 `bin` 命令，可以找到命令的入口文件 `bin/webpack-dev-server.js`。

```
1  // node_modules/webpack-dev-server/bin/webpack-dev-server.js// 生成webpack编译
   主引擎 compiler
2  let compiler = webpack(config);
3
4  // 启动本地服务
5  let server = new Server(compiler, options, log);
6  server.listen(options.port, options.host, (err) => {if (err) {throw err};
7  });
```

本地服务代码：

```
1  // node_modules/webpack-dev-server/lib/Server.jsclass Server {
2    constructor() {this.setupApp();this.createServer();
3    }
4    setupApp() {// 依赖了expressthis.app = new express();
5    }
6    createServer() {this.listeningApp = http.createServer(this.app);
7    }
8    listen(port, hostname, fn) {return this.listeningApp.listen(port,
   hostname, (err) => {// 启动express服务后，启动websocket服务
   this.createSocketServer();
9    }
10   }
11 }
```

这一小节代码主要做了三件事：

- 启动 `webpack`，生成 `compiler` 实例。`compiler` 上有很多方法，比如可以启动 `webpack` 所有编译工作，以及监听本地文件的变化。
- 使用 `express` 框架启动本地 `server`，让浏览器可以请求本地的静态资源。
- 本地 `server` 启动之后，再去启动 `websocket` 服务，如果不了解 `websocket`，建议简单了解一下[websocket速成](#)。通过 `websocket`，可以建立本地服务和浏览器的双向通信。这样就可以实现当本地文件发生变化，立马告知浏览器可以热更新代码啦！

上述代码主要干了三件事，但是源码在启动服务前又做了很多事，接下来便看看 `webpack-dev-server/lib/Server.js` 还做了哪些事？

2. 修改`webpack.config.js`的`entry`配置

启动本地服务前，调用了 `updateCompiler(this.compiler)` 方法。这个方法中有 2 段关键性代码。一个是获取 `websocket` 客户端代码路径，另一个是根据配置获取 `webpack` 热更新代码路径。

```
1  // 获取websocket客户端代码const clientEntry = `${require.resolve('../..client/'
2  )}?
3  ${domain}$
4  {sockHost}
5  ${sockPath}$
6  {sockPort}`;
7
8  // 根据配置获取热更新代码let hotEntry;
9  if (options.hotOnly) {
10     hotEntry = require.resolve('webpack/hot/only-dev-server');
11 } else if (options.hot) {
12     hotEntry = require.resolve('webpack/hot/dev-server');
13 }
```

修改后的 `webpack` 入口配置如下：

```
1  // 修改后的entry入口
2  { entry:
3    { index:
4      [
5        // 上面获取的clientEntry
6        'xxx/node_modules/webpack-dev-server/client/index.js?
http://localhost:8080',
7        // 上面获取的hotEntry
8        'xxx/node_modules/webpack/hot/dev-server.js',
9        // 开发配置的入口
```

```
10         './src/index.js'
11     ],
12     },
13 }
```

为什么要新增了 2 个文件？在入口默默增加了 2 个文件，那就意味会一同打包到 `bundle` 文件中去，也就是线上运行时。

(1) webpack-dev-server/client/index.js

首先这个文件用于 `websocket` 的，因为 `websoket` 是双向通信，如果不了解 `websocket`，建议简单了解一下[websocket速成](#)。我们在第 1 步 `webpack-dev-server` 初始化的过程中，启动的是本地服务端的 `websocket`。那客户端也就是我们的浏览器，浏览器还没有和服务端通信的代码呢？总不能让开发者去写吧hhhhhh。因此我们需要把 `websocket` 客户端通信代码偷偷塞到我们的代码中。客户端具体的代码后面会在合适的时机细讲哦。

(2) webpack/hot/dev-server.js

这个文件主要是用于检查更新逻辑的，这里大家知道就好，代码后面会在合适的时机（第5步）细讲。

3. 监听webpack编译结束

修改好入口配置后，又调用了 `setupHooks` 方法。这个方法是用来注册监听事件的，监听每次 `webpack` 编译完成。

```
1  // node_modules/webpack-dev-server/lib/Server.js// 绑定监听事件
2  setupHooks() {const {done} = compiler.hooks;// 监听webpack的done钩子, tapable提
   供的监听方法
3      done.tap('webpack-dev-server', (stats) => {this._sendStats(this.sockets,
   this.getStats(stats));this._stats = stats;
4      });
5  };
```

当监听到一次 `webpack` 编译结束，就会调用 `_sendStats` 方法通过 `websocket` 给浏览器发送通知，`ok` 和 `hash` 事件，这样浏览器就可以拿到最新的 `hash` 值了，做检查更新逻辑。

```
1  // 通过websoket给客户端发消息
2  _sendStats() {this.sockWrite(sockets, 'hash',
   stats.hash);this.sockWrite(sockets, 'ok');
3  }
```

4. webpack监听文件变化

每次修改代码，就会触发编译。说明我们还需要监听本地代码的变化，主要是通过 `setupDevMiddleware` 方法实现的。

这个方法主要执行了 `webpack-dev-middleware` 库。很多人分不清 `webpack-dev-middleware` 和 `webpack-dev-server` 的区别。其实就是因为 `webpack-dev-server` 只负责启动服务和前置准备工作，所有文件相关的操作都抽离到 `webpack-dev-middleware` 库了，主要是本地文件的编译和输出以及监听，无非就是职责的划分更清晰了。

那我们来看下 `webpack-dev-middleware` 源码里做了什么事：

```
1  // node_modules/webpack-dev-
  middleware/index.js compiler.watch(options.watchOptions, (err) => {if (err) {
  /*错误处理*/ }
2  });
3
4  // 通过“memory-fs”库将打包后的文件写入内存 setFs(context, compiler);
```

(1) 调用了 `compiler.watch` 方法，在第 1 步中也提到过，`compiler` 的强大。这个方法主要就做了 2 件事：

- 首先对本地文件代码进行编译打包，也就是 `webpack` 的一系列编译流程。
- 其次编译结束后，开启对本地文件的监听，当文件发生变化，重新编译，编译完成之后继续监听。

为什么代码的改动保存会自动编译，重新打包？这一系列的重新检测编译就归功于 `compiler.watch` 这个方法了。监听本地文件的变化主要是通过文件的生成时间是否有变化，这里就不细讲了。

(2) 执行 `setFs` 方法，这个方法主要目的就是将编译后的文件打包到内存。这就是为什么在开发的过程中，你会发现 `dist` 目录没有打包后的代码，因为都在内存中。原因就在于访问内存中的代码比访问文件系统中的文件更快，而且也减少了代码写入文件的开销，这一切都归功于 `memory-fs`。

5. 浏览器接收到热更新的通知

我们已经可以监听到文件的变化了，当文件发生变化，就触发重新编译。同时还监听了每次编译结束的事件。当监听到一次 `webpack` 编译结束，`_sendStats` 方法就通过 `websocket` 给浏览器发送通知，检查下是否需要热更新。下面重点讲的就是 `_sendStats` 方法中的 `ok` 和 `hash` 事件都做了什么。

那浏览器是如何接收到 `websocket` 的消息呢？回忆下第 2 步骤增加的入口文件，也就是 `websocket` 客户端代码。

```
1  'xxx/node_modules/webpack-dev-server/client/index.js?http://localhost:8080'
```

这个文件的代码会被打包到 `bundle.js` 中，运行在浏览器中。来看下这个文件的核心代码吧。

```
1  // webpack-dev-server/client/index.jsvar socket = require('./socket');
2  var onSocketMessage = {
3      hash: function hash(_hash) {// 更新currentHash值
4          status.currentHash = _hash;
5      },
6      ok: function ok() {
7          sendMessage('Ok');// 进行更新检查等操作
8          reloadApp(options, status);
9      },
10 };
11 // 连接服务地址socketUrl, ?http://localhost:8080, 本地服务地址
12 socket(socketUrl, onSocketMessage);
13
14 function reloadApp() {if (hot) {
15     log.info('[WDS] App hot update...');
16     // hotEmitter其实就是EventEmitter的实例var hotEmitter =
17     require('webpack/hot/emitter');
18     hotEmitter.emit('webpackHotUpdate', currentHash);
19 }
```

`socket` 方法建立了 `websocket` 和服务端的连接，并注册了 2 个监听事件。

- `hash` 事件，更新最新一次打包后的 `hash` 值。
- `ok` 事件，进行热更新检查。

热更新检查事件是调用 `reloadApp` 方法。比较奇怪的是，这个方法又利用 `node.js` 的 `EventEmitter`，发出 `webpackHotUpdate` 消息。这是为什么？为什么不直接进行检查更新呢？

个人理解就是为了更好的维护代码，以及职责划分的更明确。`websocket` 仅仅用于客户端（浏览器）和服务端进行通信。而真正做事的活还是交回给了 `webpack`。

那 `webpack` 怎么做的呢？再来回忆下第 2 步。入口文件还有一个文件没有讲到，就是：

```
1  'xxx/node_modules/webpack/hot/dev-server.js'
```

这个文件的代码同样会被打包到 `bundle.js` 中，运行在浏览器中。这个文件做了什么就显而易见了吧！先瞄一眼代码：


```

1  // node_modules/webpack/hot/dev-server.js var check = function check()
   {module.hot.check(true)
2      .then(function(updatedModules) { // 容错, 直接刷新页面 if
      (!updatedModules) {window.location.reload();return;
3          }
4  // 热更新结束, 打印信息 if (upToDate()) {
5      log("info", "[HMR] App is up to date.");
6          }
7  })
8      .catch(function(err) {window.location.reload();
9          });
10 };
11
12 var hotEmitter = require("./emitter");
13 hotEmitter.on("webpackHotUpdate", function(currentHash) {
14     lastHash = currentHash;
15     check();
16 });

```

这里 `webpack` 监听到了 `webpackHotUpdate` 事件, 并获取最新了最新的 `hash` 值, 然后终于进行检查更新了。检查更新呢调用的是 `module.hot.check` 方法。那么问题又来了, `module.hot.check` 又是哪里冒出来的了! 答案是 `HotModuleReplacementPlugin` 搞得鬼。这里留个疑问, 继续往下看。

6. HotModuleReplacementPlugin

前面好像一直是 `webpack-dev-server` 做的事, 那 `HotModuleReplacementPlugin` 在热更新过程中又做了什么伟大的事业呢?

首先你可以对比下, 配置热更新和不配置时 `bundle.js` 的区别。内存中看不到? 直接执行 `webpack` 命令就可以看到生成的 `bundle.js` 文件啦。不要用 `webpack-dev-server` 启动就好了。

(1) 没有配置的。

```

/****/ function __webpack_require__(moduleId) {
/****/
/****/ // Check if module is in cache
/****/ if(installedModules[moduleId]) {
/****/     return installedModules[moduleId].exports;
/****/ }
/****/ // Create a new module (and put it into the cache)
/****/ var module = installedModules[moduleId] = {
/****/     i: moduleId,
/****/     l: false,
/****/     exports: {}
/****/ };
/****/

```

(2) 配置了 `HotModuleReplacementPlugin` 或 `--hot` 的。

```
function __webpack_require__(moduleId) {
    // Check if module is in cache
    if(installedModules[moduleId]) {
        return installedModules[moduleId].exports;
    }
    // Create a new module (and put it into the cache)
    var module = installedModules[moduleId] = {
        i: moduleId,
        l: false,
        exports: {},
        hot: hotCreateModule(moduleId),
        parents: (hotCurrentParentsTemp = hotCurrentParents, hotCurrentParents = [], hotCu
        children: []
    }
}
```

哦~ 我们发现 `moudle` 新增了一个属性为 `hot`，再看 `hotCreateModule` 方法。这不就找到 `module.hot.check` 是哪里冒出来的。

```
function hotCreateModule(moduleId) {
    var hot = {
        // private stuff
        _acceptedDependencies: {},
        _declinedDependencies: {},
        _selfAccepted: false,
        _selfDeclined: false,
        _disposeHandlers: [],
        _main: hotCurrentChildModule !== moduleId,

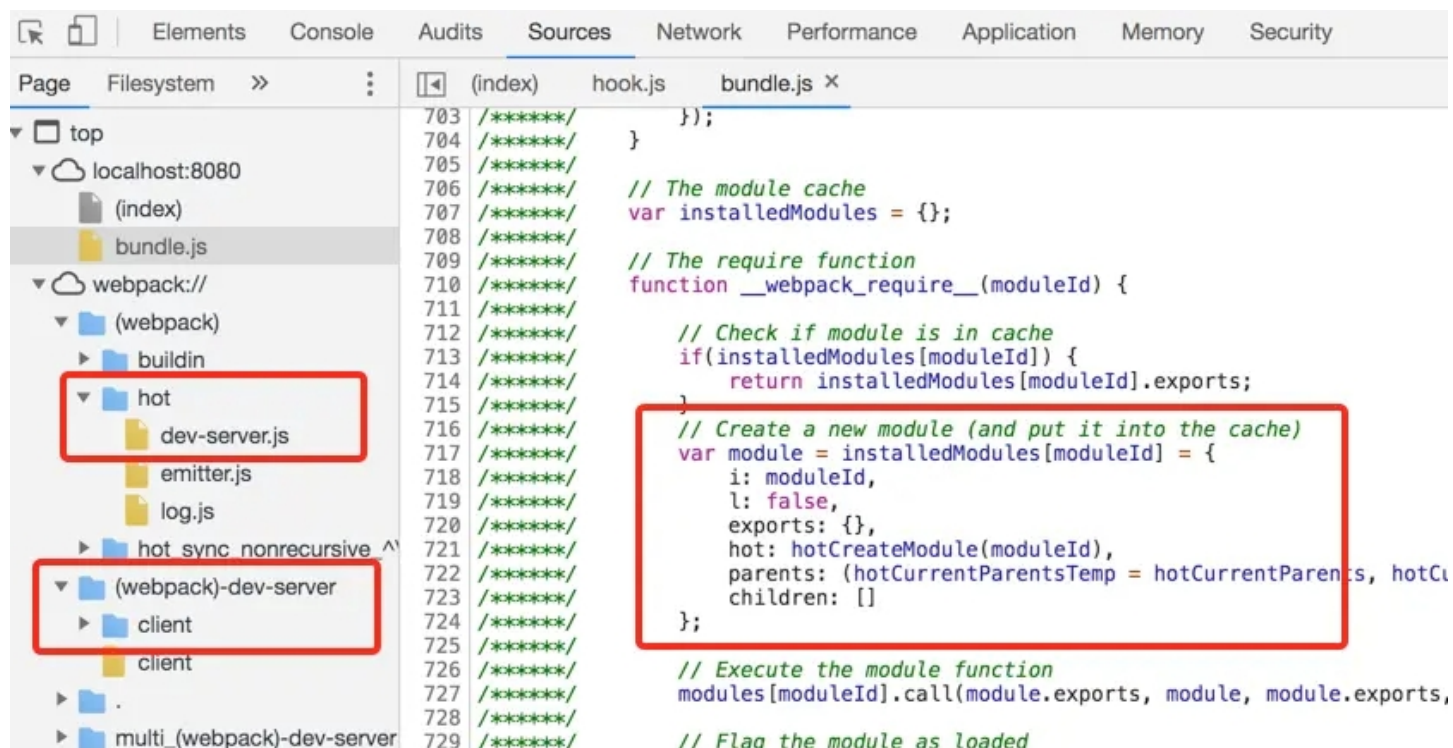
        // Module API
        active: true,
        accept: function(dep, callback) {...},
        decline: function(dep) {...},
        dispose: function(callback) {
            hot._disposeHandlers.push(callback);
        },
        addDisposeHandler: function(callback) {...},
        removeDisposeHandler: function(callback) {...},

        // Management API
        check: hotCheck,
        apply: hotApply,
        status: function(l) {...},
        addStatusHandler: function(l) {

```

经过对比打包后的文件，`webpack_require` 中的 `moudle` 以及代码行数的不同。我们都可以发现 `HotModuleReplacementPlugin` 原来也是默默的塞了很多代码到 `bundle.js` 中呀。这和第

2 步骤很是相似哦！为什么，因为检查更新是在浏览器中操作呀。这些代码必须在运行时的环境。你也可以直接看浏览器 Sources 下的代码，会发现 webpack 和 plugin 偷偷加的代码都在哦。在这里调试也很方便。



HotModuleReplacementPlugin 如何做到的？这里我就不讲了，因为这需要你对 tapable 以及 plugin 机制有一定了解

7. module.hot.check 开始热更新

通过第 6 步，我们就可以知道 module.hot.check 方法是如何来的啦。那都做了什么？之后的源码都是 HotModuleReplacementPlugin 塞入到 bundle.js 中的哦，我就不写文件路径了。

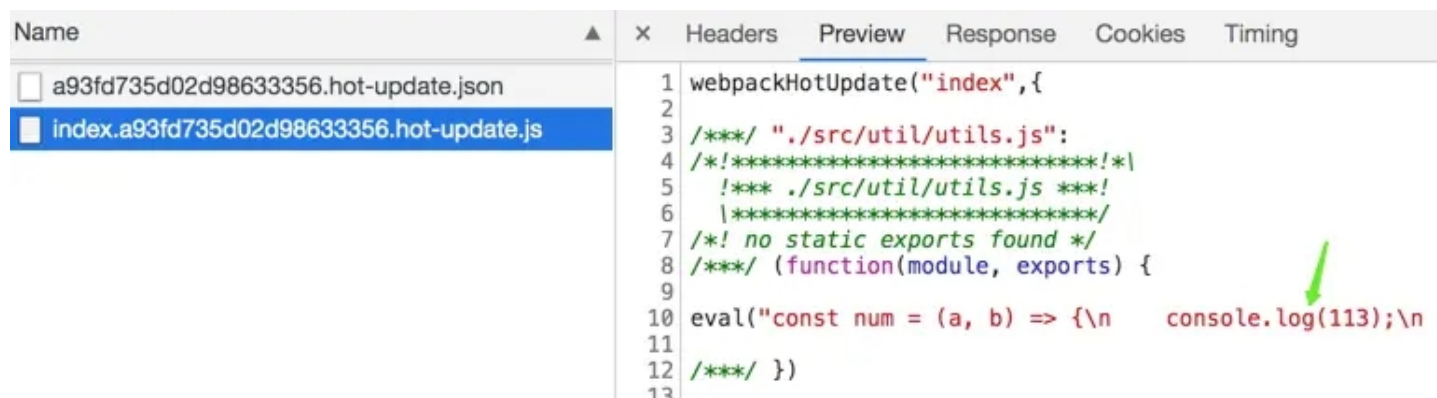
- 利用上一次保存的 hash 值，调用 hotDownloadManifest 发送 xxx/hash.hot-update.json 的 ajax 请求；
- 请求结果获取热更新模块，以及下次热更新的 Hash 标识，并进入热更新准备阶段。

```
1 hotAvailableFilesMap = update.c; // 需要更新的文件  
2 hotUpdateNewHash = update.h; // 更新下次热更新hash值  
3 hotSetStatus("prepare"); // 进入热更新准备状态
```

- 调用 hotDownloadUpdateChunk 发送 xxx/hash.hot-update.js 请求，通过 JSONP 方式。

- **function hotDownloadUpdateChunk**(chunkId) {**var** script = **document**.createElement("script");
script.charset = "utf-8";
script.src = __webpack_require__.p + ""chunkId + ".hot-update.js";**if** (null) script.crossOrigin = null;**document**.head.appendChild(script);
}

这个函数体为什么要单独拿出来，因为这里要解释下为什么使用 JSONP 获取最新代码？主要是因为 JSONP 获取的代码可以直接执行。为什么要直接执行？我们来回忆下 /hash.hot-update.js 的代码格式是怎么样的。



可以发现，新编译后的代码是在一个 **webpackHotUpdate** 函数体内部的。也就是要立即执行 **webpackHotUpdate** 这个方法。

再看下 **webpackHotUpdate** 这个方法。

```
1 window["webpackHotUpdate"] = function (chunkId, moreModules) {
2     hotAddUpdateChunk(chunkId, moreModules);
3 } ;
```

- **hotAddUpdateChunk** 方法会把更新的模块 **moreModules** 赋值给全局变量 **hotUpdate**。
- **hotUpdateDownloaded** 方法会调用 **hotApply** 进行代码的替换。

```
1 function hotAddUpdateChunk(chunkId, moreModules) { // 更新的模块moreModules赋值给
    全局变量hotUpdatefor (var moduleId in moreModules) {if
    (Object.prototype.hasOwnProperty.call(moreModules, moduleId)) {
2         hotUpdate[moduleId] = moreModules[moduleId];
3     }
4 } // 调用hotApply进行模块的替换
5 hotUpdateDownloaded();
6 }
```

8. hotApply 热更新模块替换

热更新的核心逻辑就在 `hotApply` 方法了。`hotApply` 代码有将近 400 行，还是挑重点讲了，看哭😭

①删除过期的模块，就是需要替换的模块

通过 `hotUpdate` 可以找到旧模块

```
1  var queue = outdatedModules.slice();
2  while (queue.length > 0) {
3      moduleId = queue.pop(); // 从缓存中删除过期的模块
4      module = installedModules[moduleId]; // 删除过期的依赖 delete
      outdatedDependencies[moduleId];
5      // 存储了被删掉的模块id, 便于更新代码
6      outdatedSelfAcceptedModules.push({
7          module: moduleId
8      });
9  }
```

②将新的模块添加到 modules 中

```
1  appliedUpdate[moduleId] = hotUpdate[moduleId];
2  for (moduleId in appliedUpdate) { if
    (Object.prototype.hasOwnProperty.call(appliedUpdate, moduleId)) {
3      modules[moduleId] = appliedUpdate[moduleId];
4  }
5  }
```

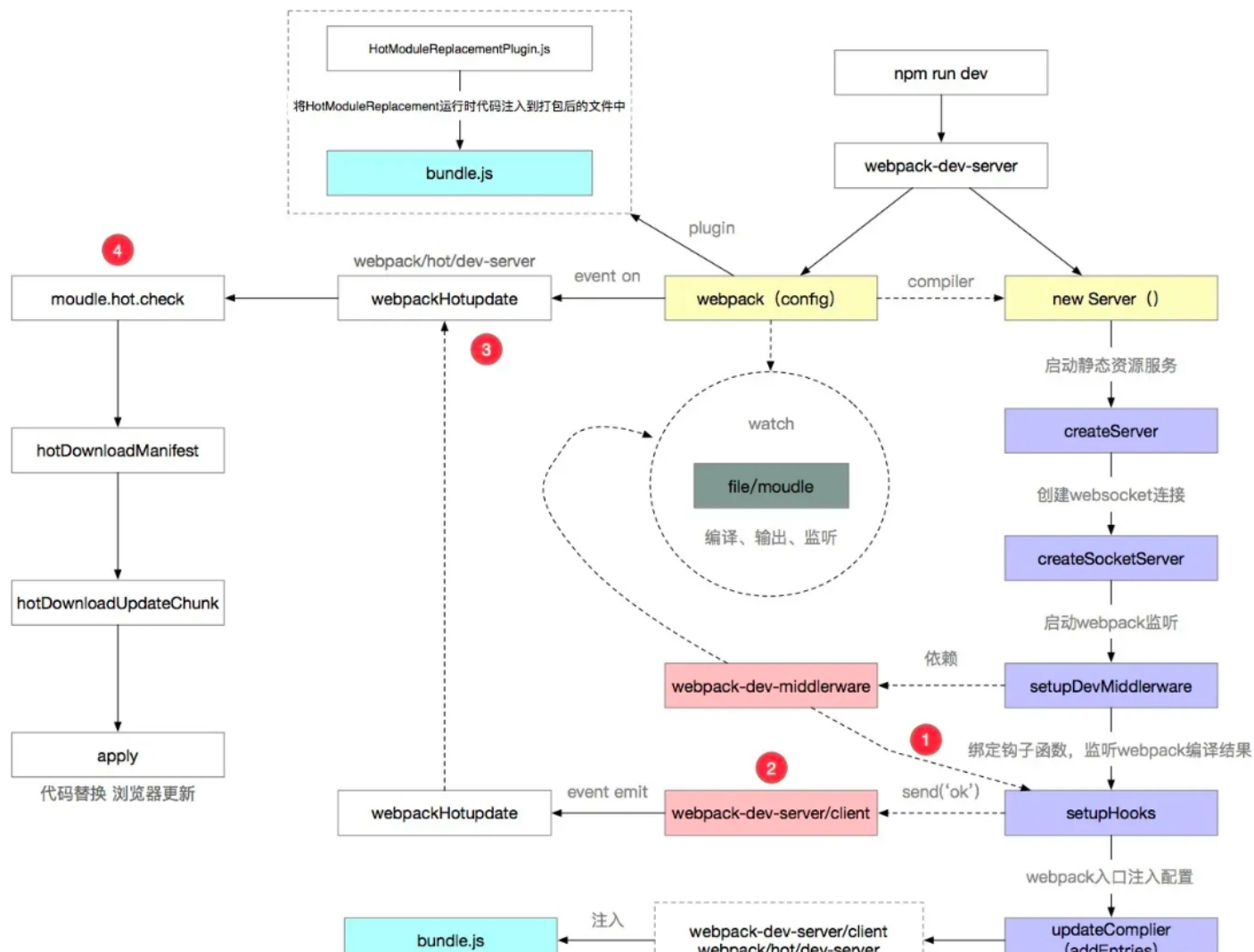
③通过__webpack_require__执行相关模块的代码

```
1  for (i = 0; i < outdatedSelfAcceptedModules.length; i++) {
2      var item = outdatedSelfAcceptedModules[i];
3      moduleId = item.module; try {
4          // 执行最新的代码
5
6      __webpack_require__
7      (moduleId);
8      } catch (err) {
9          // ...容错处理
10     }
11 }
```


hotApply 的确比较复杂，知道大概流程就好了

总结

还是以阅读源码的形式画的图，①-④的小标记，是文件发生变化的一个流程。



90. 如果用户说 web 应用感觉很慢，该如何排查？

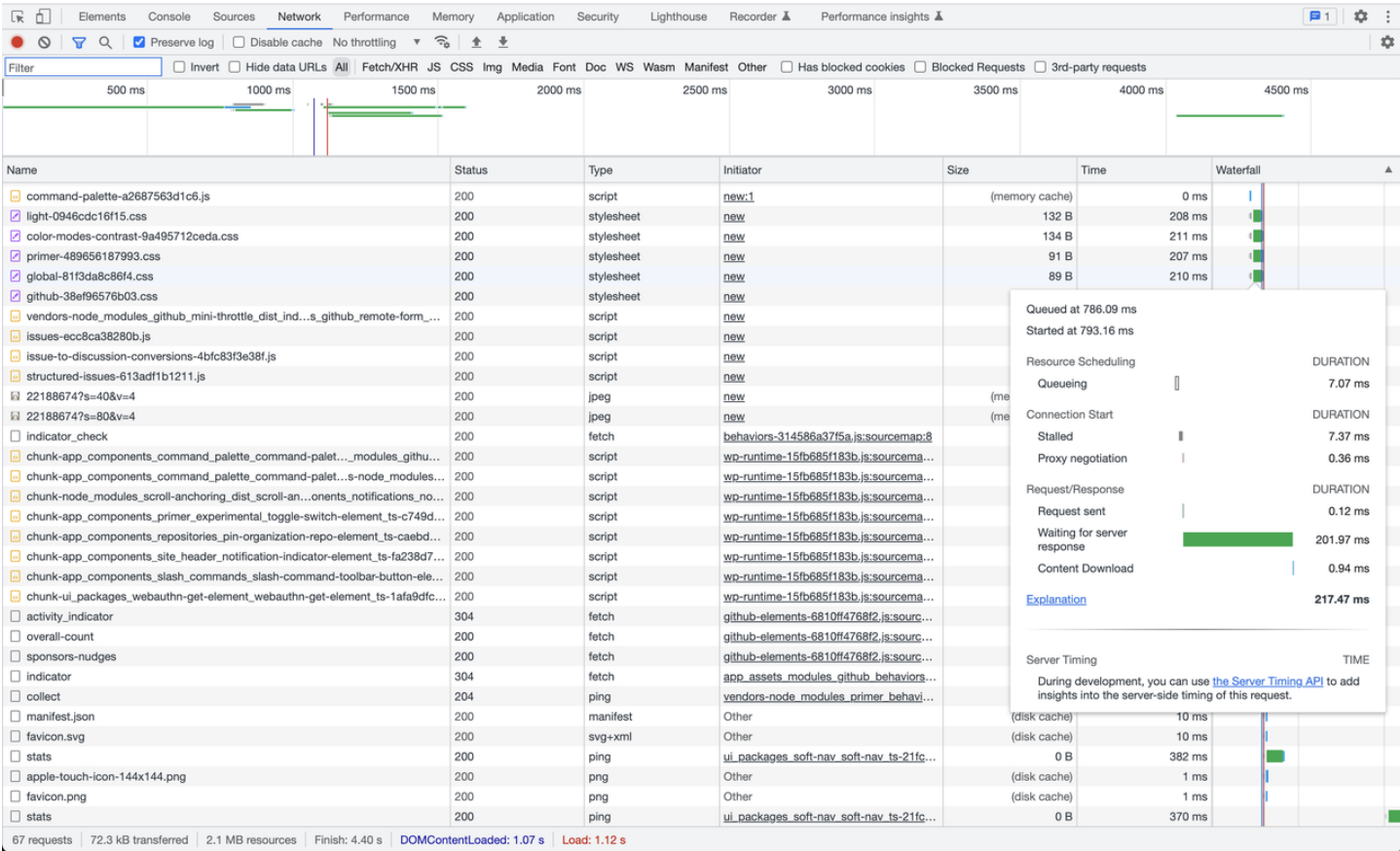
如果用户觉得 web 应用反应卡顿， 主要从哪几个方面来排查？

- 加载慢
 - 资源下载慢
 - 首屏并发请求资源过多
 - 首屏接口慢
 - 首屏对应的 JS 执行慢
 - 首屏渲染慢
 - 首屏加载静态资源过大
 -

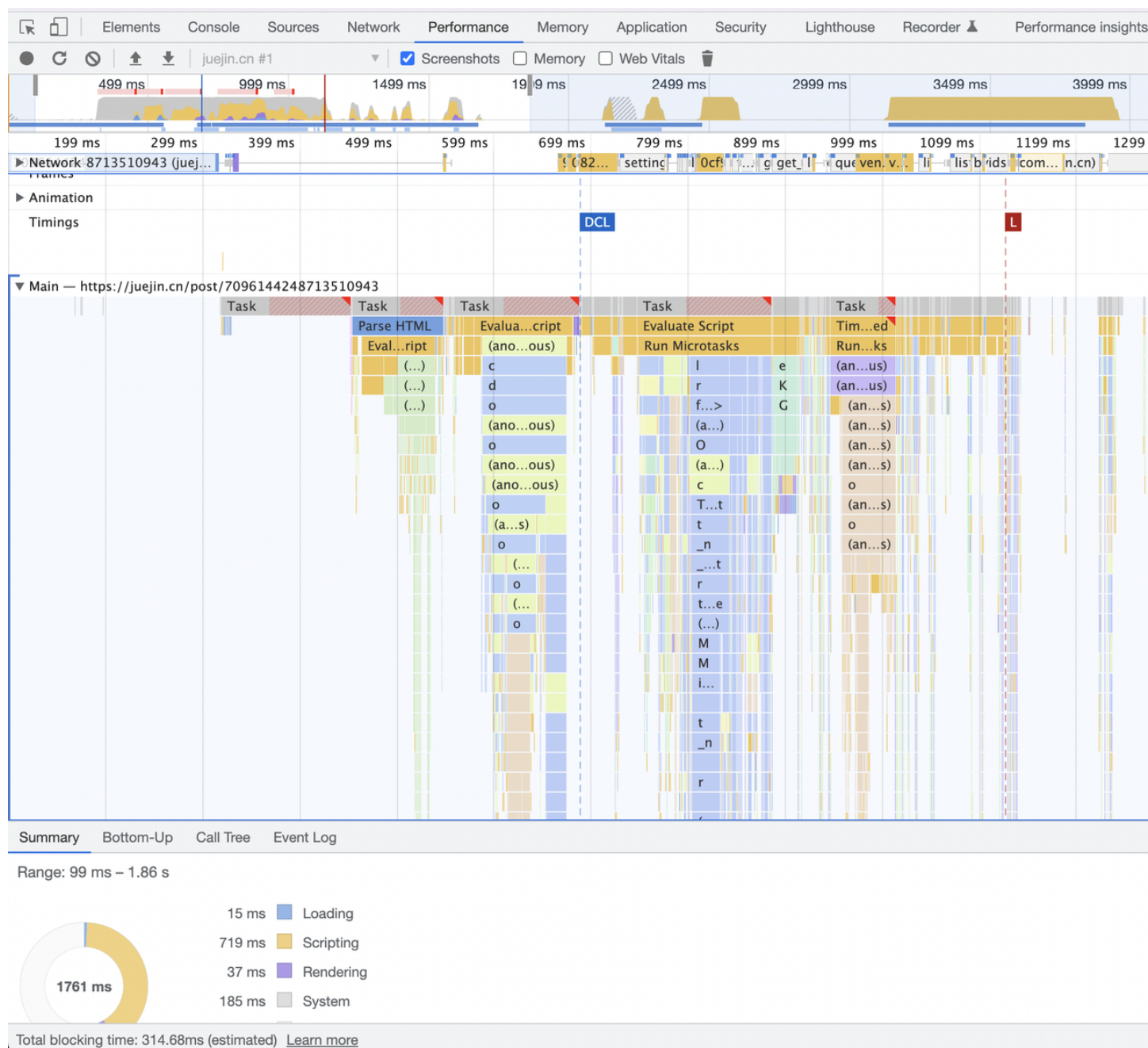
- 执行过程慢
 - 接口慢
 - long tasks 太多, 阻塞 JS 执行
 - 内存泄漏
 - 重绘重排 过多
 - 关键节点没有加 节流防抖
 -

主要排查手段有哪些

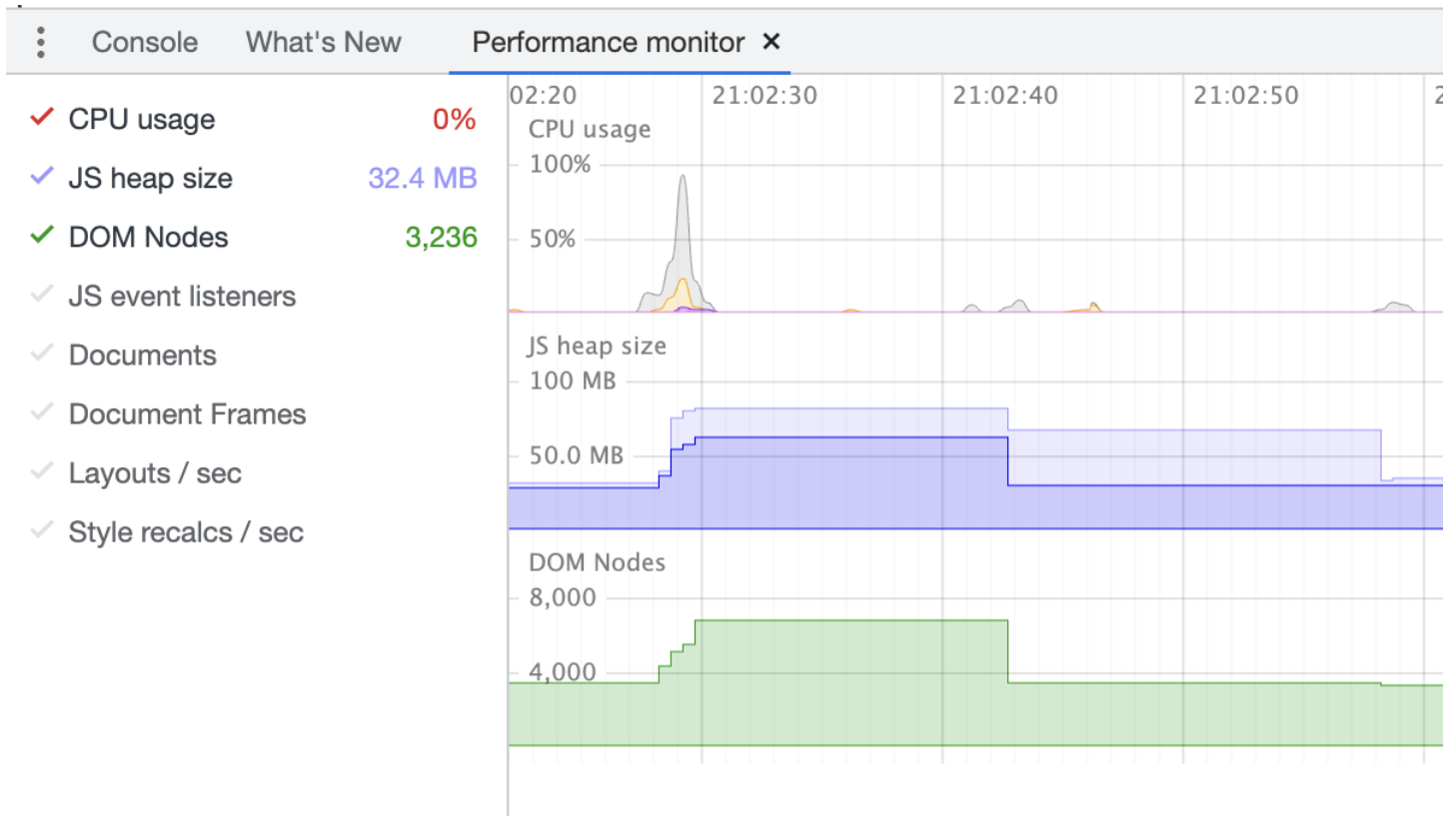
- 通过建立性能监控指标: 通过真实用户数据反馈， 来判断用户是否卡顿， 包含网络监控、运行时性能监控
- Chrome devtools: NetWork 主要排查网络问题



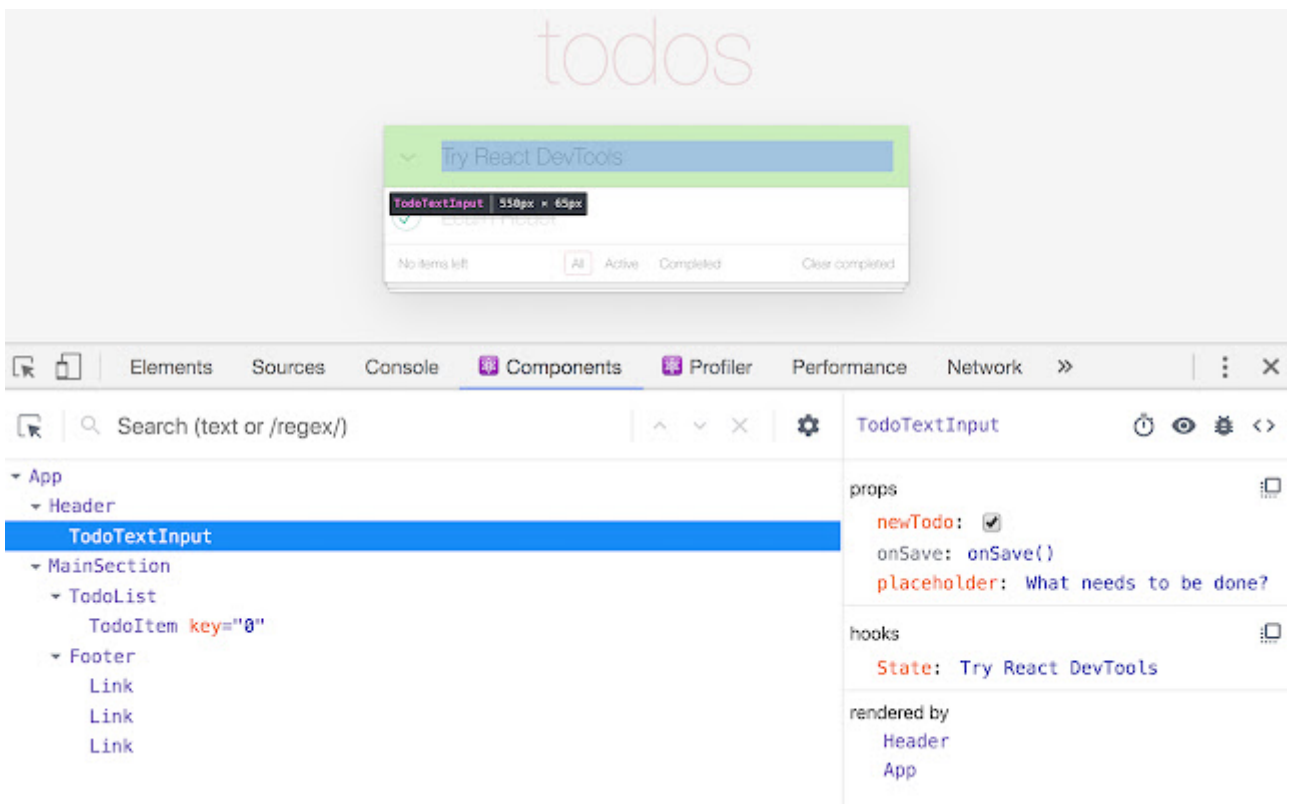
- Chrome devtools: Performance 主要细查性能运行时性能， 包含了 long tasks、render 次数、重排重绘、执行时间线、阻塞场景



- Chrome devtools: Performance monitor 主要监控用户运行时性能，看看是否有内存泄露



- React Developer Tools: 可以用于追踪 react 应用性能、渲染次数、重排重绘

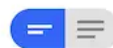


- Lighthouse: 全面分析网页性能的一个工具、支持浏览器插件



Performance

Metrics



■ First Contentful Paint

1.1 s

■ Speed Index

1.4 s

▲ Largest Contentful Paint

3.1 s

● Time to Interactive

1.2 s

● Total Blocking Time

0 ms

● Cumulative Layout Shift

0.033

@稀土掘金技术社区

- webpack-bundle-analyzer: 进行产物依赖分析、包大小分析
- 抓包: 通过抓包的方式, 看看线上请求分析、请求模拟、网络劫持之后仅仅看 JS 执行时间
- E2E测试: 通过 E2E 进行性能预检, 每次上线前进行一系列系统操作, 看看时间耗时和线上耗时波动

主要解决办法和思路

首屏加载慢的方向

- 资源加载方向
 - 使用 `tree shaking` 减少包体积
 - 代码压缩和混淆
 - 对于高版本浏览器, 直接使用 ES6 语法, 低版本浏览器再使用 ES5 (es6 语法代码量会比编译成 es5 代码量小很多, 且执行速度也快)
 - 使用 `split chunks` 进行大包拆分、小包复用
 - 使用 `gzip`
 - 使用 图片压缩
 - 使用 雪碧图
 - 图标使用 `iconfont` 加载
 - 懒加载, 仅加载首屏必要资源
 - 使用 `tailwindcss` 等技术, 复用 css
 - 使用 `微前端` 技术, 首屏仅加载当前子应用页面, 可以做到只加载整站很少的一部分代码
 - 首屏非必要依赖尽量延后到 FMP 或者 TTI 之后再加载

- 组件微前端化
- 渲染方向
 - 尽量减少重排重绘
 - 减少重复渲染（useMemo、useCallback、memo 等）
 - 减少 setState 次数（多次 setState 可以合并为一次）
 - 尽量减少 dom 节点深度
- 网络方向
 - 使用流式服务端渲染
 - 使用服务端渲染，减少首屏请求
 - 使用 SSG 静态站点生成
 - 首屏必要数据，不作客户端请求，用后端模板注入
 - 使用 BFF 进行请求聚合
 - 使用 CDN 进行网络请求分发
 - DNS Prefetch
 - 资源预加载（在闲暇时间加载后续页面所需要的资源和接口，例如：link rel preload）
 - 启用 HTTP2 多路复用
 - 在业务逻辑上，首屏必要接口提前（例如在 html 加载的那一瞬间，利用一个非常小的 js 文件将首屏需要的请求发送出去，然后缓存下来，到业务使用的时候就直接使用即可）
 - 使用缓存技术缓存资源与请求：强缓存、协商缓存、离线缓存、Service Worker 缓存、后端业务缓存

运行时卡顿方向

- 查看是否存在有 `long tasks`，有计划的拆解 `long tasks`
- 解决项目中复杂度问题
- 排查项目是否有内存泄露
- 排查特定业务流程是否有慢接口
- 高复杂计算逻辑放在 service worker 处理

91. [webpack] 如果解决重复引用 node_modules 里面的不同版本的包

解决重复引用 `node_modules` 中不同版本的包的问题，可以通过以下几种方式：

1.使用 npm 或者 yarn 的工具进行依赖的版本控制，尽量避免引用不同版本的同一个依赖库。在 package.json 文件中使用 "^"、"~"、">=" 等方式指定依赖版本，可以有效减少不同版本的包冲突问题。

2.使用 webpack 的 resolve.alias 配置选项，将需要共享的模块指定到一个目录下，然后在其它模块中使用别名引用该模块。例如，将需要共享的模块指定到 src/shared 目录下，然后在其它模块中使用别名 @Shared_CD 引用该模块，这样就可以保证在不同模块中引用相同的依赖库。

假设我们在项目中同时依赖了两个库：lodash 和 lodash-es，并且它们分别被安装在了不同的目录下，如下所示：

```
1 Copy codenode_modules/
2 |— lodash/
3 |— lodash-es/
```

我们需要在项目中同时引用这两个库，但是如果我们在代码中分别使用 import _ from 'lodash' 和 import _ from 'lodash-es'，那么 webpack 会将它们打包成两个独立的模块，导致代码体积变大。

为了解决这个问题，我们可以通过 resolve.alias 配置项将它们指向同一个模块。具体做法是在 webpack 配置文件中添加以下内容：

```
1 module.exports = { // ...
2   resolve: {
3     alias: {'lodash-es': 'lodash'}
4   }
5 }
6 }
```

这样一来，当我们在代码中使用 import _ from 'lodash-es' 时，webpack 会自动将它解析成对 lodash 的引用，从而避免了重复打包的问题。

3.使用 webpack 的 ProvidePlugin 插件，将需要共享的模块注入到全局作用域中，这样就可以在不同模块中共享相同的依赖库。例如，在 webpack 配置文件中添加以下代码：

```
1 const webpack = require('webpack');
2
3 module.exports = { // ...
4   plugins: [new webpack.ProvidePlugin({
5     $: 'jquery',
6     jQuery: 'jquery', 'window.jQuery': 'jquery',
7   })],
```

```
8     ],
9     };
```

这样在不同模块中就可以使用 \$、jQuery、window.jQuery 全局变量引用 jquery 依赖库，避免了重复引用不同版本的 jquery 包的问题。

4.使用 webpack 的 resolve.modules 配置选项，将 node_modules 目录移动到项目根目录之外，然后在 resolve.modules 中添加该目录的绝对路径，这样就可以解决不同模块中引用相同依赖库不同版本的问题。例如，在 webpack 配置文件中添加以下代码：

```
1  const path = require('path');
2
3  module.exports = {// ...
4    resolve: {
5      modules: [
6        path.resolve(__dirname, 'src'), 'node_modules',
7      ],
8    },
9  };
```

这样 webpack 在查找依赖库的时候，会先在项目根目录下的 src 目录中查找，如果没有找到再去 node_modules 目录中查找，避免了不同模块中引用相同依赖库不同版本的问题。

92. [webpack] 是如何实现 treeShaking 的

webpack实现tree shaking的原理是基于ES6模块化语法的静态特性。

在编译阶段，Webpack会根据模块的依赖关系，通过AST（抽象语法树）进行静态分析，识别出那些代码块（函数、变量、对象等）被引用并且使用了。然后将这些代码块打包输出到最终的打包文件中。在这个过程中，Webpack会自动将未被引用的代码块进行剔除，这个过程就是tree shaking。

具体来说，当Webpack在打包时遇到一个ES6模块导入语句（import），它会自动去加载这个模块并分析其导出对象。然后它会分析项目中哪些导出对象被引用了。如果一个导出对象没有被引用，那么Webpack会直接把它从最终的代码中剔除掉。

需要注意的是，tree shaking只对ES6模块生效，对于CommonJS等其他模块化规范，由于其动态加载特性，无法在静态分析阶段确定哪些代码块被引用，因此无法进行tree shaking。

另外，为了使Webpack能够正确识别和剔除未引用的代码块，开发者也需要做出一定的努力，例如将代码编写为纯函数的形式，避免使用全局变量等副作用等。

93. [webpack] loader 和 plugin 有啥区别

在Webpack中，Loader和Plugin是两个不同的概念，它们的作用和使用方式也有所不同。

Loader用于对源代码文件进行转换和处理，而Plugin用于对Webpack的编译过程进行扩展和增强。

- Loader

Loader是Webpack中的一个核心概念，它用于处理源代码文件，将它们转换成Webpack可处理的模块。Webpack在处理代码模块的过程中，会根据模块的类型来选择相应的Loader进行处理，例如，处理CSS文件需要使用css-loader，处理图片需要使用file-loader等。使用Loader可以实现代码转换、文件处理、代码压缩等功能。

Loader的使用方式是在Webpack的配置文件中定义module.rules属性，它是一个数组，每个元素是一个对象，用于描述如何处理特定类型的文件。一个Loader对象通常包括以下几个属性：

- test：用于匹配需要处理的文件类型，通常是一个正则表达式。
- use：指定需要使用的Loader，可以是一个字符串或一个数组，数组中的每个元素都是一个Loader。
- exclude/include：指定需要排除/包含的文件夹。

例如，处理CSS文件需要使用css-loader和style-loader，可以在Webpack配置文件中添加如下配置：

```
1  module: {
2    rules: [
3      {
4        test: /\.css$/,
5        use: ["style-loader", "css-loader"]
6      }
7    ]
8  }
```

- Plugin

Plugin是Webpack中的另一个核心概念，它用于扩展Webpack的功能。Plugin可以用于执行任意类型的任务，例如，生成HTML文件、压缩代码、提取公共代码等。使用Plugin可以实现Webpack无法处理的复杂任务。

Plugin的使用方式是在Webpack的配置文件中定义plugins属性，它是一个数组，每个元素是一个Plugin实例。Plugin通常包括以下几个方法：

- apply：用于安装插件，接收一个compiler对象作为参数。
- 一些Webpack钩子函数的实现。

例如，生成HTML文件需要使用HtmlWebpackPlugin，可以在Webpack配置文件中添加如下配置：

```
1  const HtmlWebpackPlugin = require('html-webpack-plugin');
2  module.exports = { // ...
3    plugins: [new HtmlWebpackPlugin({
```

```
4      title: 'My App',
5      template: './src/index.html'
6    })
7  ]
8  };
```

表格对比他们之间的差异和适用范围

区别	Loader	Plugin
输入/输出	输入文件，输出处理后的文件	可以在Webpack构建过程中处理输出结果或做额外处理
使用方式	在模块加载时直接调用	在Webpack配置中进行配置
功能	用于处理某些类型的文件	可以处理打包过程的各个环节
实现方式	导出一个函数	导出一个类
配置方式	在Webpack配置中使用	在Webpack配置中使用
作用	转换文件或模块	对整个构建过程进行自定义操作
适用场景	处理各种类型的文件，如css、图片等	执行比较复杂的操作，如代码压缩、代码分割等
使用方式	需要在Webpack中明确的指定	无法单独使用，必须在Webpack中明确的指定使用
作用对象	针对每一个文件进行处理	针对整个构建过程进行处理

总体而言，Loader主要用于针对单个文件进行处理，可以根据不同文件类型来选择对应的Loader；Plugin则是针对整个构建过程进行自定义操作，比如代码压缩、分离CSS文件、创建HTML文件等。

94. [webpack] webpack 是如何给 web 应用注入环境变量的，原理是啥

Webpack 可以通过 DefinePlugin 插件给 web 应用注入环境变量。该插件会在编译过程中替换掉代码中指定的变量，以实现在运行时替换成环境变量的值。

在 webpack 的配置文件中，需要先引入该插件，然后将需要注入的环境变量通过该插件进行配置。例如：

```
1  const webpack = require('webpack');
2
3  module.exports = { // 其他配置
4    plugins: [new webpack.DefinePlugin({'process.env': {
5      NODE_ENV: JSON.stringify('production')
6    }},
7      API_URL: JSON.stringify('http://api.example.com')
8    })
9  ]
10 };
```

上述配置中，定义了两个需要注入的变量，分别是 `process.env.NODE_ENV` 和 `API_URL`。其中，`process.env.NODE_ENV` 是一个常用的环境变量，用来标识当前是开发环境还是生产环境；`API_URL` 是一个自定义的环境变量，用来存储 API 的地址。

在代码中使用这些环境变量时，只需要直接引用即可：

```
1  if (process.env.NODE_ENV === 'production') {console.log('当前为生产环境');
2  }
3  fetch(API_URL + '/users')
4    .then(response => response.json())
5    .then(data => console.log(data));
```

Webpack 在编译时会将这些变量替换成对应的值，例如：

```
1  if ('production' === 'production') {console.log('当前为生产环境');
2  }
3  fetch('http://api.example.com'/users')
4    .then(response => response.json())
```

```
5 .then(data => console.log(data));
```

通过这种方式，我们就可以在代码中方便地使用环境变量，同时保证了在不同环境下都能正确地使用相应的变量值。

95. babel 的工作流程是如何的？

Babel 是一个 JavaScript 编译器，它的主要功能是将新版本的 JavaScript 代码转换成向后兼容的代码。Babel 的工作流程可以简单概括为以下几个步骤：

1. 解析：将 JavaScript 代码解析成 AST（抽象语法树）。
2. 转换：对 AST 进行遍历，进行代码转换。
3. 生成：将转换后的 AST 生成 JavaScript 代码。

具体来说，Babel 的工作流程如下：

1. Babel 使用 babylon 解析器将 JavaScript 代码解析成 AST，babylon 是一个基于 AST 的 JavaScript 解析器。
2. Babel 使用 babel-traverse 遍历器对 AST 进行遍历，找到需要转换的节点，进行转换。
3. Babel 使用 babel-core 转换器将 AST 转换成 JavaScript 代码。babel-core 是 babel 的核心模块，它包含了所有的转换器和插件。
4. Babel 使用 babel-generator 生成器将转换后的 AST 生成 JavaScript 代码。babel-generator 是一个将 AST 转换成 JavaScript 代码的工具。

在整个流程中，Babel 还会使用 babel-preset-env、babel-plugin-transform-runtime、babel-polyfill 等插件和工具来完成更加复杂的任务，如将 ES6 模块转换成 CommonJS 模块，使用 Polyfill 来实现一些新的 API 等。

需要注意的是，Babel 的转换过程是有损的，转换后的代码不一定与原始代码完全相同，也可能存在性能问题。因此，在使用 Babel 进行转换时，需要谨慎选择转换的规则和插件，以确保转换后的代码正确、高效。

96. [webpack] 什么情况下 webpack treeShaking 会失效？

以下是一些可能导致 webpack tree shaking 失效的情况

1. 代码中使用了动态引入（Dynamic Imports）的语法，这种情况下，webpack 无法确定哪些代码会被使用，因此不会进行 tree shaking。
2. 代码使用了函数式编程的方式，比如使用了 map、filter、reduce 等高阶函数，而这些函数很难通过静态分析确定代码的执行路径，所以可能会导致 tree shaking 失效。

3. 代码中使用了 webpack 无法识别的模块系统，比如使用了 AMD 或者 CommonJS 的语法，这种情况下 webpack 也无法进行 tree shaking。
4. 代码使用了 side effect，比如改变全局变量或者函数的参数，这种情况下 webpack 也无法进行 tree shaking。

函数式编程的方式 filter 为何会导致无法 tree shaking

函数式编程中常常使用高阶函数来组合函数，这种组合方式常常需要使用传递函数作为参数的方式，例如 map、filter 等高阶函数。这种情况下，如果参数传递的是一个函数表达式或者函数声明，那么无法进行 treeshaking。

举个例子：

```
1  // 代码中定义了一个 sum 函数function sum(a, b) {return a + b;
2  }
3
4  // 调用了 lodash 库的 filter 函数，传递一个匿名函数表达式作为参数import { filter }
   from 'lodash';
5
6  const arr = [1, 2, 3, 4, 5];
7  const result = filter(arr, item => {if (item > 10) return sum(item, 1)else
   return item;
8  });
```

上述代码中，使用了 lodash 库的 filter 函数，并且传递了一个匿名函数表达式作为参数。由于函数表达式无法被静态分析，不知道 sum 是否会被调用，因此无法进行 treeshaking，最终导致整个 sum 函数也被打包进了最终的代码中。

为什么 commonjs 模块化会导致无法 tree shaking

CommonJS 模块化语法是 Node.js 中的模块化规范，其使用了 `require()` 导入模块，使用 `module.exports` 或 `exports` 导出模块。它采用的是动态导入（`require()`）和同步加载的方式，这种导入方式无法在编译时确定所依赖的模块，因此在 Webpack 进行 Tree Shaking 时，这种导入方式的模块会被认为无法被静态分析，因而会被排除掉。

相反，ES6 模块化语法采用的是静态导入的方式，例如 `import foo from './foo.js'`，可以在编译时分析出所依赖的模块，因此支持 Tree Shaking。

因此，如果要使用 Tree Shaking，建议采用 ES6 模块化语法。如果必须使用 CommonJS 模块化规范，可以尝试使用动态导入（`import()`）语法，或者采用其他工具或手动实现 Tree Shaking。

side effect 是什么，为何会导致无法 tree shaking

在编写 JavaScript 代码时，如果一个函数除了返回值外，还对外部的变量产生了其他的影响，比如修改了全局变量、读写了文件等操作，那么这个函数就被称为有“副作用”（side effect）。因为这种函数并不是纯函数，它可能会影响其他部分的代码执行结果，不便于优化和调试。

在 Tree Shaking 的过程中，webpack 将模块打包成单独的 JavaScript 文件，它会从模块中找出哪些代码没有被使用到，并删除这些代码。但是，如果模块中存在带有副作用的代码，这些代码虽然没有被使用到，但它们仍然会被保留下来，因为这些代码可能会对其他部分的代码产生影响，因此不能简单地删除。这也是为什么带有副作用的代码会导致无法 Tree Shaking 的原因。

97. 不用使用 vue-cli ，如何创建一个完整的 vue 工程？

这个一个较为复杂和庞大的话题，不能称之为问题，只能说它是一个话题。

主要涉及到的话题如下：

1. vue 工程初始化
2. 测试集成
3. UI 库绑定、基础组件使用
4. 开发流程
5. 代码规范（甚至包含 commit 规范）
6. 多人协作与工作流
7. 构建问题
8. 上线流程
9. 线上日志与用户反馈问题排查
10. 性能保证

98. 使用 虚拟DOM 一定会比直接操作 真实 DOM 快吗？

大家惯有的思维模式下，我们普遍的认为，虚拟DOM一定会比原生DOM要快的多。

但实际上并不是这样。

仅从React的角度来说：React的官网可从来都没有说过虚拟DOM会比原生操作DOM更快。

虚拟DOM和原生操作DOM谁更快这个问题。如果要我来回答的话，一定是原生DOM比虚拟DOM更快性能更好。

值得注意的是，虚拟DOM并不是比原生DOM快，更确切的来说，虚拟DOM是比操作不当的原生DOM快。实际上，如果对原生DOM的操作得当的话，原生DOM的性能一定优于虚拟DOM。

我们来剖析一下。

虚拟DOM为什么而存在

其最核心的思想是提升开发效率而非提升性能

使用 React/Vue 这些框架的时候，我们不需要去考虑对DOM的操作，只需要关心数据的改变。我们以前还在使用JQ的时候，数据改变之后我们需要调用 `$("#id").append(node)` 等操作去手动追加DOM。而在使用React/Vue之后，我们只需要关心数据的改变。至于对DOM的一系列动作，在我们的数据改变之后，React/Vue会为我们代劳。这极大程度的提升了我们的开发效率。也是React/Vue的核心思想和初衷。

至于很多人都说，虚拟DOM会比操作原生DOM更快，这个说法并不全面。比如，首次渲染或者所有节点都需要进行更新的时候。这个时候采用虚拟DOM会比直接操作原生DOM多一重构建虚拟DOM树的操作。这会更大的占用内存和延长渲染时间。

举个例子

首次渲染👉不采用虚拟DOM的步骤

1. 浏览器接受绘制指令
2. 创建所有节点

首次渲染👉采用虚拟DOM的步骤

1. 浏览器接受绘制指令
2. 创建虚拟DOM
3. 创建所有节点

不难发现，在首次渲染的时候，采用虚拟DOM会比不采用虚拟DOM要多一个创建虚拟DOM的步骤。

注意:虚拟DOM的存在，并不是免费的，比对新旧虚拟DOM树的差异会带来一定的性能开销。

虚拟DOM的优势在于我们更新节点时候。它会检查哪些节点需要更新。尽量复用已有DOM，减少DOM的删除和重新创建。并且这些操作我们是可以通过自己手动操作javascript底层api实现的。只是我们手动操作会非常耗费我们的时间和精力。这个工作由虚拟DOM代劳，会让我们开发更快速便捷。

举个例子👉

在采用虚拟DOM的前提下

假设我们有节点A，下辖两个子节点B/C.

然后我们删除了节点C

这个时候会有两棵虚拟DOM树，

一颗是修改前的，A->B/C。

另一颗是修改后的A->B。

diff算法会去比对两颗树的差异，然后发现A->B没有更改，那么A->B节点保留，C节点执行删除动作。

那么，A->B两个节点的删除和创建渲染操作就被省略了。

如果不采用虚拟DOM的话。使用JQ那时候的模板。

我们可能会把A->B/C三个节点全部删除。

再全都重新创建。而A->B是完全没有改动的。

他们的删除和创建则完全不必要。

框架的意义

我们需要知道:不论是React还是Vue或者是Angular。这些框架本身，都是基于原生的基础上创造的。它们，底层依赖的还是javascript，并不是一门新的语言。在他们的底层逻辑下。我们使用框架所做出的一切行为，都会被框架转化为对原生DOM的操作。框架，只是一个转化语法的工具。

既然原生DOM可以创造出这些框架。当然我们使用原生DOM自然是可以写出比这些框架更好的性能。

但是:为什么对原生DOM进行操作的性能明明可以比使用框架更好。为什么大家都在使用框架，而没有人去直接对原生DOM进行操作。

这背后涉及 **成本** 和 **普适性**。

如果我们直接去操作真实DOM,当然，我们可以做到在性能上比虚拟DOM更快。但问题是，技术水准能做到这个地步的人，又有多少人呢。不说比虚拟DOM快。即使是做到和虚拟DOM不分上下的性能，拥有这种水平的前端玩家，也是寥寥无几。基于这样的客观情况下，框架的出现解决了这个问题。

框架存在的意义：在为我们提供只需要关注数据的前提下。框架本身已经做好了底层原理上的性能优化（包括但不限于,对DOM的调用,算法上的优化）已经是高度封装。这样就可以让我们使用一些简单的较为容易理解的技术去做我们原本做不到的事情。这其实就像调用网上的第三方包，某一个功能，自己写是写不出来，写出来性能也不会很好。但是同样的功能，我们去网上引入其他大神已经封装完成的第三方包。我们就会用，功能就可以实现并且性能上也过得去。

如果大家直接对DOM进行操作完成比框架更优秀的性能。这绝不是大多数人可以做到的。让大多数可以接受，框架需要做的，就是让大多数人使用尽量使用简单的技术，完成相对困难的操作。这是 **普适性**。

并且，如果完成同一个性能效果，需要我们去精通原生javascript和学习框架上的一些简单的API和结构。明显后者的学习成本更低。如果说使用框架我们所能完成的某一阶段的性能所需要的学习成本是2个月的话。那么学习javascript完成同一阶段的性能可能需要一年。

框架的初衷就是让用户使用尽量简单的技术，完成相对复杂的工作并提升一定的性能（这其中包括但不限于:可维护性，可复用性，渲染效率等）。这样，即使我们的水平不是很高，使用框架以后。项目在性能上也能过得去。

总结

1. 虚拟DOM不一定会比操作原生DOM更快。
2. 虚拟DOM的优势在于节点进行改动的时候尽量减少开销
3. React从来没说过虚拟DOM会比原生更快。

4. 框架的本质是提升开发效率，让我们的注意力更集中于数据

99. AMD 和 CMD 模块化有和区别？

AMD（Asynchronous Module Definition）和CMD（Common Module Definition）都是JavaScript模块化方案。它们的主要区别在于对依赖的处理方式上不同。

AMD是在require.js推广过程中诞生的，它的特点是提前执行，强调依赖前置。也就是说，在定义模块时就需要声明其所有依赖的模块。它的语法如下：

```
1  define(['dependency1', 'dependency2'], function(dependency1, dependency2) {  
    模块的定义  
2  });
```

CMD是在Sea.js推广过程中诞生的，它和AMD非常相似，但是更加懒惰，是依赖就近，延迟执行。也就是说，在模块中需要用到依赖时，才去引入依赖。它的语法如下：

```
1  define(function(require, exports, module) {  
    var dependency1 =  
    require('dependency1');  
    var dependency2 = require('dependency2');  
    模块的定义  
2  });
```

简单来说，AMD是提前执行、依赖前置，CMD是延迟执行、依赖就近。两种模块化方案各有优缺点，选择哪种模块化方案需要根据实际情况和个人偏好进行考虑。

100. 前端模块化发展历程？

前端模块化是指在前端开发中，通过模块化的方式组织代码，将代码按照一定规则分割成不同的模块，便于管理和维护。

前端模块化的发展历程如下：

1. 早期，前端开发采用的是全局变量的方式进行开发，即将所有代码都放在一个文件中，通过全局变量进行交互。这种方式的问题在于，代码量较大，代码耦合度高，不易维护。
2. 后来，前端开发采用了命名空间的方式进行组织代码，即将代码放在一个命名空间下，通过命名空间进行交互。这种方式解决了全局变量带来的问题，但是在开发大型应用时，仍然存在代码耦合度高、依赖管理不便等问题。
3. 2009年，CommonJS提出了一种新的模块化规范，即将每个模块封装在一个独立的文件中，通过require和exports进行模块之间的依赖管理和导出。这种方式解决了前两种方式带来的问题，但是由于该规范是同步加载模块，不适用于浏览器环境。

- 2011年，AMD规范提出，即异步模块定义规范，采用异步的方式加载模块，可以在浏览器环境下使用。该规范主要是通过require和define方法进行模块之间的依赖管理和导出。
- 2013年，CommonJS和AMD的创始人合并了两种规范，提出了新的规范——CommonJS 2.0规范。该规范在CommonJS 1.0的基础上，增加了异步加载的功能，使其可以在浏览器环境下使用。
- 2014年，ES6（即ECMAScript 2015）正式发布，引入了模块化的支持，即通过import和export语句进行模块之间的依赖管理和导出。ES6的模块化规范具有更好的可读性、可维护性和性能优势，已成为前端开发的主流方式。
- 同时，还有一些第三方库，如RequireJS、SeaJS等，提供了更加灵活和可扩展的模块化方式，使得前端开发的模块化更加便捷和高效。

101. 前端单页应用 History 路由模式， 需要如何配置 nginx?

前端单页应用使用 History 路由模式，需要将路由的控制交给前端，而不是后端处理，因此需要配置 nginx 让其将所有请求都指向 index.html 页面，让前端来处理路由。

具体配置可以参考以下 nginx 配置：

```
1  bashCopy codeserver {
2      listen      80;
3      server_name  example.com; 域名或 IP 地址
4      location / {
5          root     /path/to/your/app; 项目根目录
6          index    index.html;
7          try_files $uri $uri/ /index.html; 将所有请求都指向 index.html 页面
8      }
9  }
```

以上配置中，`try_files` 指令会尝试将请求映射到对应的文件，如果找不到，则会将请求转发到 `/index.html` 页面，由前端来处理路由。同时，`root` 指令需要指向前端应用的根目录。

102. 实现 JS 沙盒的方式有哪些？

微前端已经成为前端领域比较火爆的话题，在技术方面，微前端有一个始终绕不过去的话题就是前端沙箱

什么是沙箱

Sandboxie(又叫沙箱、沙盘)即是一个虚拟系统程序，允许你在沙盘环境中运行浏览器或其他程序，因此运行所产生的变化可以随后删除。它创造了一个类似沙盒的独立作业环境，在其内部运行的程序

并不能对硬盘产生永久性的影响。在网络安全中，沙箱指在隔离环境中，用以测试不受信任的文件或应用程序等行为的工具

简单来说沙箱（sandbox）就是与外界隔绝的一个环境，内外环境互不影响，外界无法修改该环境内任何信息，沙箱内的东西单独属于一个世界。

JavaScript 的沙箱

对于 JavaScript 来说，沙箱并非传统意义上的沙箱，它只是一种语法上的 Hack 写法，沙箱是一种安全机制，把一些不信任的代码运行在沙箱之内，使其不能访问沙箱之外的代码。当需要解析或着执行不可信的 JavaScript 的时候，需要隔离被执行代码的执行环境的时候，需要对执行代码中可访问对象进行限制，通常开始可以把 JavaScript 中处理模块依赖关系的闭包称之为沙箱。

JavaScript 沙箱实现

我们大致可以把沙箱的实现总体分为两个部分：

- 构建一个闭包环境
- 模拟原生浏览器对象

构建闭包环境

我们知道 JavaScript 中，关于作用域（scope），只有全局作用域（global scope）、函数作用域（function scope）以及从 ES6 开始才有的块级作用域（block scope）。如果要将一段代码中的变量、函数等的定义隔离出来，受限于 JavaScript 对作用域的控制，只能将这段代码封装到一个 Function 中，通过使用 function scope 来达到作用域隔离的目的。也因为需要这种使用函数来达到作用域隔离的目的方式，于是就有 IIFE（立即调用函数表达式），这是一个被称为 自执行匿名函数的设计模式

```
1  (function foo(){var a = 1;console.log(a);
2  })();// 无法从外部访问变量 nameconsole.log(a) // 抛出错误: "Uncaught
    ReferenceError: a is not defined"
```

当函数变成立即执行的函数表达式时，表达式中的变量不能从外部访问，它拥有独立的词法作用域。不仅避免了外界访问 IIFE 中的变量，而且又不会污染全局作用域，弥补了 JavaScript 在 scope 方面的缺陷。一般常见于写插件和类库时，如 JQuery 当中的沙箱模式

```
1  (function (window) {var jQuery = function (selector, context) {return new
    jQuery.fn.init(selector, context);
2      }
3      jQuery.fn = jQuery.prototype = function () {//原型上的方法，即所有jQuery对象
        都可以共享的方法和属性
4      }
    })
```

```
5     jQuery.fn.init.prototype = jQuery.fn;window.jQuery = window.$ = jQuery; //
    如果需要在外界暴露一些属性或者方法，可以将这些属性和方法加到window全局对象上去
6 })(window);
```

当将 IIFE 分配给一个变量，不是存储 IIFE 本身，而是存储 IIFE 执行后返回的结果。

```
1  var result = (function () {
2      var name = "张三";
3      return name;
4  })();
5  console.log(result); // "张三"
```

原生浏览器对象的模拟

模拟原生浏览器对象的目的是为了，防止闭包环境，操作原生对象。篡改污染原生环境；完成模拟浏览器对象之前我们需要先关注几个不常用的 API。

eval

eval 函数可将字符串转换为代码执行，并返回一个或多个值

```
1  var b = eval("({name:'张三'})")console.log(b.name);
```

由于 eval 执行的代码可以访问闭包和全局范围，因此就导致了代码注入的安全问题，因为代码内部可以沿着作用域链往上找，篡改全局变量，这是我们不希望的

new Function

Function 构造函数创建一个新的 Function 对象。直接调用这个构造函数可用动态创建函数

语法

```
new Function ([arg1[, arg2[, ...argN]],] functionBody)
```

arg1, arg2, ... argN 被函数使用的参数的名称必须是合法命名的。参数名称是一个有效的 JavaScript 标识符的字符串，或者一个用逗号分隔的有效字符串的列表;例如 “x”，“theValue”，或 “a,b”。

functionBody 一个含有包括函数定义的 JavaScript 语句的字符串。

```
1  const sum = new Function('a', 'b', 'return a + b');
2
3  console.log(sum(1, 2)); //3
```

同样也会遇到和 eval 类似的安全问题和相对较小的性能问题。

```
1  var a = 1;
2
3  function sandbox() {var a = 2;return new Function('return a;'); // 这里的 a 指
   向最上面全局作用域内的 1
4  }
5  var f = sandbox();
6  console.log(f());
```

与 eval 不同的是 Function 创建的函数只能在全局作用域中运行。它无法访问局部闭包变量，它们总是被创建于全局环境，因此在运行时它们只能访问全局变量和自己的局部变量，不能访问它们被 Function 构造器创建时所在的作用域的变量；但是，它仍然可以访问全局范围。new Function()是 eval()更好替代方案。它具有卓越的性能和安全性，但仍没有解决访问全局的问题。

with

with 是 JavaScript 中一个关键字,扩展一个语句的作用域链。它允许半沙盒执行。那什么叫半沙盒？语句将某个对象添加到作用域链的顶部，如果在沙盒中有某个未使用命名空间的变量，跟作用域链中的某个属性同名，则这个变量将指向这个属性值。如果没有同名的属性，则将抛出 ReferenceError。

```
1  function sandbox(o) {with (o){//a=5;
2      c=2;
3      d=3;console.log(a,b,c,d); // 0,1,2,3 //每个变量首先被认为是一个局部变量，
   如果局部变量与 obj 对象的某个属性同名，则这个局部变量会指向 obj 对象属性。
4  }
5  }
6  var f = {
7      a:0,
8      b:1
9  }
10 sandbox(f);
11 console.log(f);
12 console.log(c,d); // 2,3 c、d被泄露到window对象上
```

究其原理，with 在内部使用 in 运算符。对于块内的每个变量访问，它都在沙盒条件下计算变量。如果条件是 true，它将从沙盒中检索变量。否则，就在全局范围内查找变量。但是 with 语句使程序在查找变量值时，都是先在指定的对象中查找。所以对于那些本来不是这个对象的属性的变量，查找起来会很慢，对于有性能要求的程序不适合（JavaScript 引擎会在编译阶段进行数项的性能优化。其中有些优化依赖于能够根据代码的词法进行静态分析，并预先确定所有变量和函数的定义位置，才能在

执行过程中快速找到标识符。)。with 也会导致数据泄漏(在非严格模式下,会自动在全局作用域创建一个全局变量)

in 运算符

in 运算符能够检测左侧操作数是否为右侧操作数的成员。其中,左侧操作数是一个字符串,或者可以转换为字符串的表达式,右侧操作数是一个对象或数组。

```
1  var o = {
2      a : 1,
3      b : function() {}
4  }
5  console.log("a" in o); //trueconsole.log("b" in o); //trueconsole.log("c" in
    o); //falseconsole.log("valueOf" in o); //返回true, 继承Object的原型方法
    console.log("constructor" in o); //返回true, 继承Object的原型属性
```

with + new Function

配合 with 用法可以稍微限制沙盒作用域,先从当前的 with 提供对象查找,但是如果查找不到依然还能从上获取,污染或篡改全局环境。

```
1  function sandbox (src) {
2      src = 'with (sandbox) {'src + '}'return new Function('sandbox', src)
3  }
4  var str = 'let a = 1;window.name="张三";console.log(a);console.log(b)'var b =
    2
5  sandbox(str)({});
6  console.log(window.name);// '张三'
```

基于 Proxy 实现的沙箱(ProxySandbox)

由上部分内容思考,假如可以做到在使用 with 对于块内的每个变量访问都限制在沙盒条件下计算变量,从沙盒中检索变量。那么是否可以完美的解决JavaScript沙箱机制。

使用 with 再加上 proxy 实现 JavaScript 沙箱

ES6 Proxy 用于修改某些操作的默认行为,等同于在语言层面做出修改,属于一种“元编程”(meta programming)

```
1  function sandbox(code) {
2      code = 'with (sandbox) {'code + '}'const fn = new Function('sandbox',
    code)
3  return function (sandbox) {const sandboxProxy = new Proxy(sandbox, {
```

```

4         has(target, key) {return true
5     }
6     })return fn(sandboxProxy)
7 }
8 }
9 var a = 1;
10 var code = 'console.log(a)' // TypeError: Cannot read property 'log' of
    undefined
11 sandbox(code)({})

```

我们前面提到 `with` 在内部使用 `in` 运算符来计算变量，如果条件是 `true`，它将从沙盒中检索变量。理想状态下没有问题，但也总有些特例独行的存在，比如 `Symbol.unscopables`。

Symbol.unscopables

`Symbol.unscopables` 对象的 `Symbol.unscopables` 属性，指向一个对象。该对象指定了使用 `with` 关键字时，哪些属性会被 `with` 环境排除。

```

1 Array.prototype[Symbol.unscopables]
2 // {/// copyWithin: true,/// entries: true,/// fill: true,/// find:
    true,/// findIndex: true,/// keys: true//
    }Object.keys(Array.prototype[Symbol.unscopables])
3 // ['copyWithin', 'entries', 'fill', 'find', 'findIndex', 'keys']

```

由此我们的代码还需要修改如下：

```

1 function sandbox(code) {
2     code = 'with (sandbox) {'code + '}'const fn = new Function('sandbox',
    code)
3     return function (sandbox) {const sandboxProxy = new Proxy(sandbox, {
4         has(target, key) {return true
5     },
6         get(target, key) {if (key === Symbol.unscopables) return
    undefinedreturn target[key]
7     }
8     })return fn(sandboxProxy)
9 }
10 }
11 var test = {
12     a: 1,
13     log(){console.log('1111')}
14 }
15 }

```

```
16 var code = 'log();console.log(a)' // 1111,TypeError: Cannot read property
    'log' of undefined
17 sandbox(code)(test)
```

Symbol.unscopables 定义对象的不可作用属性。Unscopeable 属性永远不会从 with 语句中的沙箱对象中检索，而是直接从闭包或全局范围中检索。

快照沙箱(SnapshotSandbox)

以下是 qiankun 的 snapshotSandbox 的源码，这里为了帮助理解做部分精简及注释。

```
1  function iter(obj, callbackFn) {for (const prop in obj) {if
    (obj.hasOwnProperty(prop)) {
2      callbackFn(prop);
3    }
4  }
5 }
6
7 /**
8  基于 diff 方式实现的沙箱，用于不支持 Proxy 的低版本浏览器
9  */class SnapshotSandbox {
10     constructor(name) {this.name = name;this.proxy = window;this.type =
        'Snapshot';this.sandboxRunning = true;this.windowSnapshot =
        {};this.modifyPropsMap = {};this.active();
11     }//激活
12     active() {// 记录当前快照this.windowSnapshot = {};
13         iter(window, (prop) => {this.windowSnapshot[prop] = window[prop];
14             });
15     // 恢复之前的变更Object.keys(this.modifyPropsMap).forEach((p) => {window[p] =
        this.modifyPropsMap[p];
16     });
17     this.sandboxRunning = true;
18     }//还原
19     inactive() {this.modifyPropsMap = {};
20         iter(window, (prop) => {if (window[prop] !==
        this.windowSnapshot[prop]) {// 记录变更，恢复环境this.modifyPropsMap[prop] =
        window[prop];
21     window[prop] = this.windowSnapshot[prop];
22     }
23     });this.sandboxRunning = false;
24     }
25 }
26 let sandbox = new SnapshotSandbox();
27 //test
```



```

28 ((window) => {window.name = '张三'window.age = 18console.log(window.name,
    window.age) //      张三,18
29     sandbox.inactive() //      还原console.log(window.name, window.age) //
        undefined,undefined
30     sandbox.active() //      激活console.log(window.name, window.age) //
        张三,18
31 })(sandbox.proxy);

```

快照沙箱实现来说比较简单，主要用于不支持 Proxy 的低版本浏览器，原理是基于 `diff` 来实现的,在子应用激活或者卸载时分别去通过快照的形式记录或还原状态来实现沙箱，snapshotSandbox 会污染全局 window。

legacySandBox

qiankun 框架 singular 模式下 proxy 沙箱实现，为了便于理解，这里做了部分代码的精简和注释。

```

1  //legacySandBoxconst callableFnCacheMap = new WeakMap();
2
3  function isCallable(fn) {if (callableFnCacheMap.has(fn)) {return true;
4    }const naughtySafari = typeof document.all === 'function' && typeof
    document.all === 'undefined';const callable = naughtySafari ? typeof fn ===
    'function' && typeof fn !== 'undefined' : typeof fn ==='function';if
    (callable) {
5      callableFnCacheMap.set(fn, callable);
6    }return callable;
7  };
8
9  function isPropConfigurable(target, prop) {const descriptor =
    Object.getOwnPropertyDescriptor(target, prop);return descriptor ?
    descriptor.configurable : true;
10 }
11
12 function setWindowProp(prop, value, toDelete) {if (value === undefined &&
    toDelete) {delete window[prop];
13   } else if (isPropConfigurable(window, prop) && typeof prop !== 'symbol')
    {Object.defineProperty(window, prop, {
14       writable: true,
15       configurable: true
16     });window[prop] = value;
17   }
18 }
19
20 function getTargetValue(target, value) {/*
21   仅绑定 isCallable && !isBoundedFunction && !isConstructable 的函数对象，如
    window.console、window.atob 这类。目前没有完美的检测方式，这里通过 prototype 中是否

```

还有可枚举的拓展方法的方式来判断

```
22      @warning 这里不要随意替换成别的判断方式, 因为可能触发一些 edge case (比如在
    lodash.isFunction 在 iframe 上下文中可能由于调用了 top window 对象触发的安全异常)
23      */if (isCallable(value) && !isBoundedFunction(value) &&
    !isConstructable(value)) {const boundValue =
    Function.prototype.bind.call(value, target);for (const key in value) {
24          boundValue[key] = value[key];
25      }if (value.hasOwnProperty('prototype') &&
    !boundValue.hasOwnProperty('prototype')) {Object.defineProperty(boundValue,
    'prototype', {
26          value: value.prototype,
27          enumerable: false,
28          writable: true
29      });
30      }
31      return boundValue;
32      }
33      return value;
34      }
35
36      /**
37      基于 Proxy 实现的沙箱
38      */class SingularProxySandbox {/** 沙箱期间新增的全局变量 */
39          addedPropsMapInSandbox = new Map();
40      /** 沙箱期间更新的全局变量 */
41          modifiedPropsOriginalValueMapInSandbox = new Map();
42      /** 持续记录更新的(新增和修改的)全局变量的 map, 用于在任意时刻做 snapshot */
43          currentUpdatedPropsValueMap = new Map();
44          name;
45          proxy;
46          type = 'LegacyProxy';
47          sandboxRunning = true;
48          latestSetProp = null;
49          active() {if (!this.sandboxRunning)
    {this.currentUpdatedPropsValueMap.forEach((v, p) => setWindowProp(p, v));
50          }
51          this.sandboxRunning = true;
52          }
53          inactive() {// console.log(' this.modifiedPropsOriginalValueMapInSandbox',
    this.modifiedPropsOriginalValueMapInSandbox)// console.log('
    this.addedPropsMapInSandbox', this.addedPropsMapInSandbox)//删除添加的属性, 修改
    已有的属性this.modifiedPropsOriginalValueMapInSandbox.forEach((v, p) =>
    setWindowProp(p, v));this.addedPropsMapInSandbox.forEach((_, p) =>
    setWindowProp(p, undefined, true));
54          this.sandboxRunning = false;
55          }
56          constructor(name) {this.name = name;const {
```

```

57         addedPropsMapInSandbox,
58         modifiedPropsOriginalValueMapInSandbox,
59         currentUpdatedPropsValueMap
60     } = this;
61     const rawWindow = window; // Object.create(null) 的方式，传入一个不含有原型链的对象
62     const fakeWindow = Object.create(null);
63     const proxy = new Proxy(fakeWindow, {
64         set: (_, p, value) => { if (this.sandboxRunning) { if
65 (!rawWindow.hasOwnProperty(p)) {
66             addedPropsMapInSandbox.set(p, value);
67         } else if (!modifiedPropsOriginalValueMapInSandbox.has(p)) { // 如果当
68 前 window 对象存在该属性，且 record map 中未记录过，则记录该属性初始值 const
69 originalValue = rawWindow[p];
70             modifiedPropsOriginalValueMapInSandbox.set(p, originalValue);
71         }
72         currentUpdatedPropsValueMap.set(p, value); // 必须重新设置 window 对象
73 保证下次 get 时能拿到已更新的数据
74         rawWindow[p] = value;
75     this.latestSetProp = p;
76     return true;
77     }
78     // 在 strict-mode 下，Proxy 的 handler.set 返回 false 会抛出 TypeError，在沙箱卸载
79 的情况下应该忽略错误 return true;
80     },
81     get(_, p) { // 避免使用 window.window 或者 window.self 逃离沙箱环境，触发到真
82 实环境 if (p === 'top' || p === 'parent' || p === 'window' || p === 'self')
83 {return proxy;
84     } const value = rawWindow[p]; return getTargetValue(rawWindow, value);
85     },
86     has(_, p) { // 返回 boolean return p in rawWindow;
87     },
88     getOwnPropertyDescriptor(_, p) { const descriptor =
89 Object.getOwnPropertyDescriptor(rawWindow, p); // 如果属性不作为目标对象的自身属性
90 存在，则不能将其设置为不可配置 if (descriptor && !descriptor.configurable) {
91     descriptor.configurable = true;
92     } return descriptor;
93     },
94     });
95     this.proxy = proxy;
96     }
97 }
98
99 let sandbox = new SingularProxySandbox();
100 ((window) => { window.name = '张三'; window.age = 18; window.sex =
101 '男'; console.log(window.name, window.age, window.sex) // 张三, 18, 男
102     sandbox.inactive() // 还原 console.log(window.name,
103 window.age, window.sex) // 张三, undefined, undefined

```

```

92     sandbox.active() //          激活console.log(window.name,
    window.age,window.sex) //      张三,18,男
93   })(sandbox.proxy); //test

```

legacySandBox 还是会操作 window 对象，但是他通过激活沙箱时还原应用的状态，卸载时还原主应用的状态来实现沙箱隔离的，同样会对 window 造成污染，但是性能比快照沙箱好，不用遍历 window 对象。

proxySandbox(多例沙箱)

在 qiankun 的沙箱 proxySandbox 源码里面是对 fakeWindow 这个对象进行了代理，而这个对象是通过 createFakeWindow 方法得到的，这个方法是将 window 的 document、location、top、window 等等属性拷贝一份，给到 fakeWindow。

源码展示：

```

1  function createFakeWindow(global: Window) {// map always has the fastest
    performance in has check scenario// see https://jsperf.com/array-indexof-vs-
    set-has/23const propertiesWithGetter = new Map<PropertyKey, boolean>();const
    fakeWindow = {} as FakeWindow;
2  /*
3      copy the non-configurable property of global to fakeWindow
4      see https://developer.mozilla.org/en-
    US/docs/Web/JavaScript/Reference/Global_Objects/Proxy/handler/getOwnPropertyDe
    scriptor
5      A property cannot be reported as non-configurable, if it does not exists as
    an own property of the target object or if it exists as a configurable own
    property of the target object.
6      */Object.getOwnPropertyNames(global)
7      .filter((p) => {const descriptor =
    Object.getOwnPropertyDescriptor(global, p);return !descriptor?.configurable;
8      })
9      .forEach((p) => {const descriptor =
    Object.getOwnPropertyDescriptor(global, p);if (descriptor) {const hasGetter =
    Object.prototype.hasOwnProperty.call(descriptor, 'get');
10     /*
11         make top/self/window property configurable and writable, otherwise
    it will cause TypeError while get trap return.
12         see https://developer.mozilla.org/en-
    US/docs/Web/JavaScript/Reference/Global_Objects/Proxy/handler/get
13         > The value reported for a property must be the same as the value of
    the corresponding target object property if the target object property is a
    non-writable, non-configurable data property.
14         */if (
15         p === 'top' ||

```

```

16         p === 'parent' ||
17         p === 'self' ||
18         p === 'window' ||
19         (process.env.NODE_ENV === 'test' && (p === 'mockTop' || p ===
    'mockSafariTop'))
20     ) {
21         descriptor.configurable = true; /*
22         The descriptor of window.window/window.top/window.self in
    Safari/FF are accessor descriptors, we need to avoid adding a data descriptor
    while it was
23         Example:
24         Safari/FF: Object.getOwnPropertyDescriptor(window, 'top') ->
    {get: function, set: undefined, enumerable: true, configurable: false}
25         Chrome: Object.getOwnPropertyDescriptor(window, 'top') -> {value:
    Window, writable: false, enumerable: true, configurable: false}
26         */if (!hasGetter) {
27             descriptor.writable = true;
28         }
29     }
30     if (hasGetter) propertiesWithGetter.set(p, true);
31     // freeze the descriptor to avoid being modified by zone.js// see
    https://github.com/angular/zone.js/blob/a5fe09b0fac27ac5df1fa746042f96f05ccb6a
    00/lib/browser/define-property.ts#L71
32     rawObjectDefineProperty(fakeWindow, p, Object.freeze(descriptor));
33 }
34 });
35 return {
36     fakeWindow,
37     propertiesWithGetter,
38 };
39 }

```

proxySandbox 由于是拷贝复制了一份 fakeWindow，不会污染全局 window，同时支持多个子应用同时加载。

关于 CSS 隔离

常见的有：

- CSS Module
- namespace
- Dynamic StyleSheet
- css in js
- Shadow DOM 常见的我们这边不再赘述，这里我们重点提一下Shadow DO。

Shadow DOM

Shadow DOM 允许将隐藏的 DOM 树附加到常规的 DOM 树中——它以 shadow root 节点为起始根节点，在这个根节点的下方，可以是任意元素，和普通的 DOM 元素一样。

103. pnpm 了解多少？

pnpm，英文里面的意思叫做 `performant npm`，意味“高性能的 npm”

pnpm 相比较于 yarn/npm 这两个常用的包管理工具在性能上也有了极大的提升，根据目前官方提供的 [benchmark](#) 数据可以看出在一些综合场景下比 npm/yarn 快了大概两倍：

在这篇文章中，将会介绍一些关于 pnpm 在依赖管理方面的优化，在 monorepo 中相比较于 yarn workspace 的应用，以及也会介绍一些 pnpm 目前存在的一些缺陷，包括讨论一下未来 pnpm 会做的一些事情。

依赖管理

这节会通过 pnpm 在依赖管理这一块的一些不同于正常包管理工具的一些优化技巧。

hard link 机制

介绍 pnpm 一定离不开的就是关于 pnpm 在安装依赖方面做的一些优化，根据前面的 benchmark 图可以看到其明显的性能提升。

那么 pnpm 是怎么做到如此大的提升的呢？是因为计算机里面一个叫做 [Hard link](#) 的机制，`hard link` 使得用户可以通过不同的路径引用方式去找到某个文件。pnpm 会在全局的 store 目录里存储项目 `node_modules` 文件的 `hard links`。

举个例子，例如项目里面有个 1MB 的依赖 a，在 pnpm 中，看上去这个 a 依赖同时占用了 1MB 的 `node_modules` 目录以及全局 store 目录 1MB 的空间(加起来是 2MB)，但因为 `hard link` 的机制使得两个目录下相同的 1MB 空间能从两个不同位置进行寻址，因此实际上这个 a 依赖只用占用 1MB 的空间，而不是 2MB。

Store 目录

上一节提到 store 目录用于存储依赖的 hard links，这一节简单介绍一下这个 store 目录。

一般 store 目录默认是设置在 `${os.homedir}/.pnpm-store` 这个目录下，具体可以参考 `@pnpm/store-path` 这个 pnpm 子包中的代码：

```
1  const homedir = os.homedir()
2  if (await canLinkToSubdir(tempFile, homedir)) {
3    await fs.unlink(tempFile) // If the project is on the drive on which the OS
    home directory // then the store is placed in the home directory return
    path.join(homedir, relStore, STORE_VERSION)
4  }
```

当然用户也可以在 `.npmrc` 设置这个 store 目录位置，不过一般而言 store 目录对于用户来说感知程度是比较小的。

因为这样一个机制，导致每次安装依赖的时候，如果是个相同的依赖，有好多项目都用到这个依赖，那么这个依赖实际上最优情况(即版本相同)只用安装一次。

如果是 npm 或 yarn，那么这个依赖在多个项目中使用，在每次安装的时候都会被重新下载一次。

如图可以看到在使用 pnpm 对项目安装依赖的时候，如果某个依赖在 store 目录中不存在的话，那么就会直接从 store 目录里面去 hard-link，避免了二次安装带来的时间消耗，如果依赖在 store 目录里面不存在的话，就会去下载一次。

当然这里你可能也会有问题：如果安装了很多很多不同的依赖，那么 store 目录会不会越来越大？

答案是当然会存在，针对这个问题，pnpm 提供了一个命令来解决这个问题: [pnpm store | pnpm](#)。

同时该命令提供了一个选项，使用方法为 `pnpm store prune`，它提供了一种用于删除一些不被全局项目所引用到的 packages 的功能，例如有个包 `axios@1.0.0` 被一个项目所引用了，但是某次修改使得项目里这个包被更新到了 `1.0.1`，那么 store 里面的 1.0.0 的 axios 就就成了个不被引用的包，执行 `pnpm store prune` 就可以在 store 里面删掉它了。

该命令推荐偶尔进行使用，但不要频繁使用，因为可能某天这个不被引用的包又突然被哪个项目引用了，这样就可以不用再去重新下载这个包了。

node_modules 结构

在 pnpm 官网有一篇很经典的文章，关于介绍 pnpm 项目的 node_modules 结构

在这篇文章中介绍了 pnpm 目前的 node_modules 的一些文件结构，例如在项目中使用 pnpm 安装了一个叫做 `express` 的依赖，那么最后会在 node_modules 中形成这样两个目录结构：

```
1 node_modules/express/...
2 node_modules/.pnpm/express@4.17.1/node_modules/xxx
```

其中第一个路径是 nodejs 正常寻找路径会去找的一个目录，如果去查看这个目录下的内容，会发现里面连个 `node_modules` 文件都没有：

```
1 express
2   lib
3   History.md
4   index.js
5   LICENSE
6   package.json
7   Readme.md
```


实际上这个文件只是个软连接，它会形成一个到第二个目录的一个软连接(类似于软件的快捷方式)，这样 node 在找路径的时候，最终会找到 .pnpm 这个目录下的内容。

其中这个 `.pnpm` 是个虚拟磁盘目录，然后 express 这个依赖的一些依赖会被平铺到 `.pnpm/express@4.17.1/node_modules/` 这个目录下面，这样保证了依赖能够 require 到，同时也不会形成很深的依赖层级。

在保证了 nodejs 能找到依赖路径的基础上，同时也很大程度上保证了依赖能很好的被放在一起。

pnpm 对于不同版本的依赖有着极其严格的区分要求，如果项目中某个依赖实际上依赖的 peerDeps 出现了具体版本上的不同，对于这样的依赖会在虚拟磁盘目录 `.pnpm` 有一个比较严格的区分

综合而言，本质上 pnpm 的 `node_modules` 结构是个网状 + 平铺的目录结构。这种依赖结构主要基于软连接(即 symlink)的方式来完成。

symlink 和 hard link 机制

在前面知道了 pnpm 是通过 hardlink 在全局里面搞个 store 目录来存储 node_modules 依赖里面的 hard link 地址，然后在引用依赖的时候则是通过 symlink 去找到对应虚拟磁盘目录下(.pnpm 目录)的依赖地址。

这两者结合在一起工作之后，假如有一个项目依赖了 `bar@1.0.0` 和 `foo@1.0.0`，那么最后的 node_modules 结构呈现出来的依赖结构可能会是这样的：

```
1  node_modules
2  └─ bar // symlink to .pnpm/bar@1.0.0/node_modules/bar
3  └─ foo // symlink to .pnpm/foo@1.0.0/node_modules/foo
4  └─ .pnpm
5      └─ bar@1.0.0
6          └─ node_modules
7              └─ bar -> <store>/bar
8                  └─ index.js
9                  └─ package.json
10 └─ foo@1.0.0
11     └─ node_modules
12         └─ foo -> <store>/foo
13             └─ index.js
14             └─ package.json
```

`node_modules` 中的 bar 和 foo 两个目录会软连接到 .pnpm 这个目录下的真实依赖中，而这些真实依赖则是通过 hard link 存储到全局的 store 目录中。

兼容问题

读到这里，可能用户会好奇：像 hard link 和 symlink 这种方式在所有的系统上都是兼容的吗？

实际上 hard link 在主流系统上(Unix/Win)使用都是没有问题的，但是 symlink 即软连接的方式可能会在 windows 存在一些兼容的问题，但是针对这个问题，pnpm 也提供了对应的解决方案：

在 win 系统上使用一个叫做 [junctions](#) 的特性来替代软连接，这个方案在 win 上的兼容性要好于 symlink。

或许你也会好奇为啥 pnpm 要使用 hard links 而不是全都用 symlink 来去实现。

实际上存在 store 目录里面的依赖也是可以通过软连接去找到的，nodejs 本身有提供一个叫做 `--preserve-symlinks` 的参数来支持 symlink，但实际上这个参数实际上对于 symlink 的支持并不好导致作者放弃了该方案从而采用 hard links 的方式

Monorepo 支持

`pnpm` 在 monorepo 场景可以说算得上是个完美的解决方案了，因为其本身的设计机制，导致很多关键或者说致命的问题都得到了相当有效的解决。

workspace 支持

对于 monorepo 类型的项目，pnpm 提供了 workspace 来支持

痛点解决

Monorepo 下被人诟病较多的问题，一般是依赖结构问题。常见的两个问题就是 `Phantom dependencies` 和 `NPM doppelgangers`，用 [rush 官网](#) 的图片可以很贴切的展示着两个问题：下面会针对两个问题一一介绍。

Phantom dependencies

Phantom dependencies 被称之为幽灵依赖，解释起来很简单，即某个包没有被安装 (`package.json` 中并没有，但是用户却能够引用到这个包)。

引发这个现象的原因一般是因为 node_modules 结构所导致的，例如使用 yarn 对项目安装依赖，依赖里面有个依赖叫做 foo，foo 这个依赖同时依赖了 bar，yarn 会对安装的 node_modules 做一个扁平化处理(npm v3 之后也是这么做的)，会把依赖在 node_modules 下打平，这样相当于 foo 和 bar 出现在同一层级下面。那么根据 nodejs 的寻径原理，用户能 require 到 foo，同样也能 require 到 bar。

```
1 package.json -> foo(bar 为 foo 依赖)
2 node_modules
3   /foo
4   /bar -> 🤖 依赖
```

那么这里这个 bar 就成了一个幽灵依赖，如果某天某个版本的 foo 依赖不再依赖 bar 或者 foo 的版本发生了变化，那么 require bar 的模块部分就会抛错。

以上其实只是一个简单的例子，但是根据笔者在字节内部见到的一些 monorepo(主要为 `lerna + yarn`)项目中，这其实是个比较常见的现象，甚至有些包会直接去利用这种残缺的引入方式去减轻包体积。

还有一种场景就是在 `lerna + yarn workspace` 的项目里面，因为 `yarn` 中提供了 `hoist` 机制(即一些底层子项目的依赖会被提升到顶层的 `node_modules` 中)，这种 `phantom dependencies` 会更多，一些底层的子项目经常会去 `require` 一些在自己里面没有引入的依赖，而直接去找顶层 `node_modules` 的依赖(`nodejs` 这里的寻径是个递归上下的过程)并使用。

而根据前面提到的 `pnpm` 的 `node_modules` 依赖结构，这种现象是显然不会发生的，因为被打平的依赖会被放到 `.pnpm` 这个虚拟磁盘目录下面去，用户通过 `require` 是根本找不到的。

值得一提的是，`pnpm` 本身其实也提供了将依赖提升并且按照 `yarn` 那种形式组织的 `node_modules` 结构的 Option，作者将其命名为 `--shamefully-hoist`，即 "羞耻的 `hoist`".....

NPM doppelgangers

这个问题其实也可以说是 `hoist` 导致的，这个问题可能会导致有大量的依赖的被重复安装，举个例子：

例如有个 package，下面依赖有 `lib_a`、`lib_b`、`lib_c`、`lib_d`，其中 `a` 和 `b` 依赖 `util_e@1.0.0`，而 `c` 和 `d` 依赖 `util_e@2.0.0`。

那么早期 `npm` 的依赖结构应该是这样的：

- package
 - package.json
 - node_modules
 - lib_a1.0.0
 - lib_b1.0.0
 - _lib_c
 - node_modules <- util_e@2.0.0
 - lib_d2.0.0

这样必然会导致很多依赖被重复安装，于是就有了 `hoist` 和打平依赖的操作：

- package
 - package.json
 - node_modules
 - util_e@1.0.0
 - lib_a
 - lib_b
 - _lib_c
 - node_modules <- util_e@2.0.0
 - lib_d2.0.0

但是这样也只能提升一个依赖，如果两个依赖都提升了会导致冲突，这样同样会导致一些不同版本的依赖被重复安装多次，这里就会导致使用 `npm` 和 `yarn` 的性能损失。

如果是 pnpm 的话，这里因为依赖始终都是存在 store 目录下的 hard links，一份不同的依赖始终都只会被安装一次，因此这个是可以被彻底消除的。

目前不适用的场景

前面有提到关于 pnpm 的主要问题在于 symlink(软链接)在一些场景下会存在兼容的问题。

在里面作者提到了目前 nodejs 软连接不能适用的一些场景，希望 nodejs 能提供一种 link 方式而不是使用软连接，同时也提到了 pnpm 目前因为软连接而不能使用的场景：

- Electron 应用无法使用 pnpm
- 部署在 [lambda](#) 上的应用无法使用 pnpm

笔者在字节内部使用 pnpm 时也遇到过一些 nodejs 基础库不支持 symlink 的情况导致使用 pnpm 无法正常工作，不过这些库在迭代更新之后也会支持这一特性。

104. pnpm 和 npm 的区别？

pnpm 和 npm 是两个不同的 JavaScript 包管理工具，它们有以下区别：

1. 包的存储方式：npm 将每个包都下载到项目的 node_modules 目录中，而 pnpm 会在全局安装一个存储库，并在项目中创建一个符号链接到该存储库中的每个包。
2. 空间占用：由于 pnpm 使用符号链接，它的空间占用通常比 npm 小，因为它避免了在多个项目中重复存储相同的依赖项。
3. 安装速度：由于 pnpm 在全局安装中共享依赖项，因此安装速度通常比 npm 更快。
4. 命令行接口：pnpm 的命令行接口与 npm 不同，但它们都提供了一组相似的命令来管理包。
5. 兼容性：由于 pnpm 的存储方式不同于 npm，因此某些与 npm 兼容的工具可能无法与 pnpm 一起使用。

总的来说，pnpm 与 npm 相比具有更小的空间占用和更快的安装速度，但由于其不同的存储方式可能会导致与某些工具的不兼容。